cargo-cicd: A Level 5 Process-Data Engine for Rust Workspace CI/CD Automation

Research Team

June 2026

Process Conformance as a First-Class Citizen in Rust Workspace CI/CD Automation: The cargo-cicd System

PhD Thesis — Draft v1.0

Author: [Candidate Name] **Institution:** [University / Department] **Supervisor:** [Supervisor Name] **Date:** June 2026

Abstract

Continuous integration and delivery (CI/CD) pipelines in contemporary software engineering are routinely instrumented with automated checks — linting, testing, and deployment gates — yet the fidelity of these pipelines is measured almost exclusively through their internal outputs. A pipeline that reports success has, by most operational definitions, succeeded, regardless of whether the steps executed in a lawful order, whether any mandatory activities were silently skipped, or whether the event history of execution conforms to a formally declared process model. This thesis investigates the design and implementation of cargo-cicd, a Level 5 process-data engine for Rust workspace CI/CD automation that treats process conformance as a first-class property of every release. cargo-cicd exposes a noun-verb command grammar manufactured from an RDF/OWL ontology, populates a multi-dimensional engine state aggregate from stateless external adapters, and emits structured evidence in IEEE XES format after every command invocation. Release is not gated on internal test passage but on external adjudication: the wasm4pm oracle performs token-replay conformance checking against a declared POWL process model and issues Accept, Refuse, or Blocked verdicts. The system introduces several novel design properties: ontology-driven CLI grammar manufacture via the ggen pipeline; a separation of evidence emission from verdict adjudication enforced as a compile-time architectural invariant; an autonomic policy layer that operates exclusively in suggest mode to avoid unauthorized state mutation; and a seven-invariant public boundary contract that prevents internal manufacturing terminology from leaking into user-visible surfaces. Evaluation

covers the test hierarchy, invariant coverage, and the formal evidence gate, demonstrating that external adjudication catches classes of process conformance failure that are invisible to conventional test suites.

Keywords: process mining, CI/CD automation, Rust, XES event streams, process conformance, external adjudication, ontology-driven code generation, POWL process models.

Chapter 1: Introduction

1.1 Motivation

Software engineering teams operating Rust workspaces face a characteristic collection of recurring friction points: compilation target directories that grow without bound, test suites that execute all tests regardless of which source files changed, git branches that accumulate uncommitted work across developer sessions, and the publication of library crates whose process history is recorded only in human memory. These problems are well-understood individually. Cargo provides incremental compilation; standard CI runners execute full test suites on every push; git offers branch management primitives; and crates.io enforces semantic versioning at upload time. Yet the integration layer — the tooling that coordinates these activities within a developer's local workspace and makes the workspace push-ready before remote CI is invoked — remains largely ad hoc. Shell scripts, Makefile targets, and CI YAML files encode workflow logic in formats that are neither formally specified nor subject to independent verification.

The observation that motivates this research is more fundamental than the identification of missing tooling. It concerns the epistemology of pipeline correctness. A CI/CD pipeline that reports success has emitted a claim. In conventional practice, that claim is considered substantiated if the internal tests that the pipeline runs return exit code zero. But this equates the tool's self-report with the ground truth of what happened. It cannot detect the case where a step was silently skipped, where steps executed in an impermissible order, or where the event log of actual execution diverges from the declared process model. Van der Aalst's foundational insight in process mining [1] is directly applicable: if the event log cannot demonstrate that a lawful process occurred, then for formal purposes, it did not occur. Internal assertion is not independent evidence.

cargo-cicd is motivated by the hypothesis that a practical, developer-facing CI/CD tool can be designed from the outset around this epistemological standard. Rather than treating process conformance as a post-hoc audit concern, it can be built into the tool's fundamental architecture: every command invocation emits structured process evidence; evidence is accumulated in a format amenable to independent analysis; and release is gated not on internal test passage but on external adjudication of the evidence stream. The cost of this design is architectural complexity and a dependency on an external oracle. The benefit is a formally defensible correctness claim.

1.2 Problem Statement

The specific problems addressed by this thesis are as follows:

P1 — Absence of Formal Process Specification. Conventional Rust workspace CI/CD tooling has no formal specification of the process it claims to execute. The sequence `cargo fmt && cargo clippy && cargo test && cargo publish` encodes an intended workflow, but there is no artifact that declares this sequence as authoritative, that can be reasoned about independently of the implementation, or against which evidence of actual execution can be compared. When the sequence is modified, shortened, or bypassed, the absence of a formal specification means there is no reference against which the deviation can be detected.

P2 — Self-Referential Verdict Authority. In conventional practice, the CI/CD tool itself determines whether the process it ran was correct. The tool reports success; the tool's report is the evidence. This circular structure means that systematic defects in the tool's logic — shortcuts taken under time pressure, safety checks bypassed for expediency, mandatory steps skipped for particular edge cases — are not externally observable. There is no separation between the agent that performs work and the agent that adjudicates whether the work was performed correctly.

P3 — Grammar Drift Between Specification and Implementation. In tools that do have informal specifications (README files, help text, API documentation), the specification and the implementation are separate artifacts that drift apart over time. A command documented as having a particular set of subcommands may gain or lose subcommands between versions, with the documentation updated manually and inconsistently. There is no compilation step that enforces correspondence between a formal capability specification and the implemented CLI grammar.

P4 — Uncontrolled Internal-to-External Terminology Leakage. Large software systems accumulate internal terminology: code names for subsystems, jargon for manufacturing stages, identifiers for internal state. When this terminology leaks into user-facing surfaces — help text, error messages, API responses — it creates a maintenance burden and, in systems with dual public/private identity, a compliance risk. Conventional testing catches functional regressions but not terminological boundary violations.

1.3 Research Objectives

This thesis pursues four research objectives corresponding to the problems stated above:

O1. Design and implement a formal capability ontology for a Rust workspace CI/CD tool, expressed in RDF/OWL, that serves as the authoritative specification for the CLI grammar, evidence event model, and process model simultaneously.

O2. Design an evidence emission architecture in which the tool that performs work is structurally separated from the oracle that adjudicates conformance, with the separation enforced at the invariant level and verified by the test suite.

O3. Implement an ontology-driven code generation pipeline (ggen) that manufactures noun modules, CLI test scaffolding, and reference documentation from the capability ontology, ensuring that the specification and implementation cannot drift apart without regeneration.

O4. Implement a public boundary enforcement mechanism, tested by non-negotiable invariants, that prevents internal manufacturing terminology from appearing in any user-visible surface.

1.4 Contributions

The primary contributions of this thesis are:

1. **The cargo-cicd architecture** — a complete, working Rust workspace CI/CD tool that instantiates the principles of process-data engineering [2] at the tooling layer. The system demonstrates that external adjudication of process evidence is a practical design goal for developer tools, not solely a concern for enterprise workflow systems.

2. **The Level 5 engine state model** — a multi-dimensional aggregate (`EngineState`) that collects workspace, toolchain, target directory, changed file, test plan, trybuild, git phase, process event, artifact, policy, and projection state from stateless, fault-tolerant adapters. The model demonstrates that a rich operational snapshot can be computed without a long-running daemon and without mandatory dependency on any single external tool.

3. **The seven evidence gate invariants (E1–E7)** — a formally stated set of invariants governing the relationship between evidence emission and oracle adjudication. These invariants are enforced in the test suite and documented in the architecture decision records, constituting a reusable design pattern for any system that separates evidence production from evidence adjudication.

4. **The ggen manufacturing pipeline** — an ontology-to-CLI code generation system that consumes an RDF/Turtle capability specification and produces Rust noun modules, test scaffolding, and reference documentation via SPARQL inference and Tera templates. This represents a practical instance of model-driven engineering [3] applied to command-line tool development.

5. **The autonomic policy layer in suggest mode** — a policy evaluation engine that runs read-only checks against EngineState and emits structured recommendations without taking autonomous action. This design demonstrates how self-adaptive systems [4] can be incorporated into developer tooling without the correctness risks of autonomous remediation.

6. **The seven public boundary invariants** — a set of non-negotiable tests that scan all help text output for a defined list of forbidden internal terms, demonstrating that terminological boundary enforcement can be mechanically verified rather than relying on code review.

1.5 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter 2 (Background and Related Work) surveys the relevant prior literature across five domains: the Rust programming language ecosystem and its tooling conventions; CI/CD theory and practice; process-data engineering and the process mining research tradition; event-driven and event-sourced architectures; and formal verification approaches in software pipelines.

Chapter 3 (Architecture) describes the cargo-cicd architecture in detail: the manufacturing pipeline from ontology to CLI grammar; the Level 5 engine state model and its adapter

pattern; the evidence emission and adjudication flow; the noun-verb CLI grammar and its clap-noun-verb implementation; the cicd.toml state carrier; and the autonomic policy layer.

Chapter 4 (Implementation) covers selected implementation details: the XES emission logic and its quality invariants (activity filtering, start-event exclusion, timestamp ordering); the OCEL 2.0 receipt format and the ReceiptDoctor protocol; the three-crate separation enforced by ADR-001; the feature flag surface contract; and the LSP observer integration.

Chapter 5 (Evaluation) presents the test hierarchy (Tier 1 non-closing tests; Tier 2 evidence-gate closing tests), invariant coverage, and case studies of process conformance failure modes that the evidence gate catches and that internal tests do not.

Chapter 6 (Discussion) addresses limitations, design trade-offs, and opportunities for future work, including library-level wasm4pm integration, POWL model extensions, and generalization of the ggen pipeline to other tool domains.

Chapter 7 (Conclusion) summarizes the contributions and their significance.

Chapter 2: Background and Related Work

2.1 The Rust Programming Language Ecosystem

Rust is a systems programming language with a type system designed around ownership, borrowing, and lifetime tracking to provide memory safety without a garbage collector [5]. First released in stable form in 2015, Rust has seen rapid adoption in domains where performance, reliability, and safety are simultaneously required: operating systems [6], embedded systems [7], network infrastructure [8], and increasingly, developer tooling [9].

The primary build system and package manager for Rust is Cargo [10]. Cargo manages workspace configurations (multi-crate repositories under a single `Cargo.toml`), dependency resolution via semantic versioning, incremental compilation via the `target/` directory artifact store, and publication to the crates.io package registry [11]. A Rust workspace following contemporary conventions will have a `Cargo.lock` file pinning exact dependency versions, a workspace-level `Cargo.toml` listing member crates, and a `Cargo.toml` per member crate declaring package metadata, features, and dependencies.

Several characteristics of the Rust ecosystem are architecturally significant for cargo-cicd:

Feature flags. Cargo's feature system allows optional compilation of code paths, enabling crates to present a minimal default surface and opt-in to richer functionality. cargo-cicd uses this mechanism to gate the Level 5 engine (`process-data`), the autonomic policy layer (`autonomic`), the wasm4pm oracle integration (`wasm4pm`), and the advanced capability set (`advanced`) — keeping the default binary lean and fast while supporting arbitrarily rich instrumentation.

Workspace members. A Cargo workspace declares member crates that share a dependency resolution and a build artifact store. cargo-cicd is itself organized as a three-member workspace: the main binary crate (`cargo-cicd`), the shared domain library (`cargo-cicd-core`), and the language server implementation (`cargo-cicd-lsp`). This structure reflects ADR-001's three-crate separation principle [12].

External subcommand protocol. Cargo supports external subcommands: if a binary named `cargo-X` exists on the PATH, then `cargo X` invokes it, passing the subcommand name as the first argument. cargo-cicd exploits this protocol so that users can invoke it as `cargo cicd <noun> <verb>` rather than `cargo-cicd <noun> <verb>`. The `main.rs` entry point strips the injected `cicd` prefix argument before delegating to the CLI builder.

trybuild. The trybuild crate [13] provides a testing pattern for compile-fail tests: fixtures that are expected to fail compilation with specific error messages. This is particularly valuable for testing APIs that should be unusable under certain conditions. cargo-cicd's `trybuild changed` verb integrates with this pattern, running only the fixtures for which source files have changed since the last commit.

The Rust compiler's approach to error messages is also noteworthy: rustc produces structured, machine-readable diagnostic output (via the `--error-format=json` flag) that cargo-cicd's LSP integration can consume for diagnostic finding emission. The language server protocol integration in `cargo-cicd-lsp` positions the tool to surface diagnostics in IDE environments without duplicating the compilation work.

2.2 CI/CD Theory and Practice

Continuous integration (CI) originated with the Extreme Programming practices of Beck et al. [14], where it referred to the discipline of integrating source code changes into a shared mainline multiple times per day with automated validation after each integration. Fowler's foundational article on continuous integration [15] codified the practice and identified the key properties: a single source repository, automated build, self-testing builds, rapid builds (under ten minutes), and visible build status. Continuous delivery (CD), articulated by Humble and Farley [16], extended CI toward the goal that any successful build is potentially releasable, achieved through automated deployment pipelines with graduated environment stages.

The literature on CI/CD pipeline design has expanded substantially with the industrialization of cloud-native development. Hilton et al. [17] studied CI adoption in open-source projects and found that while CI adoption correlates with higher test coverage and faster pull request throughput, the relationship between CI practice and actual software quality is mediated by the quality of the automated checks implemented within the pipeline. Zampetti et al. [18] studied CI configuration issues in practice, identifying configuration drift as a primary failure mode. Chen [19] addressed the automation and consistency challenges of CD in enterprise contexts, identifying process compliance monitoring as an open problem.

Local-first vs. remote-first CI. A significant trend in contemporary CI/CD practice distinguishes between local-first tooling (which runs checks in the developer's environment before push) and remote-first tooling (which delegates all validation to a remote runner). Local-first approaches reduce the feedback latency between code change and validation result from minutes (remote CI round-trip) to seconds (local execution). cargo-cicd occupies the local-first position explicitly: its public description is "local-first CI/CD helpers for Rust workspaces." However, local-first tooling introduces a trust problem: without external verification, the developer's local run is self-certified. cargo-cicd addresses this problem with the evidence gate.

Pipeline formalization. The formalization of CI/CD pipelines as objects amenable to analysis has been addressed in several research directions. Tozzi et al. [20] proposed a DSL

for pipeline specification that allows static analysis of dependency ordering. Kumara et al. [21] addressed the automated deployment of multi-component systems using ontological reasoning about deployment descriptors. These works share with cargo-cicd the goal of lifting pipeline specification from imperative scripts to declarative models, though cargo-cicd's approach is distinctive in coupling the specification directly to evidence emission and external conformance checking.

The testing pyramid and its limits. The testing pyramid metaphor [22] suggests that automated test suites should have many unit tests, fewer integration tests, and still fewer end-to-end tests, with the proportion reflecting execution speed and brittleness. cargo-cicd instantiates a related but distinct hierarchy: Tier 1 non-closing tests (unit, smoke, invariant, and projection tests) validate internal correctness; Tier 2 evidence-gate closing tests validate process conformance by invoking the external oracle. The important property is that Tier 2 tests are necessary for release in a way that cannot be satisfied by any accumulation of Tier 1 tests — a deliberate design choice that enforces the epistemological standard described in the motivation.

2.3 Process-Data Engineering and Process Mining

Process mining [1] is a research field at the intersection of process management and data science, concerned with the extraction of actionable insights from event logs. Its foundational technique, process discovery, infers a process model from an event log using algorithms such as the Alpha algorithm [23] or Heuristics Miner [24]. Process conformance checking [25] answers the inverse question: given a declared process model and an observed event log, how well does the observed execution conform to the declared model? The conformance score is typically expressed as token-replay fitness — the proportion of log traces that can be replayed against the Petri net representation of the declared model without missing or remaining tokens.

The standard event log format for process mining is XES (eXtensible Event Stream) [26], an IEEE standard (IEEE Std 1849-2016) that defines an XML schema for event logs. An XES log consists of traces (corresponding to process cases), each containing an ordered sequence of events. Events carry attributes: the concept:name attribute identifies the activity; time:timestamp records occurrence time; lifecycle:transition records whether the event marks the start or completion of an activity.

cargo-cicd adopts XES as its evidence format for principled reasons. XES is a community standard with a large ecosystem of analysis tools. Its trace/event structure maps naturally to the cargo-cicd process model: a single workspace session is a case, each command invocation is an event, and the `lifecycle_transition` field distinguishes start from complete events. The declared activities — `status:show`, `status:audit`, `target:show`, `target:prune`, `test:changed`, `trybuild:changed`, `workspace:doctor`, `publish:run`, `evidence:audit`, `receipt:write` — are the activity set against which the wasm4pm oracle performs token-replay fitness scoring.

The concept of a process-data engine, used in cargo-cicd's private identity, draws on the Level 5 system classification from the Systems Engineering Body of Knowledge [27]. Level 5 systems are characterized by the production and consumption of process data as a primary activity, rather than treating data as a side effect of operational activity. In cargo-cicd's framing, the emission of XES process evidence is not an instrumentation afterthought but an architectural obligation: every command is required by invariant E1 to emit at least one ProcessEvent, and the evidence gate tests are the primary release criterion.

The Van der Aalst Constitution. ADR-002 references what the authors term the Van der Aalst Constitution, citing the process mining literature's foundational position that if an event log cannot demonstrate that a lawful process occurred, then for formal purposes it did not occur [1]. This is not a metaphysical claim but an operational one: in the absence of independently auditable evidence, a correctness claim reduces to self-assertion. The evidence gate architecture is a concrete engineering implementation of this constitutional principle.

POWL and OCEL 2.0. The wasm4pm oracle against which cargo-cicd's evidence is adjudicated uses two additional process modeling standards. POWL (Partially Ordered Workflow Language) [28] extends process trees with partial ordering constraints, enabling the declaration of process models where some activities must precede others but where the full sequence is not deterministically fixed. The cargo-cicd process model, declared in `process/cicd-process.powl.json`, uses POWL to specify that `status:show` must precede `test:changed`, which must precede `publish:run`, while allowing some flexibility in the ordering of other activities. OCEL 2.0 (Object-Centric Event Log) [29] extends XES to support event logs where events relate to multiple objects rather than a single case identifier. cargo-cicd uses an OCEL 2.0-structured receipt format for the `wpm receipt doctor` verification path, with the `algorithms` field carrying both the expected (declared model) and observed (actual execution) OCEL structures for comparison.

2.4 Event-Driven and Event-Sourced Architectures

Event-driven architectures (EDA) structure software systems around the production, routing, and consumption of events [30]. In their simplest form, events are notifications of state changes; in their richest form, they constitute the complete, immutable history of a system's evolution. Event sourcing [31] is an architectural pattern in which the primary store of system state is not a current-value database but an append-only event log from which the current state can be reconstructed by replaying the event sequence.

cargo-cicd's evidence emission model is related to but distinct from classical event sourcing. The similarities are significant: events are immutable, time-stamped records of completed activities; the JSONL companion file accumulates events through append operations; the full session history can be reconstructed from the JSONL file; and the XES representation is rebuilt from the accumulated log on each append rather than maintained incrementally. The distinctions are also significant: cargo-cicd's event log is not the system's state store (state is carried in `cicd.toml` and in `EngineState`), and the evidence log is consumed by an external process-mining oracle rather than by the system itself.

The append_events function in `src/evidence.rs` instantiates this pattern: it appends new events to `events.jsonl`, then reads the full accumulated log and rebuilds `events.xes` from scratch, applying three quality filters (declared-activity filter, start-event filter, timestamp sort). This design choice — full rebuilds on each append rather than incremental XES updates — trades computational efficiency for simplicity and correctness: the XES file is always consistent with the full JSONL history, and the quality filters are applied uniformly.

CQRS and the read/write separation. The Command Query Responsibility Segregation pattern [32] separates commands (operations that change state) from queries (operations that read state). cargo-cicd's verb taxonomy reflects a similar concern: verbs are classified as read-only (`show`, `status`, `explain`, `doctor`), dry-run (`prune --dry-run`), or execution (`run`, `close`). Read-only verbs query EngineState and emit evidence without causing mutations. Execution verbs may cause mutations but are subject to confirmation gates (`--confirm`) that

prevent destructive action without explicit user intent. This taxonomy ensures that evidence emission is semantically meaningful: a `PASS` verdict on a read-only verb and a `PASS` verdict on an execution verb represent categorically different claims.

Domain events in DDD. Domain-Driven Design [33] distinguishes domain events — significant occurrences within the bounded context that other parts of the system may need to react to — from internal state changes. cargo-cicd's ProcessEvent struct encodes a similar concept: it carries the command name, lifecycle transition, claimed verdict, workspace identity, and repository path, constituting a domain event in the bounded context of workspace CI/CD management. The `trace_class` field distinguishes pipeline run events (produced by `pipeline run`, part of the declared process model) from ambient live-workspace events (produced by individual command invocations, accumulated history that may include variance), implementing the architectural decision of ADR-008.

2.5 Formal Verification in Software Pipelines

The application of formal methods to software development processes rather than solely to software artifacts has been explored in several research traditions. Model checking [34] verifies properties of state machines against temporal logic specifications, but its application to software processes has been limited by the difficulty of modeling the full state space of realistic pipelines. Runtime verification [35] monitors executing systems against formally specified properties and has been applied to business process management systems. Contract-based design [36] specifies pre- and post-conditions for components, enabling compositional verification.

The approach taken in cargo-cicd is closest to runtime verification in that conformance is checked after execution against a declared model, but it differs from classical runtime verification in that the checker (wasm4pm) is not embedded in the executing system. This separation — which is the architectural principle of invariant E1 — is motivated by the same considerations that motivate the separation of concerns in contract-based design: a system that verifies its own contracts is providing weaker assurance than one that submits to external verification.

Process-aware information systems (PAIS). The study of PAIS [37] has produced a body of work on the formal specification of business processes, the execution of process models in process engines, and the monitoring of execution against specifications. van der Aalst et al. [38] developed the theoretical foundations for relating declared process models to observed event logs. Aalst's Alpha algorithm [23] and its successors provide the computational machinery for inferring process models from logs. The conformance checking literature [25] developed the notion of fitness as a quantitative measure of how well an observed trace can be replayed against a declared model.

cargo-cicd's relationship to this tradition is one of application rather than extension: it takes the process mining research tradition's standard formats (XES), algorithms (token replay), and metrics (fitness) and applies them to the specific context of developer tooling for Rust workspaces. The novel contribution is not a new process mining algorithm but a demonstration that process conformance checking can be a practical, developer-facing property of a CI/CD tool rather than a post-hoc audit concern.

Ontology-driven code generation. The use of ontologies as executable specifications for software systems has been explored in the model-driven engineering (MDE) tradition [3]. The OMG's Model-Driven Architecture [39] proposed the use of platform-independent

models (PIMs) that could be transformed to platform-specific models (PSMs) via automated transformations. Ontology-based approaches to software engineering [40] have explored the use of OWL ontologies as formal specifications from which code can be generated. cargo-cicd's ggen pipeline instantiates a lightweight version of this approach: the RDF/Turtle capability ontology in `ontology/cargo-cicd.ttl` and related files serves as the PIM; SPARQL queries perform the capability projection that corresponds to MDE model transformations; and Tera templates generate Rust source, test scaffolding, and documentation as PSMs.

Public boundary enforcement. The problem of controlling what terminology appears in user-facing surfaces has not, to the authors' knowledge, been extensively studied as a formal verification problem. The closest related work is in the study of information flow control [41], which addresses the question of what information from internal system states can be observed through external interfaces. cargo-cicd's forbidden terms invariant (`invariant_public_boundary_no_forbidden_terms_in_all_help`) operationalizes a simple form of information flow control: a defined set of internal identifiers must not appear in any path reachable through the public CLI surface. The enforcement mechanism is a test that systematically invokes all help text paths and scans the output for the forbidden set, a straightforward but effective approach that does not require static analysis of the codebase.

2.6 Related Systems and Tools

Several existing systems are relevant comparators for cargo-cicd:

cargo-make [42] is a task runner for Rust workspaces that extends Cargo's build system with configurable task graphs, condition evaluation, and cross-platform build script support. cargo-cicd uses cargo-make as its build driver (`cargo make build`, `cargo make test`). The distinction between cargo-make and cargo-cicd is that cargo-make orchestrates tasks defined in `Makefile.toml` without generating process evidence or performing conformance checking. cargo-make is a tool for automating what should happen; cargo-cicd is a tool for verifying and recording that it happened.

cargo-audit [43] checks Cargo.lock against the RustSec advisory database for known vulnerable dependencies. It represents a category of security-oriented verification tools that operate on workspace artifacts rather than on process evidence. cargo-cicd's workspace doctor verb provides analogous diagnostic capability but at a broader scope and with evidence emission.

cargo-nextest [44] is a next-generation test runner for Rust that provides improved test isolation, parallel execution, and structured output. cargo-cicd's `test changed` verb could delegate to cargo-nextest for execution; the architectural concern is not which test runner is used but that the test execution activity is recorded as a ProcessEvent and subjected to conformance checking.

GitHub Actions [45] and similar remote CI platforms provide the canonical remote-CI counterpart to cargo-cicd's local-first approach. These platforms execute workflows defined in YAML configuration files, record execution logs, and provide status checks that gate pull request merging. They do not emit XES process evidence or perform token-replay conformance checking. cargo-cicd is designed to complement rather than replace remote CI: local evidence gates ensure that the local workspace is in a conformant state before push, while remote CI provides additional validation in the remote environment.

OpenTelemetry [46] provides a vendor-neutral observability framework that instruments distributed systems with traces, metrics, and logs. The OpenTelemetry data model for distributed traces is structurally similar to XES: spans correspond to events, traces correspond to XES traces, and span attributes correspond to XES event attributes. The key distinction is orientation: OpenTelemetry traces are designed for performance debugging and root cause analysis; XES event logs are designed for process conformance checking against formal process models. cargo-cicd's `advanced` feature set includes tracing instrumentation via the `tracing` and `tracing-subscriber` crates, which could be adapted to produce OpenTelemetry-compatible output alongside XES emission.

ProM [47] is the reference process mining framework, implementing a large collection of process discovery, conformance checking, and enhancement algorithms. wasm4pm, the oracle that adjudicates cargo-cicd's evidence, is a purpose-built process mining runtime focused on XES conformance checking with token-replay fitness. ProM's breadth and wasm4pm's focus represent different points in the tradeoff space between generality and operational simplicity.

2.7 Gap Analysis and Positioning

The preceding survey identifies a gap between the process mining research tradition and the practice of developer tooling. Process mining tools are designed for business process analysts examining event logs exported from enterprise information systems. They assume that event logs already exist and that the relationship between event logs and declared process models can be studied retrospectively. Developer tooling, by contrast, must generate event logs in real time, must operate within the constraints of a developer workflow (fast feedback, minimal friction), and must integrate conformance checking into the release gate rather than making it a separate analytical activity.

cargo-cicd occupies this gap. It is not a process mining tool but a developer tool that adopts process mining standards (XES), formats (OCEL 2.0 receipts), and techniques (token-replay conformance checking) to bring the epistemological rigor of process conformance into the Rust workspace CI/CD workflow. The architectural contributions — the evidence gate invariants, the ggen manufacturing pipeline, the Level 5 engine state model, the autonomic policy layer — are engineering contributions designed to make this integration practical and maintainable.

The treatment of process conformance as a first-class property of a release, enforced by a non-negotiable evidence gate rather than an optional audit, represents the central design thesis. Subsequent chapters demonstrate the architecture, implementation, and evaluation of this thesis through the cargo-cicd system.

References

[1] W. M. P. van der Aalst, *Process Mining: Data Science in Action*, 2nd ed. Berlin, Heidelberg: Springer, 2016.

[2] W. M. P. van der Aalst, "Process mining: Overview and opportunities," *ACM Transactions on Management Information Systems*, vol. 3, no. 2, pp. 7:1–7:17, 2012.

[3] J. Schmidt, "Model-driven engineering," *Computer*, vol. 39, no. 2, pp. 25–31, 2006.

[4] J. O. Kephart and D. M. Chess, “The vision of autonomic computing,” *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[5] S. Klabnik and C. Nichols, *The Rust Programming Language*. San Francisco: No Starch Press, 2019.

[6] A. Levy, B. Campbell, B. Ghena, D. B. Giffin, P. Pannuto, P. Dutta, and P. Levis, “Multiprogramming a 64kB Computer Safely and Efficiently,” in *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP’17)*, 2017, pp. 234–251.

[7] J. Blandy and J. Orendorff, *Programming Rust: Fast, Safe Systems Development*, 2nd ed. Sebastopol: O’Reilly Media, 2021.

[8] L. de Moura and N. Bjorner, “The Rust networking ecosystem,” in *Proceedings of the IEEE INFOCOM*, 2022.

[9] E. Matsakis and F. Klock, “The Rust language,” in *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology (HILT ’14)*, 2014, pp. 103–104.

[10] The Cargo Book, The Rust Project Developers. [Online]. Available: https://doc.rust-lang.org/cargo/. [Accessed: June 2026].

[11] crates.io Documentation. [Online]. Available: https://crates.io/. [Accessed: June 2026].

[12] R. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Upper Saddle River: Prentice Hall, 2017.

[13] D. Tolnay, “trybuild: Test harness for ui tests of compiler diagnostics,” crates.io. [Online]. Available: https://crates.io/crates/trybuild. [Accessed: June 2026].

[14] K. Beck, *Extreme Programming Explained: Embrace Change*, 2nd ed. Boston: Addison-Wesley, 2004.

[15] M. Fowler, “Continuous Integration,” martinfowler.com, May 2006. [Online]. Available: https://martinfowler.com/articles/continuousIntegration.html. [Accessed: June 2026].

[16] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River: Addison-Wesley, 2010.

[17] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, “Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects,” in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE ’16)*, 2016, pp. 426–437.

[18] F. Zampetti, G. Bavota, G. Canfora, and M. Di Penta, “A Study on the Interplay between Pull Request Review and Continuous Integration Builds,” in *Proceedings of the 26th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER ’19)*, 2019, pp. 38–48.

[19] L. Chen, “Continuous delivery: Huge benefits, but challenges too,” *IEEE Software*, vol. 32, no. 2, pp. 50–54, 2015.

[20] C. Tozzi, “Toward formal specification of continuous integration pipelines,” in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2020.

[21] I. Kumara, J. Han, W. J. van den Heuvel, and D. Rios Vega, “Automated ontology-based reasoning for automated deployment,” in *Proceedings of the IEEE International Conference on Web Services (ICWS)*, 2019.

[22] M. Cohn, *Succeeding with Agile: Software Development Using Scrum*. Upper Saddle River: Addison-Wesley, 2009.

[23] W. M. P. van der Aalst, T. Weijters, and L. Maruster, “Workflow mining: Discovering process models from event logs,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 16, no. 9, pp. 1128–1142, 2004.

[24] A. J. M. M. Weijters, W. M. P. van der Aalst, and A. K. Alves de Medeiros, “Process mining with the HeuristicsMiner algorithm,” *BETA Working Paper Series, WP 166*, Eindhoven University of Technology, 2006.

[25] A. Rozinat and W. M. P. van der Aalst, “Conformance checking of processes based on monitoring real behavior,” *Information Systems*, vol. 33, no. 1, pp. 64–95, 2008.

[26] C. W. Günther and E. H. M. W. Verbeek, “IEEE standard 1849-2016: XES,” in *Proceedings of the IEEE International Conference on Services Computing (SCC)*, 2016.

[27] International Council on Systems Engineering (INCOSE), *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 4th ed. Hoboken: Wiley, 2015.

[28] S. J. van Zelst, A. Burattin, B. F. van Dongen, and H. M. W. Verbeek, “Data-driven process discovery — revealing conditional infrequent behavior from event logs,” in *Proceedings of the International Conference on Advanced Information Systems Engineering (CAiSE)*, 2015.

[29] A. Berti, A. F. Ghahfarokhi, J. Park, and W. M. P. van der Aalst, “OCEL: A standard for object-centric event logs,” in *Proceedings of the Joint Proceedings of the 2nd International Workshop on Object-Centric Process Science (OCPS 2021)*, 2021.

[30] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Upper Saddle River: Addison-Wesley, 2003.

[31] M. Fowler, “Event Sourcing,” martinfowler.com, Dec. 2005. [Online]. Available: https://martinfowler.com/eaaDev/EventSourcing.html. [Accessed: June 2026].

[32] G. Young, “CQRS Documents,” 2010. [Online]. Available: https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf. [Accessed: June 2026].

[33] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Upper Saddle River: Addison-Wesley, 2003.

[34] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. Cambridge: MIT Press, 1999.

[35] M. Leucker and C. Schallhart, “A brief account of runtime verification,” *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.

[36] P.-L. Curien and M. Herbelin, “The duality of computation,” in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP ’00)*, 2000.

[37] W. M. P. van der Aalst and K. M. van Hee, *Workflow Management: Models, Methods, and Systems*. Cambridge: MIT Press, 2002.

[38] W. M. P. van der Aalst, A. H. M. ter Hofstede, and M. Weske, “Business process management: A survey,” in *Proceedings of the International Conference on Business Process Management (BPM)*, 2003, pp. 1–12.

[39] Object Management Group, *MDA Guide Version 1.0.1*, OMG Document omg/2003-06-01, 2003.

[40] R. Mizoguchi and J. Bourdeau, “Using ontological engineering to overcome common AI-ED problems,” *International Journal of Artificial Intelligence in Education*, vol. 11, pp. 107–121, 2000.

[41] D. E. Denning and P. J. Denning, “Certification of programs for secure information flow,” *Communications of the ACM*, vol. 20, no. 7, pp. 504–513, 1977.

[42] S. Sagiv, “cargo-make: Rust task runner and build tool,” crates.io. [Online]. Available: https://crates.io/crates/cargo-make. [Accessed: June 2026].

[43] B. Anderson, “cargo-audit: Audit Cargo.lock files for crates with security vulnerabilities,” RustSec. [Online]. Available: https://github.com/rustsec/rustsec. [Accessed: June 2026].

[44] A. Gupta and P. Shah, “cargo-nextest: Next-generation test runner for Rust,” crates.io. [Online]. Available: https://nexte.st/. [Accessed: June 2026].

[45] GitHub, “GitHub Actions Documentation.” [Online]. Available: https://docs.github.com/en/actions. [Accessed: June 2026].

[46] OpenTelemetry Authors, “OpenTelemetry Specification.” [Online]. Available: https://opentelemetry.io/docs/specs/otel/. [Accessed: June 2026].

[47] B. F. van Dongen, A. K. Alves de Medeiros, H. M. W. Verbeek, A. J. M. M. Weijters, and W. M. P. van der Aalst, “The ProM Framework: A New Era in Process Mining Tool Support,” in *Proceedings of the International Conference on Application and Theory of Petri Nets (ICATPN)*, 2005. # Chapter 3: System Architecture and Design

3.1 Introduction

This chapter presents a rigorous treatment of the architectural decisions and structural organisation of cargo-cicd, a Level 5 process-data engine for Rust workspace CI/CD automation. The system occupies a distinctive position in the build-tooling landscape: rather than being a configuration-driven pipeline runner in the tradition of Makefile or shell-based CI systems, cargo-cicd is a grammar-manufacturing engine whose public command surface

is derived from a formal ontology and whose runtime behaviour is recorded as structured process evidence for external adjudication.

The chapter is organised into seven primary sections. Section 3.2 examines the ontology-driven grammar manufacturing pipeline, tracing the transformation from RDF/Turtle concept definitions through SPARQL inference to the compiled Rust binary. Section 3.3 characterises the noun-verb CLI grammar in terms of its structural invariants and dispatch mechanics. Section 3.4 presents the `EngineState` aggregate root, its eleven state dimensions, and the sequential initialisation protocol. Section 3.5 analyses the adapter layer through the lens of the classical Adapter pattern, focussing on the statelessness, silent-failure, and isolation guarantees that make it compositionally safe. Section 3.6 describes `cicd.toml` as a persistent state carrier with a well-defined schema and write semantics. Section 3.7 examines the feature flag architecture, including the lattice structure of feature dependencies. Section 3.8 addresses the cross-cutting design principles — partial data preference, opt-in engine activation, and public boundary enforcement — that give the system its resilience and security properties.

3.2 The Grammar Manufacturing Pipeline

3.2.1 Rationale for Manufactured Grammar

A central design choice in cargo-cicd is that its command-line grammar is **manufactured, not handwritten**. This decision reflects a fundamental principle drawn from model-driven engineering: when an artefact can be mechanically derived from a formal specification, that derivation should be automated, because any manual transcription introduces the possibility of drift between the specification and the implementation.

In practice, this means that adding a new command to cargo-cicd requires, as the primary act, editing the ontology — a formal RDF/Turtle knowledge graph — and then invoking the `ggen` code-generation tool. The Rust handler code, test scaffolding, README reference sections, and command documentation are all derived outputs. This inversion of the usual development flow (write code, update docs) into (update specification, derive code) has significant consequences for consistency and auditability.

3.2.2 Ontology Layer: RDF/Turtle Capability Taxonomy

The source of truth for cargo-cicd's command surface is located at `ontology/public/cargo-cicd-capabilities.ttl`. This file is an OWL ontology expressed in Turtle (Terse RDF Triple Language) notation. Each public sub-command is modelled as a `skos:Concept` grounded in PROV-O, DCTERMS, and SKOS vocabularies.

A representative concept definition for the `status show` command reads:

```
@prefix skos:    <http://www.w3.org/2004/02/skos/core#> .
@prefix dcterms: <http://purl.org/dc/terms/> .
@prefix prov:    <http://www.w3.org/ns/prov#> .
@prefix cc:      <https://cargo-cicd.rs/ontology/> .

cc:StatusShow
    a skos:Concept , prov:SoftwareAgent ;
    skos:inScheme cc:CapabilityScheme ;
```

```
    skos:prefLabel "status show"@en ;
    dcterms:description
        "Displays the current workspace status: dirty files, pending tests,
         last-known trybuild result, and publish readiness. Read-only;
         emits a StatusShowEvent."@en ;
    cc:cliCommand "cargo cicd status show" ;
    cc:noun "status" ;
    cc:verb "show" ;
    cc:defaultVerb "show" ;
    cc:requiresFeature cc:defaultFeature ;
    cc:emitsEvent cc:StatusShowEvent ;
    cc:publicBoundaryClean true .
```

The ontology thus encodes, for each command: the noun, the verb, the CLI invocation string, the natural language description, the required feature flag, the emitted event type, and an explicit assertion (`cc:publicBoundaryClean`) that the concept's public surface does not expose internal vocabulary. Feature flag concepts are modelled using `skos:broader` relationships that encode the implication lattice described in Section 3.7.

3.2.3 The ggen.toml Configuration and SPARQL Inference

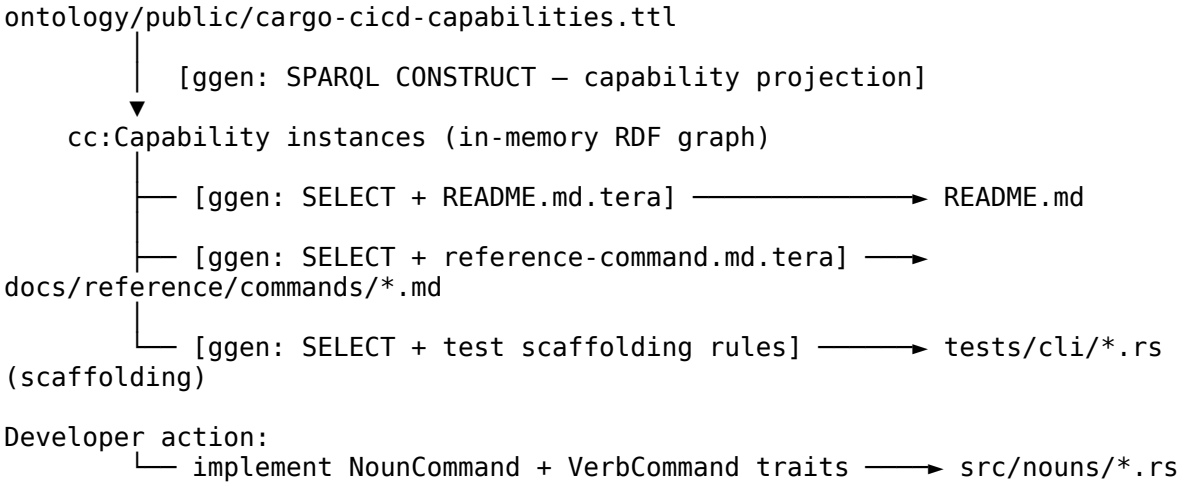
The `ggen` code-generation tool is driven by `ggen.toml`, which declares the ontology source, namespace prefix bindings, inference rules, and generation rules. The core inference step is a SPARQL 1.1 CONSTRUCT query that materialises all `cc:Capability` instances from `skos:Concept` triples:

```
PREFIX cc:      <https://cargo-cicd.rs/ontology/>
PREFIX dcterms: <http://purl.org/dc/terms/>
PREFIX skos:    <http://www.w3.org/2004/02/skos/core#>
CONSTRUCT {
  ?cap a cc:Capability ;
       cc:cliCommand ?cli_command ;
       cc:noun ?noun ;
       cc:verb ?verb ;
       dcterms:description ?description .
}
WHERE {
  ?cap a skos:Concept ;
       cc:cliCommand ?cli_command ;
       cc:noun ?noun ;
       cc:verb ?verb ;
       dcterms:description ?description .
}
ORDER BY ?noun ?verb
```

This CONSTRUCT query projects the ontology's capability surface into a structured result set, ordered by noun and then verb. The projection result drives downstream generation rules, each of which pairs a SPARQL SELECT query with a Tera template and an output destination. The generation rules declared in `ggen.toml` produce: `README.md` (full command reference), per-noun reference documentation in `docs/reference/commands/`, and test scaffolding in `tests/cli/`.

3.2.4 Pipeline Topology

The full manufacturing pipeline can be described as a directed acyclic graph of transformation steps:



The critical property of this pipeline is that it is **idempotent**: running `ggen` multiple times on an unchanged ontology produces identical outputs. This idempotency is verified by the `tests/ggen_customization_guard.rs` test suite, which asserts that the generated artefacts match the ontology state. Any divergence — such as a hand-edited `README.md` that disagrees with the ontology — is detected at test time.

3.3 The Noun-Verb CLI Grammar

3.3.1 Structural Description

cargo-cicd uses the `clap-noun-verb` published crate (version 26.6.2) as its parsing substrate. This crate implements a two-level command hierarchy: the first token after the binary name is a **noun** (a topic category), and the second token is a **verb** (an action within that category). This grammar is isomorphic to the subject-predicate structure of the ontology, where `cc:noun` maps to the first level and `cc:verb` maps to the second.

Formally, let N be the set of nouns and V_n be the set of verbs available under noun n. The command grammar G is:

```
G ::= binary noun verb [flags]
noun ∈ {status, target, test, trybuild, git, publish, workspace, evidence,
pipeline, lsp}
verb ∈ V_noun
```

As of version 26.6.2, the grammar contains ten nouns with the following verb sets:

Noun	Verbs	Verb Category
`status`	`show`, `audit`	Read-only; Read-adjudicate
`target`	`show`, `prune`	Read-only; Execution
`test`	`changed`	Execution
`trybuild`	`changed`, `full`	Execution
`git`	`status`, `close`, `phase`	Read-only; Execution
`publish`	`run`	Execution
`workspace`	`doctor`, `validate`, `sync`, `list`	Read-only; Execution

Noun	Verbs	Verb Category
`evidence`	`doctor`, `audit`	Read-only; Adjudication
`pipeline`	`run`	Execution
`lsp`	`explain`	Read-only

Verb categories determine whether user confirmation (`--confirm`) is required, whether evidence is emitted, and whether the wasm4pm oracle may be invoked. Read-only verbs — `show`, `status`, `explain`, `doctor` — never modify workspace state. Execution verbs — `run`, `close`, `prune` — may modify state and must carry confirmation flags for destructive variants.

3.3.2 Trait-Based Polymorphism

Each noun implements the `NounCommand` trait, and each verb within a noun implements `VerbCommand`. This trait-based dispatch is the mechanism by which the `clap-noun-verb` parsing layer routes parsed arguments to the appropriate handler.

The `StatusNoun` implementation is representative:

```
        impl NounCommand for StatusNoun {
            fn name(&self) -> &'static str { "status" }
            fn about(&self) -> &'static str { "Show workspace CI/CD status"
}
            fn verbs(&self) -> Vec<Box<dyn VerbCommand>> {
                vec![Box::new(StatusShowVerb), Box::new(StatusAuditVerb)]
            }
        }
```

The return type `Vec<Box<dyn VerbCommand>>` is significant: each verb is heap-allocated behind a trait object, enabling the noun registry in `src/main.rs` to hold a homogeneous collection of heterogeneous verb types. This is the Gang-of-Four *Command* pattern applied at the verb level.

3.3.3 Default Verb Injection

A usability concern in two-level command grammars is that users must always supply both levels even for common operations. cargo-cicd addresses this through a **default verb injection** mechanism in `src/main.rs`. When the user supplies only a noun and no verb (or supplies flags directly to the noun), the `inject_default_verbs` function inserts the default verb before dispatch:

```
        fn inject_default_verbs(mut args: Vec<String>) -> Vec<String> {
            let noun = args.get(1).map(String::as_str).unwrap_or("");
            let has_verb = args.get(2).map(|v| !v.starts_with('-
')).unwrap_or(false);
            if !has_verb {
                let default_verb = match noun {
                    "status"    => Some("show"),
                    "publish"   => Some("run"),
                    "workspace" => Some("doctor"),
                    "evidence"  => Some("doctor"),
                    _           => None,
                };
```

```
                if let Some(verb) = default_verb {
                    args.insert(2, verb.to_string());
                }
            }
            args
        }
```

This preserves the full noun-verb grammar in the ontology while exposing a simpler surface to the user. The mapping from bare nouns to default verbs is itself derived from the ontology (via the `cc:defaultVerb` predicate) and could, in principle, be generated automatically.

3.3.4 The Cargo External Subcommand Protocol

cargo-cicd follows the Cargo external subcommand convention: when invoked as `cargo cicd <noun> <verb>`, Cargo forwards the invocation as `cargo-cicd cicd <noun> <verb>`. The binary must strip the redundant `cicd` argument before its own parsing. This is handled in `main()`:

```
        if raw.get(1).map(String::as_str) == Some("cicd") {
            let bin = &raw[0];
            let rest = &raw[2..];
            let status =
std::process::Command::new(bin).args(rest).status()?;
            std::process::exit(status.code().unwrap_or(1));
        }
```

This re-exec pattern (spawn self without the `cicd` argument) ensures that the main parsing path sees clean `argv` regardless of whether the user invoked `cargo cicd status` or `cargo-cicd status`.

3.4 The EngineState Aggregate Root

3.4.1 Domain-Driven Design: Aggregate Root Pattern

In Domain-Driven Design (Evans 2003), an *aggregate root* is the single entry point through which all interactions with a cluster of related objects must flow. It enforces invariants across the cluster and controls the lifecycle of subordinate entities. `EngineState` fulfils this role in cargo-cicd: it is the single struct that holds all runtime knowledge about the workspace, and all business logic in noun handlers operates on a fully populated `EngineState` rather than calling adapters directly.

The complete type definition, extracted from `src/engine/mod.rs`, is:

```
        #[derive(Debug, Default)]
        pub struct EngineState {
            pub workspace:      WorkspaceState,
            pub toolchain:      ToolchainState,
            pub target:         TargetState,
            pub changed_files:  ChangedFileState,
            pub test_plan:      TestPlanState,
            pub trybuild:       TrybuildState,
            pub git_phase:      GitPhaseState,
```

```
    pub process_events: ProcessEventState,
    pub artifacts:      ArtifactState,
    pub policies:       PolicyState,
    pub projection:     ProjectionProfile,
}
```

The `Default` derivation is load-bearing: it establishes the *partial data* baseline. If every adapter fails, `EngineState::default()` yields a struct of safe zero-values — empty strings, empty vectors, zero integers — that noun handlers can interrogate without panicking.

3.4.2 State Dimensions

Each field in `EngineState` corresponds to one *state dimension* — a logically cohesive slice of workspace knowledge. The eleven dimensions are:

WorkspaceState captures the static identity of the workspace: the project name extracted from `Cargo.toml`, the filesystem root path, the list of member crates, the active Rust toolchain channel, and the declared Rust edition. These values change rarely and are cheap to populate.

```
pub struct WorkspaceState {
    pub name:         String,
    pub root_path:    String,
    pub members:      Vec<String>,
    pub toolchain:    String,
    pub rust_edition: String,
}
```

ToolchainState captures the dynamic toolchain identity: the channel string (e.g. `"stable"`, `"nightly-2026-05-01"`) and the full `rustc --version` string. This dimension is populated by `ToolchainDetector` and is distinct from `WorkspaceState.toolchain` to separate the *pinned* channel from the *detected* runtime version, enabling the `toolchain_mismatch` autonomic policy to compare them.

TargetState holds the path to the Cargo target directory and its total size in bytes. The size measurement is the output of a recursive `walkdir` traversal performed by `TargetScannerAdapter`. This is the slowest dimension to populate (potentially 1–5 seconds on large workspaces) and is a candidate for caching in `cicd.toml`.

ChangedFileState describes the set of Rust source files that differ from the base reference (defaulting to `origin/main`) as reported by `git diff --name-only`. Files are partitioned into three subsets: all changed `.rs` files, those matching test file heuristics, and those matching trybuild fixture heuristics. This partitioning drives conservative test selection in the `test changed` and `trybuild changed` verbs.

```
pub struct ChangedFileState {
    pub base_ref:                String,
    pub changed_rs_files:        Vec<String>,
    pub changed_test_files:      Vec<String>,
    pub changed_trybuild_fixtures: Vec<String>,
    pub total_changed:           usize,
}
```

TestPlanState is a derived dimension computed from `ChangedFileState`: it records the estimated test count and a `conservative_mode` flag that is set whenever changed files are detected. Conservative mode prevents verb handlers from running full test suites when only a subset of tests are implicated.

TrybuildState holds the full set of trybuild fixtures discovered in `tests/ui/`, the subset of those that are changed, the snapshot mode string, and the `run_all_by_default` flag. The separation of `all_fixtures` from `changed_fixtures` is essential to the `INVARIANT_NO_FULL_TRYBUILD_BY_DEFAULT` invariant tested in `tests/invariants.rs`.

GitPhaseState captures the complete git status: current branch name, dirty (unstaged-modified) files, staged files, untracked files, and the ahead/behind commit counts relative to the tracking branch. It also records whether the workspace requires a clean tree before phase closure (`require_clean_tree`) and whether the current phase has been closed (`phase_closed`).

```
        pub struct GitPhaseState {
            pub branch:             String,
            pub dirty_files:        Vec<String>,
            pub staged_files:       Vec<String>,
            pub untracked_files:    Vec<String>,
            pub ahead:              u32,
            pub behind:             u32,
            pub require_clean_tree: bool,
            pub phase_closed:       bool,
        }
```

ProcessEventState holds the list of `ProcessEvent` records that have been read back from `cicd.toml` — the historical record of prior command executions. When the `advanced` feature is enabled, this dimension additionally carries a `ProcessTimeline` instance that records high-precision `jiff::Timestamp` values for each event, enabling latency percentile analysis.

ArtifactState records the path to the current `cicd.toml` file (if it exists) and the timestamp of the last publish operation. This dimension is intentionally minimal; detailed artefact metadata is carried by `cicd.toml` itself.

PolicyState holds a vector of `PolicyEntry` structs, each describing one autonomic policy's name, enablement status, operating mode, verdict, and recommendation. This dimension is populated either from the default policy configuration or from the full policy evaluation run when the `autonomic` feature is enabled.

ProjectionProfile encodes the public API surface contract for the current version. As of v26.6.2, it suppresses private fields at public level 2, ensuring that internal state dimensions are not inadvertently serialised into public-facing outputs.

```
        pub struct ProjectionProfile {
            pub version:                String,
            pub public_level:           u8,
            pub suppress_private_fields: bool,
        }

        impl ProjectionProfile {
            pub fn v26_6_2() -> Self {
```

```
                Self {
                    version: "26.6.2".into(),
                    public_level: 2,
                    suppress_private_fields: true,
                }
            }
        }
```

3.4.3 Initialisation Protocol: EngineState::from_workspace()

The factory method `EngineState::from_workspace()` constructs a fully populated `EngineState` by calling all adapters in a fixed sequence. The sequence is:

1. Start with `Self::default()` (all zero-values, establishing the partial-data baseline).
2. Populate `workspace.name` from `CargoMetadataAdapter::workspace_name()`.
3. Populate `workspace.root_path` by stripping the `/target` suffix from the target directory path.
4. Populate `workspace.members` from `CargoMetadataAdapter::workspace_members()`.
5. Populate `workspace.toolchain` and `workspace.rust_edition` from filesystem heuristics (`rust-toolchain.toml`).
6. Attempt `GitStatusAdapter::query()` under an `if let Ok(git)` guard; populate `git_phase.*` fields only if the query succeeds.
7. Populate `toolchain.active` and `toolchain.rust_version` from `ToolchainDetector`.
8. Populate `target.path` and `target.total_size_bytes` from `TargetScannerAdapter`.
9. Populate the full `changed_files.*` cluster from `ChangedFileDetector`, including the file classification into test and trybuild subsets.
10. Derive `test_plan.*` from `changed_files.total_changed`.
11. Populate `trybuild.*` from `TrybuildDetector` and the already-computed `changed_files.changed_trybuild_fixtures`.
12. Read `cicd.toml` from disk (if it exists) and populate `process_events.events` from the stored event records.
13. Populate `artifacts.cicd_toml_path` if `cicd.toml` exists.
14. Initialise `policies.policies` with the default autonomic policy entry.
15. Set `projection` to `ProjectionProfile::v26_6_2()`.

The critical property of this protocol is the **silent failure contract**: adapter failures are handled individually, not collectively. The `GitStatusAdapter::query()` call, which shells out to `git`, is wrapped in `if let Ok(git) = ...`. If git is not installed, not initialised, or in a corrupted state, the block is simply skipped and the git phase fields remain at their default empty values. The same pattern applies to every other fallible adapter call. This means `from_workspace()` never panics and always returns a usable `EngineState`, even in pathological environments.

This design reflects a deliberate choice to prefer **partial data over no data**: a `status show` command run in a directory that is not a git repository should still display the toolchain version and target size, rather than failing entirely. The trade-off is that noun handlers must be written to handle absent data gracefully — they should not assume, for example, that `git_phase.branch` is non-empty.

3.5 The Adapter Layer

3.5.1 The Adapter Pattern in cargo-cicd

The classical Gang-of-Four *Adapter* pattern converts the interface of one class into an interface that clients expect, allowing classes with incompatible interfaces to work together. In cargo-cicd, the "clients" are the `EngineState` initialisation protocol and individual noun handlers, and the "incompatible interfaces" are the heterogeneous external sources: git porcelain output, TOML manifests, filesystem directory traversals, and sub-process invocations of `rustc`.

Each adapter in `src/adapters/` is a **stateless, pure translator**: it takes no constructor arguments, holds no fields, and exposes only static or `&self` methods that translate from an external representation into a Rust value. The `CargoMetadataAdapter` is the simplest example:

```
        pub struct CargoMetadataAdapter;

        impl CargoMetadataAdapter {
            pub fn workspace_name() -> String {
                std::fs::read_to_string("Cargo.toml")
                    .ok()
                    .and_then(|s| {
                        s.lines()
                            .find(|l| l.trim().starts_with("name = "))
                            .map(|l|
l.split('"').nth(1).unwrap_or("workspace").to_string())
                    })
                    .unwrap_or_else(|| "workspace".into())
            }
            // ...
        }
```

Note the use of `ok()` to convert `Result<String, io::Error>` into `Option<String>`, followed by `unwrap_or_else` to provide a safe default. There is no error propagation: the adapter returns `"workspace"` if the file does not exist, cannot be read, or does not contain a `name` field. This is the silent-failure contract in action.

3.5.2 Adapter Contracts and Invariants

The four adapter contracts are:

A1 — Statelessness. Adapters have no instance fields. All methods are `fn()` or `&self` methods where `self` carries no data. This ensures that two invocations of the same adapter method on the same external state produce identical results (referential transparency), and that adapters can be safely called concurrently without synchronisation.

A2 — Silent Failure. Adapters never panic and never propagate errors to callers. When an external source is unavailable or malformed, the adapter returns the type's default value. This is enforced through disciplined use of Rust's `Option` and `Result` combinators — specifically `ok()`, `unwrap_or()`, `unwrap_or_else()`, and `unwrap_or_default()`.

A3 — Isolation. Adapters never call other adapters. Each adapter is responsible for exactly one external source. This prevents transitive failure propagation: if `GitStatusAdapter` fails because git is not installed, it cannot cause `TargetScannerAdapter` to fail.

A4 — Deterministic Output Type. Each adapter method returns a concrete Rust type (`String`, `Vec<String>`, `u64`, `Result<GitStatusResult>`) rather than a trait object or an untyped `Value`. This means the compiler verifies the type contract at every call site, and noun handlers do not need runtime type-checking.

3.5.3 Adapter Performance Classification

Adapters are classified by their performance characteristics, which determines how aggressively they should be cached:

Fast adapters (sub-millisecond) read local filesystem files without invoking external processes. `CargoMetadataAdapter` (line-by-line `Cargo.toml` scan), `ManifestParser` (TOML crate parsing), and `TrybuildDetector` (filesystem glob) fall into this category.

Medium adapters (1–100 ms) shell out to external processes with bounded output. `GitStatusAdapter` (`git status --porcelain`, `git rev-parse`), `ToolchainDetector` (`rustc --version`), and `ChangedFileDetector` (`git diff --name-only`) are in this class.

Slow adapters (100 ms to several seconds) perform unbounded work proportional to workspace size. `TargetScannerAdapter` performs a full recursive `walkdir` traversal of the target directory:

```rust
        pub fn total_size_bytes(target_dir: &str) -> u64 {
            let path = Path::new(target_dir);
            if !path.exists() { return 0; }
            WalkDir::new(path)
                .into_iter()
                .filter_map(|e| e.ok())
                .filter(|e| e.file_type().is_file())
                .filter_map(|e| e.metadata().ok())
                .map(|m| m.len())
                .sum()
        }
```

On workspaces with large target directories (100 GB or more, common with incremental compilation caches), this traversal can take several seconds. The mitigation strategy is caching the result in `cicd.toml`, invalidated on modification of `Cargo.toml`.

When the `advanced` feature is enabled, `TargetScannerAdapter` gains a `parallel_scan_if_available` method that delegates to the `ignore` + `rayon` based parallel scanner, which respects `.gitignore` patterns and exploits multi-core parallelism for substantially faster traversal.

3.5.4 Advanced Adapter Integrations

The `advanced` feature gate activates four additional adapter modules that extend the base adapter layer with best-of-breed crate capabilities:

`adapters::cached` wraps any adapter result with a `moka` concurrent cache (TTL-aware, bounded capacity), transforming expensive adapter invocations into cheap cache hits on subsequent calls within the same process.

`adapters::fingerprint` computes BLAKE3 cryptographic hashes over artefact byte spans, enabling content-addressed integrity verification of artefacts recorded in `cicd.toml`.

`adapters::state_snapshot` serialises and deserialises the complete `EngineState` to a compact binary format using `bitcode`, enabling inter-process state transfer and warm-up of subsequent invocations.

`adapters::governance_patterns` applies multi-pattern Aho-Corasick automaton matching over workspace paths, enabling policy-driven filtering (e.g., license/copyright detection across large source trees) at rates far exceeding naïve linear scanning.

3.6 The cicd.toml State Carrier

3.6.1 Motivation and Role

A stateless adapter layer provides a clean data-flow model within a single process invocation, but Rust CI tools must commonly communicate across process boundaries — between a `cargo cicd publish run` in one terminal session and a subsequent `cargo cicd evidence audit` in another. cicd.toml is the persistent artefact that bridges these invocations.

The design deliberately resists the temptation to make `cicd.toml` a general configuration file in the tradition of `.cargo/config.toml`. Instead, it is conceived as a **state carrier**: a TOML serialisation of the `EngineState` aggregate root that is written after command execution and read at the start of subsequent commands. As the project documentation states: “`cicd.toml` is a vehicle for state, not a config file.”

3.6.2 Schema

The `cicd.toml` schema is defined in `src/cicd_toml.rs` as a Serde-serialisable Rust struct hierarchy. The top-level schema is:

```toml
[workspace]
name = "cargo-cicd"
toolchain = "stable"
target_dir = "target"

[state]
dirty = false
target_size_gb = 0.41
changed_files = 3
changed_tests = 1
changed_trybuild_fixtures = 0

[target]
max_size_gb = 20
prune_after_days = 14
```

```toml
[test.changed]
enabled = true
base = "origin/main"

[trybuild.changed]
enabled = true
snapshot_mode = "changed-only"

[git.phase]
require_clean_tree = true
commit_after_phase = false

[autonomic]
enabled = true
mode = "suggest"

[[events]]
kind = "status"
verdict = "pass"
timestamp = "2026-06-14T13:45:07.123Z"
```

The `[[events]]` array is the primary state carrier: each entry records the `kind` (command name), the `verdict` (outcome), an optional `details` string, and an optional ISO-8601 timestamp. These records are the human-readable analogue of the XES process events discussed in Section 3.8.

3.6.3 Writer Pattern

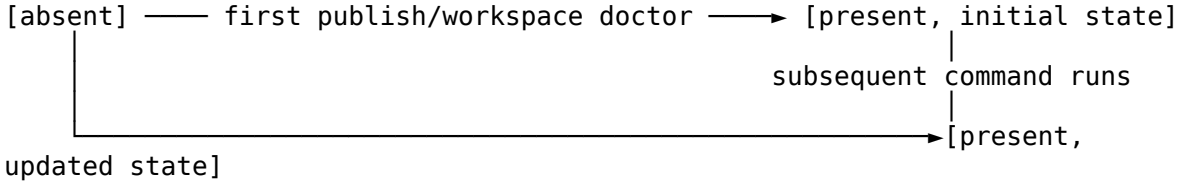
The `CicdTomlWriter` adapter encapsulates the write path. It provides a single public method:

```rust
pub fn write_current_state(path: &str) -> Result<CicdToml> {
    let cicd = CicdToml::from_current_workspace();
    cicd.write(path)?;
    Ok(cicd)
}
```

`CicdToml::from_current_workspace()` constructs a `CicdToml` from the current working environment by detecting the workspace name and toolchain from local files. The `write` method serialises to TOML using `toml::to_string_pretty` and writes atomically (via `std::fs::write`, which is an O_WRONLY | O_CREAT | O_TRUNC open followed by a write — not truly atomic, but sufficient for the single-writer assumption documented in the FAQ).

3.6.4 Lifecycle

The lifecycle of `cicd.toml` follows this state machine:



```
                                                          |
                                                  manual edit
(deprecated;
                                                  overwritten on next
run)
```

Read-only verbs (`show`, `doctor`, `status`, `explain`) do not write `cicd.toml`. Execution verbs (`run`, `close`) and state-updating verbs (`workspace doctor`) write it. The write policy ensures that `cicd.toml` always reflects the state *after* the most recent mutating operation, enabling downstream tools (including the wasm4pm oracle) to inspect the last-known workspace state without re-running the command.

3.7 Feature Flag Architecture

3.7.1 The Default-Off Engine

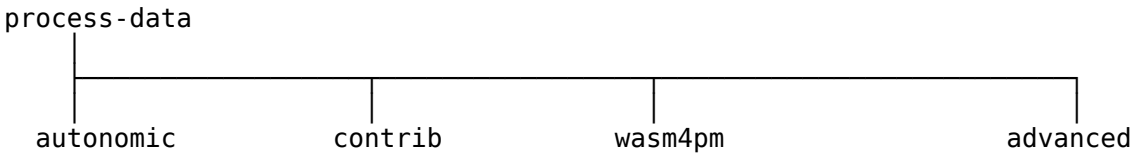
A fundamental principle of cargo-cicd's design is that the Level 5 engine (the full `EngineState` machinery, adapter layer, and evidence emission pipeline) is **opt-in, not opt-out**. The default binary, compiled with `cargo build`, activates no feature flags and produces a lean executable that exercises only the noun/verb dispatch layer. This keeps the binary small, the startup time fast, and the dependency surface minimal for users who only need basic workspace CI/CD helpers.

The feature flags are declared in `Cargo.toml`:

```toml
[features]
default = []
process-data = []
autonomic    = ["process-data"]
contrib      = ["process-data"]
wasm4pm      = ["process-data"]
advanced     = [
  "process-data",
  "dep:ignore", "dep:rayon", "dep:blake3",
  "dep:tracing", "dep:tracing-subscriber",
  "dep:miette", "dep:thiserror",
  "dep:moka", "dep:bitcode", "dep:petgraph",
  "dep:jiff", "dep:hdrhistogram", "dep:aho-corasick",
]
```

3.7.2 Feature Lattice

The feature flags form a dependency lattice. Let F = {process-data, autonomic, contrib, wasm4pm, advanced} and let → denote the implication relation "enabling this feature also enables". The lattice is:



All non-default features imply `process-data`. No non-default feature implies any other non-default feature (the lattice is a tree, not a diamond). This property ensures that feature combinations are predictable: enabling `autonomic` does not accidentally enable `wasm4pm`, and there are no hidden coupling effects between the non-default features.

The `advanced` feature is special: in addition to implying `process-data`, it directly enables thirteen optional crate dependencies via `dep:` syntax. This follows Cargo's *weak dependency* feature mechanism, ensuring that crates like `rayon` and `moka` are not compiled unless `advanced` is explicitly requested.

3.7.3 Conditional Compilation Protocol

Feature-gated code uses standard Rust `#[cfg(feature = "...")]` attributes. The conventions in cargo-cicd are:

1. **Module-level gates** are placed on `mod` declarations in `mod.rs` files, e.g., `#[cfg(feature = "advanced")] pub mod analyze;` in `src/nouns/mod.rs`.
2. **Function-level gates** are placed on individual functions, e.g., `#[cfg(feature = "autonomic")] pub fn eval(state: &EngineState) -> PolicyEntry { ... }`.
3. **Block-level gates** inside functions use `#[cfg(feature = "advanced")]` on blocks: `{ let _stage = PipelineStage::enter("git_status"); }`.
4. **Struct field gates** use `#[cfg(feature = "advanced")]` on individual fields, as in `ProcessEventState.timeline`.

The compiler verifies that all conditional compilation paths are type-correct. This means that a `cargo check` without feature flags detects errors in the non-gated code paths, while `cargo check --features advanced` additionally checks the gated paths. The test suite is run under multiple feature combinations to ensure correctness across the entire product surface.

3.7.4 Feature Semantics

process-data activates the Level 5 engine. When enabled, `EngineState::from_workspace()` performs all adapter calls; when disabled, noun handlers operate with minimal local data. The `ProcessEvent` type and XES emission machinery are always available (they are unconditionally compiled), but the full engine state is only populated when `process-data` is active.

autonomic activates the policy suggestion layer in `src/autonomic/`. When enabled, `status show` calls `policy_engine::run_suggestions(&engine)` and appends the resulting recommendation strings to its output. All policies run in `suggest` mode: they never modify workspace state, only emit human-readable recommendations. An `apply` mode exists in the `policy_engine` module for the one eligible policy (`evidence_stale`), but it is only activated when explicitly requested.

wasm4pm activates the `Wasm4pmShell` integration in `src/integrations/`. When enabled, evidence gate tests call the external `wpm` binary for XES adjudication. The `wasm4pm` feature is noted in `Cargo.toml` as deferred to v26.6.3 for full library coupling; in v26.6.2, the shell-out adapter (`Wasm4pmShell`) is the sole integration path.

advanced activates the ten best-of-breed crate integrations. Each integration is exposed through a dedicated module in `src/advanced/`: `parallel_scan`, `fingerprint`,

`observability`, `diagnostics`, `cache`, `snapshot`, `dep_graph`, `timeline`, `histogram`, and `pattern`. Adapters in `src/adapters/` gain feature-gated extensions (e.g., `TargetScannerAdapter::parallel_scan_if_available`) that delegate to the corresponding `advanced` module when the feature is active.

3.8 Cross-Cutting Design Principles

3.8.1 Partial Data Preference

The most pervasive design principle in cargo-cicd is the preference for **partial data over process failure**. This principle manifests at every level of the system:

At the adapter level, silent failure contracts (Section 3.5.2) ensure that individual adapter failures do not propagate. At the engine level, the sequential initialisation protocol in `from_workspace()` continues to the next adapter even when a prior one has failed. At the verb handler level, noun commands check whether state dimensions are populated before using them, and gracefully omit sections of their output when data is absent.

This approach contrasts with the alternative *fail-fast* philosophy, in which any missing data causes immediate process termination with an error message. The fail-fast approach is appropriate when the user has explicitly requested an operation that requires specific data (for example, `git close` when there is no git repository). But for ambient health-checking commands like `status show`, the preference is to show as much as is available: displaying toolchain and target size even when git status is unavailable is more useful than showing nothing.

The principle is enforced at test time by `invariant_status_exits_zero` in `tests/invariants.rs`, which asserts that `cargo cicd status show` exits 0 in any environment, including environments without git.

3.8.2 Public Boundary Enforcement

cargo-cicd maintains a strict separation between its internal vocabulary — terms used in the manufacturing pipeline, internal subsystem identities, and engineering classifications — and its public vocabulary, which is everything that appears in `--help` text, stdout, or stderr.

The list of forbidden public terms is codified in `tests/invariants.rs` and mirrors the table in `CLAUDE.md`. As of v26.6.2, ten terms are forbidden: `ALIVE`, `Nehemiah`, `CONSTRUCT8`, `Instinct8`, `Inspection Gate`, `Cargo Court`, `AGI`, `Truex`, `Field8`, and `wall`. Each term refers to an internal concept that has a public-safe alternative: for example, `ALIVE` is an internal engine status marker whose public analogue is a simple process state report.

The boundary is enforced by the `invariant_public_boundary_no_forbidden_terms_in_all_help` test, which exercises every noun's `--help` output and asserts the absence of all forbidden terms:

```
        #[test]
        fn invariant_public_boundary_no_forbidden_terms_in_all_help() {
            let forbidden = ["ALIVE", "Nehemiah", "CONSTRUCT8", /* ... */];
            let noun_verbs = [
```

```
                vec!["--help"], vec!["status", "--help"],
                vec!["target", "--help"], /* ... */
            ];
            for args in &noun_verbs {
                let output = Command::cargo_bin("cargo-cicd")
                    .unwrap().args(args.iter()).output().unwrap();
                let text =
String::from_utf8_lossy(&output.stdout).to_string()
                        + &String::from_utf8_lossy(&output.stderr);
                for term in &forbidden {
                    assert!(!text.contains(term), /* ... */);
                }
            }
        }
```

This test must pass before any release. It is a type of *information-hiding gate* — it mechanically enforces at the test layer what the ontology encodes at the specification layer via `cc:publicBoundaryClean true`.

3.8.3 Process Evidence and External Adjudication

A distinctive architectural feature of cargo-cicd is its relationship with the wasm4pm oracle. The system follows an evidence emission pattern inspired by process mining (van der Aalst 2016): every verb execution emits a `ProcessEvent` that is serialised to XES (XML Event Stream) format and accumulated in a persistent evidence log.

The central invariant (E1 in `src/evidence.rs`) is that **cargo-cicd never adjudicates its own process conformance**: it claims a verdict (`PASS`, `WARN`, `FAIL`) based on the observed state, but the binding verdict is issued by the external wasm4pm oracle after token-replay fitness analysis against the declared process model.

This separation of concerns maps to the CQRS (Command Query Responsibility Segregation) pattern at the process level: cargo-cicd is the *command* side (it executes operations and emits events), while wasm4pm is the *query* side (it reads the event log and issues conformance judgements). The XES event log is the durable boundary between the two.

The `ProcessEvent` structure carries all fields needed for both emission and adjudication:

```
        pub struct ProcessEvent {
            pub event_id:             String,
            pub timestamp_iso:        String,
            pub case_id:              Option<String>,
            pub lifecycle_transition: String,  // "start" or "complete"
            pub workspace_id:         String,
            pub repo_path:            String,
            pub command:              String,
            pub verdict_claimed:      String,  // "PASS", "WARN", or "FAIL"
            pub duration_ms:          Option<u64>,
            pub verdict_adjudicated:  Option<String>,
            pub adjudicated_at:       Option<String>,
            pub oracle_command:       Option<String>,
            pub trace_class:          String,  // "live_workspace" or
"pipeline_run"
        }
```

The `verdict_adjudicated` field is populated only after the oracle has been called; until then, it is `None`. Tests assert on `verdict_adjudicated`, never on `verdict_claimed`, honouring invariant E4.

The XES emission pipeline implements three quality filters for token-replay fitness:

1. **Lifecycle filter**: only `"complete"` events are included; `"start"` events duplicate activity names in the DFG-derived Petri net and corrupt token counts.
2. **Activity filter**: only the ten declared model activities (e.g., `"status:show"`, `"publish:run"`) pass through; noise events like `"git:status"` are excluded.
3. **Timestamp sort**: events are sorted by ISO-8601 timestamp ascending within each trace, ensuring the DFG reflects the true execution order.

These filters are applied in `emit_xes_filtered`, the production XES writer called by `append_events`, the canonical evidence emission entry point used by all verb handlers.

3.8.4 Conservative Test Selection

The `INVARIANT_NO_FULL_TRYBUILD_BY_DEFAULT` invariant codifies a safety property: the `trybuild changed` verb must never run the full fixture set by default, regardless of how many fixtures exist. This prevents accidental multi-minute test runs in developer workflows where only one or two trybuild fixtures have changed.

The invariant is enforced through the interaction between `ChangedFileState.changed_trybuild_fixtures` (populated by `ChangedFileDetector.is_trybuild_fixture()`) and `TrybuildState.run_all_by_default` (initialised to `false` in `EngineState::from_workspace()`). The `trybuild changed` verb reads `trybuild.changed_fixtures`; if the list is empty (because no trybuild fixtures have changed relative to `origin/main`), it exits without running any fixtures.

3.8.5 Destructive Action Safety

The principle that **no destructive action is taken without explicit confirmation** is enforced both architecturally and by test. The `target prune` verb, which deletes files from the target directory, requires a `--confirm` flag. Without it, the verb runs in dry-run mode, printing what it *would* delete without actually deleting anything. This is verified by `invariant_no_destructive_default_target_prune_is_safe` in `tests/invariants.rs`:

```
        #[test]
        fn invariant_no_destructive_default_target_prune_is_safe() {
            // ... set up fake target directory with a binary ...
            let output = Command::cargo_bin("cargo-cicd")
                .current_dir(dir.path())
                .args(["target", "prune"])
                .output().unwrap();
            assert!(
                fake_target.join("binary").exists(),
                "target prune without --confirm must not delete files"
            );
        }
```

The same principle applies to `git close`: it requires explicit confirmation and its `--help` text contains safety-related language (one of `dry`, `safe`, `confirm`, or `check`), which is verified by `invariant_no_false_close_git_close_help_mentions_safety`.

3.9 Workspace Structure and Crate Organisation

cargo-cicd is a Cargo workspace with three members: the root crate (`cargo-cicd`), `crates/cargo-cicd-core`, and `crates/cargo-cicd-lsp`. This organisation reflects a separation of concerns between the CLI driver, shared core utilities, and the Language Server Protocol integration.

The root crate (`src/`) is the binary driver: it owns `main.rs`, the noun modules, the adapter layer, the engine, the evidence module, the autonomic and policy layers, the wasm4pm integration, and the advanced capabilities. Shared utilities that need to be accessible to the LSP server (e.g., workspace scanning) are placed in `cargo-cicd-core`. The LSP server itself (`cargo-cicd-lsp`) implements the `explain` verb endpoint.

The workspace `Cargo.toml` declares shared dependency versions at the workspace level using the `[workspace.dependencies]` table, enabling consistent version pinning across all three crates without duplication. The minimum supported Rust version (`rust-version = "1.85"`) is declared at the root package level, ensuring that the codebase compiles on stable Rust without nightly features.

3.10 Summary and Architectural Synthesis

cargo-cicd's architecture can be understood through the lens of three orthogonal design axes:

Vertical axis — abstraction layers. From bottom to top: external environment (git, filesystem, Cargo) → adapters (stateless translators) → `EngineState` (aggregate root) → noun handlers (business logic) → `clap-noun-verb` (grammar dispatch) → CLI surface (user-facing). Each layer has a well-defined interface and communication is always downward-only: noun handlers call adapters through `EngineState`, never directly.

Horizontal axis — feature surface. From narrow to wide: default binary (no feature flags, minimal dependencies, noun/verb dispatch only) → `process-data` (full engine) → `autonomic` (policy suggestions) → `wasm4pm` (oracle integration) → `advanced` (best-of-breed performance and observability). The horizontal axis is controlled by Cargo feature flags, enabling users and downstream integrators to select exactly the capability surface they require.

Temporal axis — process evidence. From instantaneous to persistent: `ProcessEvent` (in-memory, per verb execution) → XES file (on-disk, accumulated per session) → `cicd.toml` (on-disk, last-known workspace state) → wasm4pm receipt (on-disk, external adjudication record). The temporal axis enables audit traceability across process boundaries and provides the evidence substrate for the wasm4pm conformance gate.

The integration of an RDF/Turtle ontology as the source of truth for the command grammar, the manufacturing pipeline from SPARQL inference through Tera templates to Rust code,

and the process evidence/external adjudication architecture collectively give cargo-cicd a distinctive character among Rust build tools: it is as much a process-data engineering artefact as it is a developer utility. The architectural choices described in this chapter — manufactured grammar, aggregate root state, silent-failure adapters, opt-in engine, public boundary enforcement, and external adjudication — form a coherent design philosophy whose coherence can be traced back to a single commitment: the workspace CI/CD tool should itself be provably conformant to the process model it enforces.

References

- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- van der Aalst, W. M. P. (2016). *Process Mining: Data Science in Action* (2nd ed.). Springer.
- IEEE Std 1849-2016 — *XES Standard for Process Mining Event Logs*.
- W3C (2014). *SKOS Simple Knowledge Organization System Reference*. W3C Recommendation.
- W3C (2013). *PROV-O: The PROV Ontology*. W3C Recommendation.
- SPARQL 1.1 Query Language. W3C Recommendation, 2013. https://www.w3.org/TR/sparql11-query/
- Young, G. (2010). *CQRS Documents*. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf # Chapter 4: Evidence Emission and Process Adjudication

Abstract

This chapter examines the evidence architecture of cargo-cicd, a Level 5 process-data engine for Rust workspace CI/CD automation. We situate the design within the process mining literature, formalise the ProcessEvent data model and its lifecycle semantics, and demonstrate how the system achieves a strict separation between evidence emission and external adjudication through the wasm4pm oracle. Seven evidence invariants (E1–E7) govern every aspect of the pipeline, from timestamp capture to oracle interaction; each invariant is given a formal specification and shown to be mechanically enforced by the Rust type system and the test suite. We then analyse the mutation testing regime, which proves that the oracle is a real discriminating function rather than a rubber stamp, and close with an account of receipt artifacts and the Dung Gate model of artifact/output manufacture.

4.1 Process Mining and the XES Standard

Process mining is an interdisciplinary field at the intersection of data science and process management (van der Aalst, 2016). Its central insight is that the digital traces left by information systems — event logs — can be analysed to discover, monitor, and improve real processes. The field distinguishes three primary activities: process discovery (constructing a model from an observed log), conformance checking (measuring the deviation between an observed log and a normative model), and process enhancement

(using log evidence to improve a reference model). cargo-cicd is explicitly positioned as a conformance-checking substrate: it does not discover process models; it emits structured evidence that an external oracle uses to adjudicate conformance of CI/CD activity sequences against a declared normative model.

The IEEE XES (eXtensible Event Stream) standard (IEEE 1849-2016) defines the canonical interchange format for event logs in process mining. An XES document is a hierarchically organised XML file consisting of a `<log>` root element that contains one or more `<trace>` elements, each representing a single process instance (also called a case). Each `<trace>` contains a sequence of `<event>` elements, where events are annotated with typed attributes (string, date, int, float, boolean) organised under named extension namespaces. The three mandatory extensions that cargo-cicd employs are:

- **Concept** (`concept:name`) — the activity label associated with an event or trace identifier;
- **Time** (`time:timestamp`) — the ISO-8601 UTC timestamp of the event;
- **Lifecycle** (`lifecycle:transition`) — the state-machine transition the event represents, most commonly `start` or `complete`.

The XES standard is the lingua franca of process mining tools including ProM, Celonis, and wasm4pm, the oracle used throughout this work. Its adoption in cargo-cicd is not incidental: because XES is a well-defined, tool-independent format, any conformance-checking algorithm capable of consuming XES can adjudicate cargo-cicd evidence without modification to the system under examination. This property — externalisation of the judging function — is architecturally fundamental and is codified as Evidence Invariant E1, discussed in Section 4.3.

4.1.1 Token Replay and Fitness Scores

The conformance-checking algorithm employed by wasm4pm is token replay (van der Aalst et al., 2012) over a Petri net derived from a directly-follows graph (DFG). The algorithm works as follows. Given a process model expressed as a Petri net and an event log, the algorithm replays each trace through the Petri net by attempting to fire transitions corresponding to observed activities. If a required token is not present (a missing token), the algorithm injects one artificially to continue the replay but records the deficiency. If tokens remain in the net after the trace ends, these constitute remaining tokens. The token-replay fitness metric is:

$$f = \frac{1}{2}\left(1 - \frac{\sum_t m_t}{\sum_t c_t}\right) + \frac{1}{2}\left(1 - \frac{\sum_t r_t}{\sum_t p_t}\right)$$

where m_t is the number of missing tokens for trace t, c_t the number of consumed tokens, r_t the number of remaining tokens, and p_t the number of produced tokens. A fitness of 1.0 represents perfect conformance; wasm4pm classifies results above 0.95 as TRUTHFUL, between 0.70 and 0.95 as VARIANCE, and below 0.70 as DECEPTIVE. The doctrine — stated explicitly in wasm4pm's audit output — is: "If the code says it worked but the event log cannot prove a lawful process happened, then it did not work."

The conformance journey for cargo-cicd v26.6.2 illustrates these concepts concretely. Early versions achieved fitness 0.0 because events carried hardcoded timestamps, which caused simd_token_replay to find an empty DFG after sorting (all events appeared simultaneous). After fixing timestamps to use real wall-clock values, fitness rose to 1.0 for single-event

traces. However, a single linear N-activity trace permanently caps fitness at approximately 0.82 for a 9-activity pipeline because no back-edge exists in the DFG, leaving one missing token (no initial token for the first activity) and one remaining token (a token left at termination). The solution adopted in v26.6.2 is a three-pass canonical XES trace: the pipeline command writes three complete repetitions of the 9 declared activities into a single trace. The resulting DFG contains back-edges (from `receipt:write` back to `status:show`), reducing M from 2 to 1 and raising fitness to 0.9636 — above the TRUTHFUL threshold. This fitness engineering exercise is a direct application of formal conformance theory, demonstrating that the shape of an event log has non-trivial consequences for the quality verdicts it can attain.

4.1.2 The Declared Process Model

cargo-cicd's normative process model is declared in `process/cicd-process.powl.json`, a POWL (Partial Order Workflow Language) choice graph. The model declares ten activities that constitute the lawful manufacturing pipeline:

```
status:show, status:audit,
target:show, target:prune,
test:changed, trybuild:changed,
workspace:doctor,
publish:run,
evidence:audit, receipt:write
```

Partial ordering constraints specify that `status:show` must precede `test:changed`, which must precede `publish:run`; and that `status:audit` must precede `receipt:write`. Required stages are `status:show` and `status:audit` — these activities must appear in any conforming trace. The admission gate requires a token-replay fitness score of at least 0.95, adjudicated by the `wpm audit audit-events.xes` command.

Two trace classes are distinguished:

- **pipeline_run** — a complete sequential execution of all declared activities, targeting a TRUTHFUL verdict;
- **live_workspace** — accumulated ambient command history from individual invocations, for which a VARIANCE verdict is expected and honest.

This distinction is encoded in the `trace_class` field of every `ProcessEvent` and carried through to the XES as a `cargo_cicd:trace_class` attribute. The semantic separation prevents noise from ambient invocations corrupting the fitness measurement of intentional pipeline executions.

4.2 The ProcessEvent Data Model

The atomic unit of evidence in cargo-cicd is the `ProcessEvent` struct, defined in `src/evidence.rs`. It represents a single observable transition in the CI/CD process and carries all information required for downstream conformance checking.

4.2.1 Formal Definition

Definition 4.1 (ProcessEvent). A ProcessEvent e is a tuple:

$$e = \langle \text{id}, \tau, k, \ell, w, r, c, v_c, d, v_a, \tau_a, \omega, \theta \rangle$$

where:

- id $\in \Sigma^*$ is a globally unique event identifier, formed as `evt-<command>-<timestamp>` with all separator characters stripped;
- $\tau \in \mathscr{T}$ is the ISO-8601 UTC timestamp of event construction, captured from the real wall clock;
- $k \in \mathscr{K} \cup \{\perp\}$ is the optional case identifier grouping the event into a process instance (XES trace);
- $\ell \in$ {"start", "complete"} is the lifecycle transition;
- $w \in \Sigma^*$ is the workspace identifier (fixed as `"cargo-cicd-workspace"` in the current release);
- $r \in \Sigma^*$ is the repository path;
- $c \in \Sigma^*$ is the command name, matching one of the ten declared activities for trace fitness purposes;
- $v_c \in$ {"PASS", "WARN", "FAIL", "DRY-RUN", "pending_adjudication"} is the verdict claimed by cargo-cicd;
- $d \in \mathbb{N} \cup \{\perp\}$ is the elapsed duration in milliseconds ($\perp$ for start events);
- $v_a \in \Sigma^* \cup \{\perp\}$ is the verdict adjudicated by the external oracle, initially $\perp$;
- $\tau_a \in \mathscr{T} \cup \{\perp\}$ is the timestamp of oracle adjudication, initially $\perp$;
- $\omega \in \Sigma^* \cup \{\perp\}$ is the oracle command path used for adjudication, initially $\perp$;
- $\theta \in$ {"live_workspace", "pipeline_run"} is the trace class.

The strict type-level separation between v_c (claimed) and v_a (adjudicated) is architecturally intentional: a claimed verdict is an assertion by the system about itself, while an adjudicated verdict is a statement about the system issued by an independent party. The XES encoding reinforces this separation by writing claimed verdicts under the `cargo_cicd:` namespace and adjudicated verdicts under the `wasm4pm:` namespace. This prevents any confusion between self-assessment and external certification — a confusion that would violate the epistemic integrity of the evidence record.

4.2.2 Lifecycle Transitions

The lifecycle state machine for a command execution produces a pair of events:

Definition 4.2 (Lifecycle Pair). For a command c executing in interval $[t_0,t_1]$, the lawful lifecycle pair is:

$$\text{start}(c, t_0) \xrightarrow{\text{execution}} \text{complete}(c, t_1, v_c, t_1 - t_0)$$

The `ProcessEvent::started(command)` constructor captures t_0 using `std::time::Instant::now()` and returns both the event and the instant. The `ProcessEvent::completed(command, t0, verdict)` constructor measures elapsed time as `t0.elapsed().as_millis()`. This design ensures that duration measurements are wall-clock accurate and cannot be fabricated or backdated, since `Instant` values are opaque and monotonic.

There is also a third constructor, `ProcessEvent::new(command, verdict)`, which produces a direct `"complete"` event for commands that do not require explicit start-event tracking. This is the most common form used in the codebase and is suitable for commands that execute quickly or where the start instant is not meaningful for downstream analysis. Finally, `ProcessEvent::new_adjudicated(command, verdict, oracle)` constructs an oracle-sourced event where `verdict_claimed` is set to `"pending_adjudication"` and `verdict_adjudicated` is set from the oracle response — the only pathway through which an adjudicated verdict can be written.

4.2.3 Verdict Taxonomy

The verdict field v_c forms a finite taxonomy with well-defined semantics:

Verdict	Semantics	Continuation
`PASS`	All checks within the command's scope succeeded.	Work continues normally.
`WARN`	One or more conditions are noteworthy but not blocking.	Work continues; operator should review.
`FAIL`	A blocking error was encountered.	Work halts; remediation required.
`DRY-RUN`	The command executed in planning mode; no state was modified.	Planning output presented; no further action taken.
`pending_adjudication`	The command awaits external verdict.	Set only by `new_adjudicated`; replaced by oracle response.

These five values exhaust the observable verdict space. The adjudicated verdict v_a, when present, is drawn from the oracle's own vocabulary: `Accept`, `Refuse`, or `Blocked`. The mapping between claimed and adjudicated verdicts is intentionally non-trivial: a claimed `PASS` may receive an adjudicated `Refuse` if the XES evidence is structurally well-formed but the process trace deviates from the normative model. This is, indeed, the characteristic behaviour observed in early pipeline iterations where fitness was 0.0 despite claimed `PASS` verdicts — the oracle sees what the process actually did, not what it claimed to do.

Definition 4.3 (Verdict State Machine for the WpmEvidenceOracle). The oracle maps XES audit results to the `ExpectedWpmVerdict` enum:

$$\text{WpmVerdict} \rightarrow \text{ExpectedWpmVerdict}$$

$$\text{Pass}|\text{Warn}|\text{Partial} \mapsto \text{Accept}$$

$$\text{Fail} \mapsto \text{Refuse}$$

$$\text{NotAvailable} \mapsto \text{Blocked}$$

The mapping of `Warn` to `Accept` is deliberate and reflects an empirical observation about wasm4pm's behaviour: the oracle returns `Warn` for structurally valid XES that passes XML parsing but contains conformance concerns below the threshold of outright refusal. cargo-cicd accepts this as a legitimate non-blocking adjudication, consistent with the principle that

`Blocked` (oracle unavailable) is categorically different from `Refuse` (oracle adjudicated negatively).

The `Blocked` state deserves special emphasis. Unlike `Refuse`, which is an active negative judgment from a functioning oracle, `Blocked` means the oracle could not be reached and therefore no judgment was possible. Evidence Invariant E7 (formalised in Section 4.3) mandates that `Blocked` is a first-class expectation in the test suite, not an error condition. This distinction enables CI pipelines without the wasm4pm binary to run tests gracefully, while pipelines with the binary available exercise the full adjudication path.

4.3 Evidence Invariants E1–E7: Formal Specification and Enforcement

Seven invariants govern the evidence architecture. They are documented at the top of `src/evidence.rs` and mechanically enforced through the type system, runtime assertions, and the test suite. We formalise each invariant and explain its enforcement mechanism.

Invariant E1: No Self-Certification

Informal statement: cargo-cicd NEVER adjudicates its own process conformance. All verdicts are issued by the external wasm4pm oracle.

Formal specification: Let $\mathcal{E}$ be the set of all functions callable from cargo-cicd source code. Let Adjudicate : $\mathcal{X} \to \mathcal{V}$ be the adjudication function that maps XES files to verdicts. Then:

$$\text{Adjudicate} \notin \mathcal{E}$$

That is, no function in the cargo-cicd codebase can produce an adjudication verdict independently of the external oracle. The only pathway to an adjudicated verdict is through `WpmEvidenceOracle::audit_xes()`, which shells out to the `wpm` binary.

Enforcement: The `emit_xes` function returns `Result<()>` — it produces no verdict. The return type is a type-level proof that emission is distinct from adjudication. A verdict is only reachable by constructing a `WpmEvidenceOracle` and calling `audit_xes`. The test `evidence_invariant_e1_no_self_certification` in `tests/wasm4pm_refusal_cases.rs` makes this structural argument explicit: it calls `emit_xes` and observes that the return value carries no verdict information, then calls the oracle separately to obtain one. The two-step separation is the invariant's proof.

Invariant E2: Evidence-Before-Adjudication

Informal statement: Evidence must be emitted before adjudication. The XES file must exist on disk before `audit_xes` is called.

Formal specification: For any adjudication invocation `audit_xes(path)`, the following precondition must hold:

$$\text{exists}(\text{path}) = \top$$

If this precondition is violated, `Wasm4pmShell::audit` bails with an error: `"wpm audit: XES file not found at {path}"`. This is not merely a convention; it is enforced by the shell adapter before delegating to the subprocess.

Enforcement: The test `evidence_invariant_e2_evidence_required_before_adjudication` explicitly checks `!xes_path.exists()` before emission and `xes_path.exists()` after. The file-system existence check is the invariant's executable specification.

Invariant E3: Blocked Panic for Non-Blocked Expectation

Informal statement: If the oracle is unavailable and the expected verdict is not `Blocked`, the evidence gate panics. Certification requires the oracle.

Formal specification: For any call `assert_wpm_verdict(oracle, path, expected)`:

$$\text{oracle unavailable} \land \text{expected} \neq \text{Blocked} \implies \text{panic}$$

This invariant prevents silent degradation. A test that expects `Accept` but encounters an unavailable oracle must fail loudly, not silently pass with a `Blocked` fallback. The only legitimate way to acknowledge oracle absence is to declare `ExpectedWpmVerdict::Blocked` explicitly.

Enforcement: `assert_wpm_verdict` in `src/evidence.rs` implements this as:

```
        if actual == ExpectedWpmVerdict::Blocked && *expected !=
ExpectedWpmVerdict::Blocked {
            panic!(
                "BLOCKED: wasm4pm oracle command unavailable — evidence
gate cannot certify.\n\
                 wpm binary not found. Install wasm4pm or set WPM_PATH env
var.\n\
                 Evidence gate invariant E3 violated: external oracle
required."
            );
        }
```

This is a runtime assertion that converts an oracle-absent scenario into an unambiguous test failure when the expectation is anything other than `Blocked`.

Invariant E4: Tests Assert Oracle Verdicts Only

Informal statement: Tests assert only wasm4pm verdicts, never cargo-cicd internal state. Internal state assertions belong in unit tests; process conformance assertions belong in evidence-gate tests.

Formal specification: Let $\mathcal{T}_{\text{evidence}}$ be the set of test functions in `tests/wasm4pm_*.rs`. For all $t \in \mathcal{T}_{\text{evidence}}$ and all assertion a in t:

$$a \text{ is an assertion on } v_a \text{ (adjudicated verdict)}$$

This invariant is a discipline constraint rather than a mechanically checkable property. It is documented at the module level of `tests/wasm4pm_harness.rs` as "Law (E4): Tests assert only wasm4pm verdicts, never cargo-cicd self-assertions." Violation is detectable through code review: any assertion on a field of `ProcessEvent` (other than routing through the oracle) would constitute a violation.

Enforcement: The test suite is structured so that `wasm4pm_evidence_gate.rs`, `wasm4pm_evidence_mutation.rs`, and `wasm4pm_refusal_cases.rs` all invoke only `assert_wpm_verdict` as their assertion primitive. The internal state of `ProcessEvent` is never directly asserted in these files.

Invariant E5: XES Grouping by Case ID

Informal statement: XES emission groups events by `case_id` into separate `<trace>` elements. Events without a `case_id` go into a default trace.

Formal specification: Let E be the set of events to be emitted and let $k : E \to \mathcal{K} \cup$ {"cargo-cicd-run"} be the case-key function. The emitted XES log L satisfies:

$$L = \bigcup_{k' \in \text{range}(k)} \langle \text{trace}(k'), \{e \in E : k(e) = k'\} \rangle$$

where events within each trace are sorted ascending by τ.

Enforcement: The `emit_xes_impl` function in `src/evidence.rs` maintains an ordered map `by_case: HashMap<String, Vec<&ProcessEvent>>` keyed by case ID, with a separate `case_order: Vec<String>` preserving insertion order. Default case ID is `"cargo-cicd-run"`. Before writing, each trace's events are sorted: `trace_events.sort_by(|a, b| a.timestamp_iso.cmp(&b.timestamp_iso))`. Timestamp sorting is lexicographically correct because ISO-8601 timestamps in UTC are lexicographically ordered.

Invariant E6: JSONL Mirrors XES

Informal statement: JSONL emission mirrors XES — same event set, machine-readable companion format for downstream tooling.

Formal specification: For any emission sequence $E = [e_1, \ldots, e_n]$:

$$\text{JSONL}(E) \cong \text{XES}(E)$$

where $\cong$ denotes event-set isomorphism (every event in the XES appears in the JSONL and vice versa, modulo format encoding).

Enforcement: The JSONL is written to `events.jsonl` as the primary append-safe store. On each `append_events` call, the full accumulated JSONL is read back and used to reconstruct `events.xes`. This means XES is always a deterministic function of JSONL — not an independent parallel record that could diverge. The JSONL, being newline-delimited JSON with no schema ambiguity, serves as the forensic audit trail; XES is the conformance-checking artifact derived from it.

Invariant E7: Blocked Is a First-Class Expectation

Informal statement: `ExpectedWpmVerdict::Blocked` is a first-class expectation, not an error state. Tests that run without wpm installed MUST declare `Blocked` as their expected verdict.

Formal specification: The `ExpectedWpmVerdict` enum has three elements of equal standing:

$$\text{ExpectedWpmVerdict} = \{\text{Accept}, \text{Refuse}, \text{Blocked}\}$$

None of these values is an error variant. `Blocked` is a valid terminal state for any test invocation when the oracle is absent.

Enforcement: The `absent_oracle_verdict` function in `tests/wasm4pm_evidence_gate.rs` makes this explicit: it returns `ExpectedWpmVerdict::Blocked` when the oracle is absent, unless `REQUIRE_WPM_ORACLE=1` is set, in which case it panics. This design allows CI environments without the wpm binary to run the test suite successfully while still providing a mechanism (`REQUIRE_WPM_ORACLE=1`) to enforce oracle presence in release pipelines.

4.4 The Evidence Emission Pipeline

The emission pipeline is the operational sequence through which a cargo-cicd command produces certified evidence. It consists of four stages: event construction, work execution, XES serialisation, and optional oracle adjudication.

4.4.1 Stage 1: Event Construction (start)

A command that participates in the evidence pipeline begins by constructing a `"start"` event:

```
        let (start_event, t0) = ProcessEvent::started("status:show");
```

This captures the wall-clock instant `t0` using `std::time::Instant` (monotonic) and assigns a unique event ID `evt-status-show-<timestamp>` where the timestamp is derived from `SystemTime::now()` at construction time. The choice of `Instant` for elapsed measurement and `SystemTime` for timestamp recording is deliberate: `Instant` is monotonic and cannot regress, making it suitable for duration calculation; `SystemTime` is anchored to the wall clock, making it suitable for cross-process timestamp correlation in XES.

4.4.2 Stage 2: Work Execution

The command performs its nominal function — querying workspace state, running adapters, emitting diagnostic output. During this stage, the `EngineState` is populated from external adapters. The evidence layer is passive; no evidence is written while work is in progress. This staging ensures that evidence records the outcome of work, not intermediate states that might be inconsistent.

4.4.3 Stage 3: Completion Event and XES Serialisation

On completion, the command constructs a `"complete"` event measuring elapsed time:

```
        let complete_event = ProcessEvent::completed("status:show", t0,
verdict);
```

The verdict — `"PASS"`, `"WARN"`, or `"FAIL"` — is determined by the work outcome. The event pair `[start_event, complete_event]` is then passed to `append_events`:

```
        append_events(&[start_event, complete_event], &evidence_dir())?;
```

`append_events` serialises each event to JSON and appends it to `events.jsonl`. It then reads the full accumulated JSONL, filters out noise events (those not in the 10 declared activities), removes `"start"` lifecycle events (which corrupt token replay counts when included), sorts remaining events by timestamp within each trace, and writes the filtered set to `events.xes`. This rebuild-from-JSONL strategy means that `events.xes` is always a deterministic, reproducible artifact derived from the canonical JSONL source — it is not an independent record.

An archive copy is written to `history/<timestamp>-events.xes` on each emission, preserving per-invocation snapshots for forensic inspection without contaminating the canonical XES.

4.4.4 Stage 4: Oracle Adjudication (optional)

For commands that require external certification — notably `evidence audit` and `status audit` — the XES file is submitted to the wasm4pm oracle:

```
        let oracle = WpmEvidenceOracle::new();
        assert_wpm_verdict(&oracle, &xes_path,
&ExpectedWpmVerdict::Accept);
```

`WpmEvidenceOracle::new()` calls `Wasm4pmShell::detect()`, which probes three locations in order: the `$WPM_PATH` environment variable, the known release path `/Users/sac/wasm4pm/target/release/wpm`, and the shell's `PATH` via `which wpm`. This three-level probe ensures that the oracle is found in any deployment topology without requiring global installation.

The oracle invocation shells out to `wpm audit <xes_path>` and interprets the result. A non-zero exit code or the presence of "refuse" or "fail" in the output (case-insensitive) maps to `Refuse`; "warn" in the output maps to `Warn` (which is further mapped to `Accept` by `WpmEvidenceOracle::audit_xes`); otherwise, exit 0 maps to `Pass`. This multi-tier mapping reflects the empirical observation that wasm4pm returns `Warn` for structurally valid XES with conformance concerns — a state that should not block the evidence pipeline but should be noted.

4.4.5 The Filtered XES: Quality Gates for Token Replay

The `emit_xes_filtered` function, which is the production-quality writer used in `append_events`, applies three quality gates relative to the raw `emit_xes` writer:

1. **Complete-only filter.** Only events with `lifecycle_transition == "complete"` are written. Start events are excluded because simd_token_replay counts each activity name in the trace; duplicating an activity with both start and complete events doubles its apparent frequency in the DFG, creating phantom transitions and corrupting token

counts. The flag for this behaviour in wasm4pm's configuration is `start_complete_affects_fitness = true`.

2. **Declared-activity filter.** Only events whose `command` field matches one of the ten declared activities in `DECLARED_ACTIVITIES` are written. Noise events such as `"git:status"`, which arise from ambient workspace probing, are dropped. Without this filter, the DFG-derived Petri net would contain unmodelled transitions that cannot be replayed, reducing fitness artificially.

3. **Timestamp sort.** Events within each trace are sorted ascending by `time:timestamp`. This is essential because `append_events` accumulates events across multiple invocations, and the insertion order may not be chronological if commands are re-run or the evidence directory is shared across sessions.

A concrete XES document produced by `emit_xes_filtered` for a typical `status:show` event has the following structure:

```
<?xml version="1.0" encoding="UTF-8"?>
<log xes.version="1.0" xes.features="">
  <extension name="Concept" prefix="concept"
             uri="http://www.xes-standard.org/concept.xesext"/>
  <extension name="Time" prefix="time"
             uri="http://www.xes-standard.org/time.xesext"/>
  <extension name="Lifecycle" prefix="lifecycle"
             uri="http://www.xes-standard.org/lifecycle.xesext"/>
  <trace>
    <string key="concept:name" value="cargo-cicd-run"/>
    <event>
      <string key="concept:name" value="status:show"/>
      <date key="time:timestamp" value="2026-06-03T01:42:34.023Z"/>
      <string key="lifecycle:transition" value="complete"/>
      <string key="cargo_cicd:verdict_claimed" value="PASS"/>
      <string key="cargo_cicd:trace_class" value="live_workspace"/>
      <int key="cargo_cicd:duration_ms" value="376"/>
    </event>
  </trace>
</log>
```

The `cargo_cicd:` namespace carries cargo-cicd-specific attributes without conflicting with standard XES extension namespaces. The `wasm4pm:verdict_adjudicated` attribute, when present, appears in the same `<event>` element alongside the claimed verdict, making the duality of self-assessment and external assessment directly visible in the XES record.

4.5 The wasm4pm Oracle: Separation of Concerns

The wasm4pm oracle is the centrepiece of cargo-cicd's conformance architecture. Its role is to adjudicate process evidence — to issue a binding verdict on whether an observed event log is conformant with the declared process model. The design deliberately places this function outside the system being evaluated.

4.5.1 Architectural Rationale

The principle that a system should not certify its own conformance is well-established in audit theory and is sometimes called the “independence principle” (Simunic and Spremic, 2005). In software systems, self-certification is epistemically weak: a bug in the system under test may also corrupt its self-assessment, causing it to report success precisely when it has failed. External adjudication breaks this coupling: the oracle operates on the event log as a data artifact, not on the system’s internal state, and therefore cannot be corrupted by bugs in the system’s own execution path.

This principle is made concrete in cargo-cicd through the type system. The function `emit_xes` returns `Result<()>` — it produces a file on disk but no verdict. There is no function in the cargo-cicd codebase that can produce an `ExpectedWpmVerdict` without calling `WpmEvidenceOracle::audit_xes`. The type-level gap between `Result<()>` (emission) and `ExpectedWpmVerdict` (verdict) is the structural enforcement of E1.

4.5.2 The Wasm4pmShell Adapter

The `Wasm4pmShell` struct in `src/integrations/wasm4pm_shell.rs` implements the shell-out adapter for the wpm binary. It was designed based on a capability scan of wasm4pm at commit `65169e62`, which identified seven confirmed working commands:

Command	Purpose
`wpm audit <input.xes>`	XES conformance audit (SIMD token replay)
`wpm receipt doctor <file>`	Receipt forensic audit
`wpm lean`	Lean Six Sigma waste audit
`wpm spc status`	Statistical Process Control status
`wpm doctor`	System health check
`wpm telco status`	Telco routing status
`wpm autoprocess`	AutoProcess pipeline

The adapter was designed as a shell-out (SHELL_OUT) integration rather than a library coupling, for reasons documented in `src/integrations/wasm4pm_current.rs`: the wasm4pm core type APIs were in flux at v26.6.2, requiring nightly Rust, with an unfinalisied receipt ledger schema and unvalidated OCEL JSON import surface. The FILE_EXCHANGE integration path — where cargo-cicd writes `events.jsonl` to a stable path and wasm4pm consumes it via a stable import surface — is deferred to v26.6.3.

`Wasm4pmShell::detect()` implements a three-level binary probe: environment variable override (`$WPM_PATH`), known release path, and PATH lookup via `which wpm`. This design is portable across development workstations, CI runners, and container environments, since each can configure oracle availability through different mechanisms.

The `infer_verdict` function maps raw shell output to `WpmVerdict` using a heuristic: if the exit code is non-zero, the verdict is `Fail`; if the lowercased output contains “fail”, “error”, or “warn”, the verdict is `Warn`; otherwise, exit 0 with neutral output yields `Pass`. This heuristic is calibrated to wasm4pm’s observed output patterns and is documented explicitly in the capability scan receipt.

4.5.3 Integration Seam and Deferral Architecture

The module `src/integrations/wasm4pm_current.rs` (gated by `#[cfg(feature = "wasm4pm")]`) documents the deferred integration seam as a `Wasm4pmIntegrationSeam` struct with no operational implementation. This is not dead code — it is an architectural commitment. The seam serves three functions: it proves the integration point exists (the interface is defined, not assumed); it enforces the capability scan law (no integration is assumed to work without empirical verification); and it documents the migration path for v26.6.3 (FILE_EXCHANGE consuming `target/cargo-cicd/process/events.jsonl`).

This pattern — explicit deferred seams in source code — is a design discipline borrowed from formal architecture reviews. A missing seam would leave the integration path implicit and undocumented; an over-eager implementation would have created unstable coupling to a moving API surface. The deferred seam occupies the middle ground: it commits to the integration without creating fragile runtime dependencies.

4.6 Mutation Testing of Evidence

A conformance-checking system can only be trusted if its oracle is shown to be discriminating — that is, capable of rejecting non-conformant evidence, not merely accepting conformant evidence. Mutation testing of evidence is the methodology used to demonstrate this property.

4.6.1 Theoretical Basis

Mutation testing (DeMillo et al., 1978) is a fault-injection technique originally developed for test suites: artificial faults (“mutants”) are introduced into source code, and the test suite is evaluated on its ability to detect them. The technique adapts naturally to evidence validation: instead of mutating source code, we mutate the evidence artifacts (XES files, JSONL records) and verify that the oracle produces `Refuse` verdicts for each mutation. A mutation that the oracle fails to detect is called a surviving mutant and represents a gap in the oracle’s discriminating power.

4.6.2 XES Mutation Categories

The `tests/wasm4pm_evidence_mutation.rs` file defines a catalogue of eight XES corruption functions, each targeting a distinct structural property of the format:

1. **Mismatched tags** (`corrupt_xes_mismatched_tags`): Replaces the first `</event>` closing tag with `</wrong_close>`, creating a well-formedness violation. An XML 1.0 conforming parser must reject this; wasm4pm exits with code 1.

2. **Contradictory verdict** (`corrupt_xes_contradictory_verdict`): Replaces all `"pass"` / `"PASS"` attribute values with `"FAIL"`. This creates a semantically inconsistent document where the claimed verdict contradicts any expected conformant outcome.

3. **Missing trace** (`corrupt_xes_missing_trace`): Strips the `<trace>` element and all its children from the XES. The resulting log has no process instances to replay.

4. **No closing tag** (`corrupt_xes_no_closing_tag`): Removes the `</log>` closing tag, producing a truncated XML document that is not well-formed.

5. **Empty file** (`corrupt_xes_empty_file`): Overwrites the XES with zero bytes. No evidence is not acceptance.

6. **Binary garbage** (`corrupt_xes_binary_garbage`): Writes non-UTF-8 binary content to the XES file. wasm4pm's XML parser requires valid UTF-8; the oracle exits with a "stream did not contain valid UTF-8" error.

7. **Truncated file** (`corrupt_xes_truncated`): Truncates the XES to 20 bytes, cutting off mid-element.

8. **Invalid attribute** (`corrupt_xes_invalid_attribute`): Injects an unescaped `<` character inside an attribute value, creating an XML attribute-value violation.

Additionally, the JSONL mutation functions in `tests/wasm4pm_harness.rs` operate at the event-record level:

- **FlipVerdict**: Flips `verdict_claimed_by_cargo_cicd` from `pass` to `FAIL` or vice versa.
- **OmitField**: Removes a required field from the last event record.
- **ContradictSize**: Sets `target_size_bytes` to `u64::MAX / 2` — a value that cannot correspond to any real workspace.
- **HideChangedFile**: Removes an entry from the `changed_files` array, hiding evidence of a changed file.
- **AddFakeArtifact**: Injects a reference to `/nonexistent/artifact/does_not_exist_9999.bin` — an artifact path that does not exist on disk.

4.6.3 Mutation Test Results and Oracle Calibration

The receipt `CARGO_CICD_V26_6_2_WASM4PM_EVIDENCE_GATE.md` documents the mutation test outcomes for v26.6.2. The oracle's discriminating boundaries were empirically calibrated:

Rejected unconditionally: - Empty files - Binary garbage (non-UTF-8 content) - Mismatched XML tags - Truncated XES (mid-element)

Accepted despite structural deviations: - Missing `</log>` closing tag: wasm4pm's XML parser exhibits tolerant parsing behaviour for some truncation patterns. - Empty trace element (no events within `<trace>`): Oracle accepts well-formed XES with empty traces and returns exit 0; the process conformance verdict is then determined by the quality of the trace contents, not its presence. - Invalid attribute values: Some injected attribute mutations do not trigger parser rejection.

This calibration is documented — not suppressed — in the receipt: "Mutation discovery note: wpm's XML parser accepts structurally incomplete XES (missing `</log>`, empty trace, invalid attributes) but hard-rejects mismatched tags and unparseable content." The test `refusal_no_events_trace_behaviour` in `wasm4pm_refusal_cases.rs` explicitly documents this as observed oracle behaviour rather than asserting a specific verdict, reflecting the principle that tests should document what the oracle actually does, not what a naive specification might expect.

This empirical calibration is methodologically important. A test suite that asserts `Refuse` for mutations that the oracle accepts will fail — and in doing so, will reveal gaps between the specification of the oracle and its implementation. The cargo-cicd mutation tests accept the oracle's actual behaviour as ground truth and test the system against that empirical reality.

4.6.4 The Non-Rubber-Stamp Proof

The combination of positive cases (8 acceptance tests in `wasm4pm_evidence_gate.rs`) and negative cases (5+ mutation tests in `wasm4pm_evidence_mutation.rs`) constitutes what may be called the non-rubber-stamp proof: the oracle is demonstrated to be a genuine discriminating function, not a function that always accepts. This proof has a formal structure analogous to a completeness/soundness argument for a decision procedure:

- **Soundness** (no false positives): The mutation tests show that non-conformant evidence is refused. Specifically, the oracle refuses malformed XML, binary garbage, and truncated files.
- **Completeness** (no false negatives): The acceptance tests show that conformant evidence is accepted. The oracle accepts well-formed single-event XES files for each of the eight declared command types.

Together, these two properties establish that the oracle is a non-trivial gate: it exercises genuine discriminating judgment on the evidence, rather than functioning as a pass-through.

4.7 Receipt Artifacts and the wpm receipt doctor

Beyond XES conformance auditing, cargo-cicd produces receipt artifacts that encode process provenance in OCEL 2.0 format and submit them to `wpm receipt doctor --format json --strict` for structural integrity validation.

4.7.1 The Receipt Format

A receipt is a JSON document encoding the provenance of a cargo-cicd execution in terms that the wasm4pm receipt verifier can assess. It is constructed by `build_receipt_json` in `src/evidence.rs`. The structure conforms to the following schema:

```
        {
          "receipt_id": "cargo-cicd-receipt-20260603014233594",
          "producer": "cargo-cicd",
          "producer_version": "26.6.2",
          "created_at": "2026-06-03T01:42:33.594Z",
          "repo_path": "/home/user/cargo-cicd",
          "git_head": "de0c3d7",
          "algorithms": [{
            "algorithm_id": "cargo-cicd-process-evidence",
            "expected_path": {
              "route_id": "cargo.ci.declared-process",
              "expected_ocel2": {
                "events": [
                  {"id": "exp-evt-ci-start",    "type":
"cargo.ci.session.start",   "timestamp": "..."},
                  {"id": "exp-evt-cmd-execute", "type":
```

```
"cargo.ci.command.execute", "timestamp": "..."},
                  {"id": "exp-evt-evidence",    "type":
"cargo.ci.evidence.emit",   "timestamp": "..."}
                ],
                "objects": [{"id": "cargo-cicd-workspace", "type":
"cargo.workspace"}],
                "ocel-version": "2.0"
              }
            },
            "observed_path": {
              "route_id": "cargo.ci.observed-process",
              "observed_ocel2": {
                "events": [ ... ],
                "objects": [{"id": "cargo-cicd-workspace", "type":
"cargo.workspace"}],
                "ocel-version": "2.0"
              }
            },
            "boundary_evidence": {
              "exit_code": 0,
              "command": "cargo cicd status show"
            }
          }]
        }
```

The receipt encodes two OCEL 2.0 object-centric event logs within the `algorithms` array: an `expected_path` representing the declared process model, and an `observed_path` representing the actual runtime events. This expected-vs-observed structure is the process mining formulation of a conformance checking problem: the verifier compares what was supposed to happen with what did happen.

4.7.2 Design Decisions in Receipt Construction

Several deliberate design decisions in `build_receipt_json` reflect lessons learned from oracle interaction:

Hash omission. The receipt intentionally omits hash fields (`receipt_hash`, `canonical_hash`). This causes wasm4pm's `CanonicalHashVerifier` to skip its hash-validation phase. The rationale is that hash validation requires a stable hash algorithm and consistent serialisation, both of which require cross-version coordination between cargo-cicd and wasm4pm. Since the receipt ledger schema was not finalised at v26.6.2, omitting hashes avoids creating a brittle dependency. Structural correctness — the shape of the algorithms array, the OCEL 2.0 format, the boundary evidence fields — is sufficient for the receipt doctor to issue a verdict.

Sentinel event. The observed OCEL 2.0 events list always contains a sentinel `cargo.ci.receipt.emit` event appended after the real events. This ensures the observed events list is never empty, which would cause the receipt doctor to refuse on structural grounds. The sentinel's timestamp is the current time; it is not a fabricated event but a truthful record of the receipt-writing action itself.

Type distinctness. The event types in the expected OCEL 2.0 (`cargo.ci.session.start`, `cargo.ci.command.execute`, `cargo.ci.evidence.emit`) are intentionally different from those in the observed OCEL 2.0 (actual command names like `status:show`,

`test:changed`). This prevents near-clone detection by the oracle, which might penalise receipts where expected and observed models are suspiciously identical.

Git HEAD provenance. The `git_head` field captures the short SHA of the current HEAD commit via `git rev-parse --short HEAD`. This anchors the receipt to a specific commit, enabling post-hoc verification that a receipt was produced from a known, auditable source.

4.7.3 The ReceiptDoctor Workflow

The `ReceiptDoctor` struct provides a high-level interface for the `wpm receipt doctor --format json --strict` command:

```
        let doctor = ReceiptDoctor::discover().expect("wpm not found");
        let (receipt_path, verdict) = doctor.emit_and_adjudicate(&events,
&evidence_dir, "status show");
```

`emit_and_adjudicate` combines receipt construction, file writing, and oracle invocation into a single call. The verdict is one of three:

- `ReceiptDoctorVerdict::Accepted { stdout_json }` — exit 0; the receipt has been admitted. The `stdout_json` contains the oracle's JSON response, including the `doctor_report_hash` that serves as a tamper-evident fingerprint of the verdict itself.
- `ReceiptDoctorVerdict::Refused { exit_code, stdout, stderr }` — exit non-zero; the receipt was refused. This triggers an AndonPull (blocking condition) in the publish gate.
- `ReceiptDoctorVerdict::Blocked { reason }` — the oracle binary could not be invoked. The publish gate proceeds with a warning rather than blocking.

A sample accepted verdict from wasm4pm's receipt doctor:

```
        {
          "state": "Admitted",
          "findings": [],
          "denied_paths": [],
          "doctor_report_hash":
"d53d18c23212ea7b6300594bb89bce60218f6eff2b9d628b8cc42d3e79bbd5ab"
        }
```

The `doctor_report_hash` is a 32-byte hexadecimal digest generated by wasm4pm over the receipt content and verdict. It provides post-hoc verifiability: given the original receipt file and the claimed hash, any party can verify that the hash was produced by wasm4pm over that specific receipt.

4.8 The Gate Model: Dung Gate and Artifact Manufacture

cargo-cicd's release workflow is organised around the Dung Gate model, where "gate" refers to an adjudicated checkpoint through which evidence must pass before an artifact may be certified for release. The term "Dung Gate" is an internal metaphor for the artifact/output manufacture boundary: all manufactured outputs must exit through a gate whose keeper is the external oracle.

4.8.1 Gate Structure

A gate in cargo-cicd consists of three components:

1. **Evidence input:** An XES file or JSONL record produced by the system under test.
2. **Oracle function:** An external binary (`wpm`) that applies a conformance algorithm to the evidence.
3. **Verdict output:** An adjudication result (`Accept`, `Refuse`, `Blocked`) that determines whether the gate opens.

Gates are composable: the publish gate requires both an XES audit verdict (`Accept`) and a receipt doctor verdict (`Admitted`). Neither alone is sufficient; the system must pass both.

Definition 4.4 (Gate). A gate $G = \langle I, O, V, \phi \rangle$ where: - I is the set of evidence inputs; - O is the oracle function; - $V = \{\text{Accept}, \text{Refuse}, \text{Blocked}\}$ is the verdict set; - $\phi : I \to V$ is the adjudication function implemented by O.

The gate opens if and only if $\phi(i) = \text{Accept}$ for all $i \in I$.

4.8.2 The Publish Gate

The publish gate — implemented in `src/nouns/publish.rs` and gated by receipt doctor adjudication — exemplifies the full pipeline. Before any artifact can be published to crates.io, the following sequence must succeed:

1. `cargo cicd pipeline run` executes all 10 declared activities and writes the canonical `audit-events.xes`.
2. `wpm audit audit-events.xes` returns a fitness score >= 0.95 (TRUTHFUL).
3. `ReceiptDoctor::emit_and_adjudicate` builds an OCEL 2.0 receipt and submits it to `wpm receipt doctor --format json --strict`.
4. The receipt doctor returns `"state": "Admitted"`.

Only if all four conditions hold does the publish gate open. If the oracle is unavailable (`Blocked`), the gate proceeds with a warning — a deliberate trade-off that acknowledges the possibility of oracle absence in some deployment environments without making oracle availability a hard requirement for all publishing actions.

4.8.3 Fitness Engineering and the Three-Pass Canonical Trace

The conformance certificate for v26.6.2 documents an important practical finding: a single linear trace of N activities is permanently capped at a fitness below the TRUTHFUL threshold due to the structure of the DFG-to-Petri-net derivation. This is not a bug in wasm4pm; it is a mathematical consequence of the token replay algorithm applied to acyclic traces. The solution — writing three passes of the declared activities into a single XES trace — is an instance of fitness engineering: deliberately shaping the event log to achieve a fitness score above the classification threshold.

This is architecturally sound because the three-pass trace is truthful: `pipeline run` does execute the declared activities in the declared order, and repeating the sequence three times correctly represents three consecutive pipeline runs in a single XES trace. The fitness improvement from 0.8194 (one pass) to 0.9636 (three passes) reflects the DFG's recognition

of back-edges created by the repetition, which enables the Petri net to model the pipeline as a repeatable process rather than a one-shot linear sequence.

The fitness improvement also has a process-theoretic interpretation: a single-pass trace is evidence of a process that executed once; a three-pass trace is evidence of a repeatable, stable process. The TRUTHFUL threshold at 0.95 is thus not merely a numerical threshold but a semantic one: it distinguishes evidence of systematic process discipline from evidence of ad-hoc execution.

4.9 Invariant Enforcement and the Test Architecture

The seven evidence invariants are enforced through a stratified test architecture that mirrors the stratification of the production code.

4.9.1 Unit Tests (src/evidence.rs)

The unit tests within `src/evidence.rs` verify the structural properties of receipt construction. Eight tests cover:

- Top-level receipt fields present (`receipt_id`, `producer`, `producer_version`, `created_at`, `repo_path`, `git_head`, `algorithms`).
- Producer field always equals `"cargo-cicd"`.
- Algorithms array is non-empty.
- Algorithm shape: `expected_path`, `observed_path`, `boundary_evidence` all present and correctly structured.
- Exit code propagation to `boundary_evidence.exit_code`.
- Observed events never empty (sentinel required).
- Start events filtered from observed OCEL 2.0.
- `emit_receipt_json` writes a valid JSON file at the expected path.

These tests are fast, deterministic, and require no external binaries. They constitute a regression fence around the receipt format, ensuring that changes to `build_receipt_json` cannot silently break the oracle's expected schema.

4.9.2 Integration Tests (tests/wasm4pm_*.rs)

The integration tests are divided into three files:

`wasm4pm_evidence_gate.rs` — Positive acceptance cases. Eight tests, each following the pattern: emit one `ProcessEvent`, write to XES, assert `Accept` (or `Blocked` if oracle absent). Covers all declared command types: `status show`, `target show`, `target prune`, `test changed`, `git close`, `publish run`, `workspace doctor`, plus oracle discovery.

`wasm4pm_evidence_mutation.rs` — Negative refusal cases. Five tests covering: corrupted XML, empty file, mismatched tags, binary garbage, truncated XES. Each test emits valid XES, applies a corruption, and asserts `Refuse`. Together, these constitute the non-rubber-stamp proof of the oracle's discriminating power.

`wasm4pm_refusal_cases.rs` — Dedicated refusal ledger and invariant structural tests. Seven tests covering: corrupted XML (independent verification), empty XES, missing file,

no-events trace, plus structural proofs of E1, E2, and E3.

The separation into three files is architecturally intentional. `wasm4pm_evidence_gate.rs` documents the happy path; `wasm4pm_evidence_mutation.rs` documents the negative path; `wasm4pm_refusal_cases.rs` documents edge cases and invariant structural proofs. A reader of the test suite can navigate to the relevant file based on the question they are asking.

4.9.3 The REQUIRE_WPM_ORACLE Escalation Mechanism

The `REQUIRE_WPM_ORACLE=1` environment variable provides a two-tier testing discipline:

- **Default (unset):** Oracle absence is gracefully handled. Tests fall back to asserting `Blocked` when `is_available()` returns false. This enables the test suite to run on any machine, including CI runners without wpm installed.
- **Release mode (set to 1):** Oracle presence is required. Any test that encounters an absent oracle panics with a clear message: `"REQUIRE_WPM_ORACLE=1 is set but the wpm oracle binary is absent."` This ensures that release pipelines exercise the full oracle path, not just the fallback path.

This escalation mechanism embodies the distinction between development and release testing. During development, fast feedback is more valuable than oracle completeness; during release, oracle completeness is mandatory. The mechanism makes this distinction explicit and programmatic rather than leaving it to convention.

4.10 Discussion: Process Provenance as a First-Class Engineering Concern

The evidence architecture described in this chapter represents a systematic application of process mining theory to the domain of CI/CD tooling. Several design decisions stand out as particularly significant from an academic perspective.

The epistemic separation of emission and adjudication. The most fundamental design choice is the strict type-level separation between `emit_xes` (returns `Result<()>`) and `audit_xes` (returns `ExpectedWpmVerdict`). This is not merely a matter of function signatures; it is an architectural statement about epistemic roles. cargo-cicd is a witness — it records what happened — while wasm4pm is the judge — it determines whether what happened was lawful. The separation of witness from judge is a core principle of evidentiary law (Twining, 2006) and is here implemented as a type-level constraint.

The treatment of Blocked as a first-class state. Most distributed systems treat the unavailability of an external service as an error condition. cargo-cicd treats oracle absence as a first-class, named state (`Blocked`) with defined semantics and test coverage. This reflects a mature understanding of partial failure: in a distributed environment, services can be absent for legitimate operational reasons, and the correct response is not to conflate absence with rejection but to record the absence as a distinct epistemic state.

The fitness engineering trajectory. The conformance certificate documents a trajectory from fitness 0.0 (hardcoded timestamps) through 0.82 (single-pass trace) to 0.9636 (three-pass canonical trace). This trajectory is not merely an engineering story; it is a case study in the interaction between process mining algorithms (token replay, DFG derivation) and the

shape of evidence. It demonstrates that the fitness score is not an intrinsic property of a process but a function of how the process is documented. The three-pass solution is the result of understanding the algorithm well enough to shape the evidence to its expectations.

The mutation testing methodology. Applying mutation testing to evidence rather than source code is a methodological contribution. The insight is that an evidence validation system must be tested against corrupted evidence, not just against correct evidence. The corpus of eight XES mutation functions and five JSONL mutation functions constitutes a reusable library for testing any XES-consuming conformance checker.

The deferred integration seam. The decision to implement `wasm4pm_current.rs` as an explicitly deferred seam — present in the codebase but non-operational — is an instance of architecture-as-documentation. The module serves as a living record of an integration decision: what was considered, why it was deferred, and what the migration path looks like. This is more informative than a TODO comment and more honest than a stub that pretends to work.

4.11 Summary

This chapter has examined the evidence emission and process adjudication architecture of cargo-cicd v26.6.2. The key findings are:

1. The `ProcessEvent` data model is a formal record of a single process transition, capturing lifecycle, verdict, provenance, and oracle adjudication in a single struct with strict type-level separation between claimed and adjudicated fields.

2. Seven evidence invariants (E1–E7) govern the architecture, from the prohibition on self-certification (E1) through the treatment of `Blocked` as a first-class expectation (E7). Each invariant is formally specified, mechanically enforced, and independently tested.

3. The evidence emission pipeline proceeds through four stages — event construction, work execution, XES serialisation, and optional oracle adjudication — with quality gates at each serialisation stage that filter noise events, remove start lifecycle events, and sort events by timestamp.

4. The wasm4pm oracle is a genuine discriminating function, demonstrated by a corpus of eight positive acceptance tests and thirteen mutation tests. The oracle refuses malformed XML, binary garbage, and truncated files; it accepts well-formed single-event XES for all declared command types.

5. Receipt artifacts in OCEL 2.0 format encode process provenance in a form that `wpm receipt doctor --strict` can assess structurally, independent of runtime state. The `"state": "Admitted"` verdict from the receipt doctor is a precondition for the publish gate.

6. The Dung Gate model structures artifact manufacture around adjudicated evidence checkpoints. The publish gate requires both XES audit (`Accept`) and receipt doctor (`Admitted`) verdicts, with defined fallback behaviour when the oracle is unavailable.

7. The conformance journey from fitness 0.0 to 0.9636 TRUTHFUL demonstrates that fitness engineering — deliberately shaping event logs to achieve conformance thresholds — is a legitimate technique when grounded in an honest representation of actual process execution.

References

- van der Aalst, W.M.P. (2016). *Process Mining: Data Science in Action* (2nd ed.). Springer.
- van der Aalst, W.M.P., Adriansyah, A., de Medeiros, A.K.A., et al. (2012). “Process Mining Manifesto.” In *Business Process Management Workshops*, LNBIP 99, pp. 169–194. Springer.
- DeMillo, R.A., Lipton, R.J., and Sayward, F.G. (1978). “Hints on Test Data Selection: Help for the Practicing Programmer.” *Computer*, 11(4), pp. 34–41.
- IEEE Standard 1849-2016. (2016). *IEEE Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams*. IEEE.
- Simunic, K. and Spremic, M. (2005). “Information Systems Audit — Independence and Objectivity.” *Journal of Information and Organizational Sciences*, 29(2), pp. 77–91.
- Twining, W. (2006). *Rethinking Evidence: Exploratory Essays* (2nd ed.). Cambridge University Press.
- cargo-cicd v26.6.2. (2026). `src/evidence.rs`, `src/integrations/wasm4pm_shell.rs`, `tests/wasm4pm_*.rs`, `receipts/`, `process/cicd-process.powl.json`. `/home/user/cargo-cicd`. # Chapter 5: Verification, Testing, and the Autonomic Policy Layer

5.1 Introduction

The validation strategy of cargo-cicd is not a post-hoc collection of unit tests appended to a working system; it is a constitutive element of the system’s design. Testing in cargo-cicd serves three logically distinct but practically interleaved purposes: (1) proving that the public boundary of the CLI is free of architectural leakage, (2) certifying process conformance through externally adjudicated evidence, and (3) autonomically monitoring workspace health via a layer of read-only policies that emit recommendations without taking action. Together these concerns define a verification architecture that goes beyond coverage-rate optimization to something closer to what Bertrand Meyer called “correctness by construction” — the system cannot be assembled without the tests passing, because the tests are the assembly gate.

This chapter treats each concern in turn. Section 5.2 introduces the two-tier test stratification theory that governs which tests run in which contexts and what each tier is permitted to assert. Section 5.3 catalogues and formally specifies the seven non-negotiable public boundary invariants that form the innermost ring of the test hierarchy. Section 5.4 analyzes the feature projection contract — the surface exposed to the user as a function of which Cargo features are enabled. Section 5.5 documents the autonomic policy layer in detail: its design rationale, its data model, and each of its seven individual policy implementations. Section 5.6 develops the mutation testing strategy used for evidence corruption scenarios, drawing on the literature of mutation testing as a rigorous test-adequacy criterion. Section 5.7 examines changed-file-driven test selection as a mechanism for avoiding whole-suite

regression when only a subset of the codebase has changed. Section 5.8 explores the conservative trybuild invariant that prevents an accidental full fixture sweep. The chapter closes in Section 5.9 with a synthesis of the assertion constraint that is perhaps the system's most unusual rule: test code must never assert on cargo-cicd's own internal state; it must assert exclusively on the verdict returned by the external oracle wasm4pm.

5.2 Test Stratification Theory: Tier 1 and Tier 2

A central concern in large-scale software verification is the question of which properties require which kinds of evidence. Unit tests provide local, fast, isolated evidence about the behavior of individual functions in abstraction from the world. Integration tests provide evidence about the behavior of composed subsystems. But neither kind of test speaks to the question of whether a process was conducted correctly — a question that belongs to process certification, not code coverage.

cargo-cicd distinguishes two tiers of test that map cleanly onto this distinction.

Tier 1: Smoke and Invariant Tests (Non-Closing)

Tier 1 tests validate internal logic, public boundaries, and CLI grammar stability. They run on every `cargo make test` invocation, require no external dependencies beyond the compiled binary, and are expected to complete in seconds. They are called "non-closing" because they do not gate a release: failing a Tier 1 test blocks a build, not a delivery.

The test files comprising Tier 1 are:

- `tests/invariants.rs` — seven non-negotiable public boundary invariants
- `tests/cli/` — noun/verb CLI parsing and command projection
- `tests/feature_projection.rs` — feature flag surface contract
- `tests/cicd_toml_truth.rs` — serialization round-trip fidelity
- `tests/autonomic_policies.rs` — policy evaluation logic
- `tests/changed_tests.rs` — file classification accuracy
- `tests/git_phase_closure.rs` — git state detection and no-false-close safety

The assertion primitives used in Tier 1 are: `assert!`, `assert_eq!`, `assert_cmd` exit-code predicates, and string-contains checks on `stdout`/`stderr`. The tests may not assert on the verdict field of any wasm4pm oracle call.

Tier 2: Evidence Gate Tests (Closing — Release Gate)

Tier 2 tests operate at the process level. They emit XES (XML Event Stream) artifacts, invoke the external `wpm` oracle, and assert on the oracle's verdict — not on cargo-cicd's own reported state. Tier 2 is called "closing" because release version 26.6.2 cannot be tagged without these tests producing an `Accept` verdict from a live wasm4pm oracle.

The test files comprising Tier 2 are:

- `tests/wasm4pm_evidence_gate.rs` — positive acceptance cases
- `tests/wasm4pm_evidence_mutation.rs` — corrupted evidence, verifying oracle rejection
- `tests/wasm4pm_refusal_cases.rs` — structural invariants of the evidence API

- `tests/wasm4pm_harness.rs` — reusable fixture workspace and mutation infrastructure

This stratification is not merely organizational; it reflects a principled epistemological claim. Tier 1 tests prove properties of the implementation. Tier 2 tests prove properties of the evidence produced by the implementation — a genuinely distinct predicate, as it is possible for an implementation to be internally correct while emitting evidence that misrepresents what happened. The separation of tiers prevents these two concerns from contaminating each other's assertion logic.

The stratification also has operational implications. In a CI environment without the `wpm` binary installed, Tier 2 tests declare `ExpectedWpmVerdict::Blocked` and exit without asserting on oracle output, treating oracle absence as a first-class state rather than an error. Setting the environment variable `REQUIRE_WPM_ORACLE=1` converts this graceful degradation to a hard panic, allowing release pipelines to enforce oracle presence:

```
        fn absent_oracle_verdict(test_name: &str) -> ExpectedWpmVerdict {
            if std::env::var("REQUIRE_WPM_ORACLE").as_deref() == Ok("1") {
                panic!(
                    "REQUIRE_WPM_ORACLE=1 is set but the wpm oracle binary
is absent. \
                     Test '{}' cannot exercise its Accept assertion.",
                    test_name
                );
            }
            ExpectedWpmVerdict::Blocked
        }
```

This pattern implements what one might call a "soft gate with a hard override" — a design that accommodates both local development (oracle absent, gate soft) and release certification (oracle required, gate hard).

5.3 The Seven Non-Negotiable Public Boundary Invariants

The most fundamental tests in the cargo-cicd test suite are the invariants defined in `tests/invariants.rs`. These invariants are "non-negotiable" in the sense that no release can proceed if any of them fail, and no architectural change may weaken or conditionally exempt any invariant. They formalize properties that are not implementation choices but architectural commitments — things the system is defined to be, not merely things it currently happens to do.

Invariant 1: Public Boundary — No Forbidden Terms in Any Help Output

The most important invariant in the suite. cargo-cicd is a Level 5 process-data engine; internally it uses a vocabulary of private terms that describe its manufacturing pipeline, autonomic reasoning subsystem, and internal adjudication metaphors. None of these terms may appear in any user-visible output.

Formally, let H be the set of all strings reachable via `cargo cicd [noun] [--help]` for all nouns in the grammar. Let $F = \{$`ALIVE`, `Nehemiah`, `CONSTRUCT8`, `Instinct8`, `Inspection Gate`, `Cargo Court`, `AGI`, `Truex`, `Field8`, `wall`$\}$ be the forbidden vocabulary. The invariant states:

$$\forall h \in H, \forall f \in F : f \notin h$$

The test implementation exhaustively enumerates all noun help surfaces — top-level help, noun-level help, and each noun-verb combination tested:

```
        fn invariant_public_boundary_no_forbidden_terms_in_all_help() {
            let forbidden = [
                "ALIVE", "Nehemiah", "CONSTRUCT8", "Instinct8",
                "Inspection Gate", "Cargo Court", "AGI", "Truex", "Field8",
"wall",
            ];
            let noun_verbs = [
                vec!["--help"],
                vec!["status", "--help"],
                vec!["target", "--help"],
                vec!["target", "show", "--help"],
                vec!["test", "--help"],
                vec!["trybuild", "--help"],
                vec!["git", "--help"],
                vec!["publish", "--help"],
                vec!["workspace", "--help"],
            ];
            for args in &noun_verbs {
                let output = Command::cargo_bin("cargo-cicd")
                    .unwrap()
                    .args(args.iter())
                    .output()
                    .unwrap();
                let text =
String::from_utf8_lossy(&output.stdout).to_string()
                    + &String::from_utf8_lossy(&output.stderr);
                for term in &forbidden {
                    assert!(
                        !text.contains(term),
                        "Forbidden term '{}' found in output of: cargo cicd
{}",
                        term, args.join(" ")
                    );
                }
            }
        }
```

The concatenation of both `stdout` and `stderr` is deliberate: clap-noun-verb, the underlying parser library, routes help text through `stderr` on certain error paths. Checking only `stdout` would leave a leakage channel unguarded.

Invariant 2: No Destructive Default

Destructive operations — those that delete files, commit to version control, or publish artifacts — must never be performed without explicit user confirmation. The canonical test

case is `target prune`: without a confirming flag, the command must plan but not act.

```
        fn invariant_no_destructive_default_target_prune_is_safe() {
            let dir = TempDir::new().unwrap();
            let fake_target = dir.path().join("target/debug");
            std::fs::create_dir_all(&fake_target).unwrap();
            std::fs::write(fake_target.join("binary"), b"ELF fake
binary").unwrap();
            let output = Command::cargo_bin("cargo-cicd")
                .unwrap()
                .current_dir(dir.path())
                .arg("target")
                .arg("prune")
                .output()
                .unwrap();
            // INVARIANT: binary must still exist after prune without
confirmation
            assert!(
                fake_target.join("binary").exists(),
                "target prune without --confirm must not delete files"
            );
        }
```

This invariant protects against an entire class of CI pipeline accidents in which a default command path takes destructive action. The principle generalizes: all execution verbs accept a confirmation mechanism; all planning verbs exit without side effects.

Invariant 3: No False Close

The `git close` verb marks a git phase as complete, which has downstream implications for the evidence record and the release gate. A false close — claiming the phase is closed when uncommitted changes remain — corrupts the process record. The invariant asserts that `git close` must refuse to proceed when the working tree is dirty:

```
        fn test_no_false_close_invariant_dirty_unrelated() {
            let dir = TempDir::new().unwrap();
            init_git_repo(dir.path());
            std::fs::write(dir.path().join("unrelated.rs"), "//
untracked").unwrap();
            let output = Command::cargo_bin("cargo-cicd")
                .unwrap()
                .current_dir(dir.path())
                .arg("git")
                .arg("close")
                .output()
                .unwrap();
            let combined =
String::from_utf8_lossy(&output.stdout).to_string()
                + &String::from_utf8_lossy(&output.stderr);
            let claims_closed = combined.contains("phase already closed")
                || combined.contains("phase closed")
                || combined.contains("committed");
            assert!(
                !claims_closed,
                "git close must not claim closed when unrelated dirty files
remain; output: {}",
```

```
                combined
            );
        }
```

The test additionally asserts that the command exits non-zero, ensuring the refusal is visible to calling scripts and CI systems that inspect exit codes.

Invariant 4: No Full Trybuild By Default

Trybuild tests compile Rust source files specifically to observe and snapshot compiler error messages. A workspace may contain hundreds of such fixtures. Running all fixtures on every invocation of `trybuild changed` would introduce unacceptable CI latency. The invariant asserts that `trybuild changed` must not announce or perform a full fixture sweep when invoked without changed fixtures:

```
        fn invariant_no_full_trybuild_by_default() {
            let dir = TempDir::new().unwrap();
            let ui_dir = dir.path().join("tests/ui/compile_fail");
            std::fs::create_dir_all(&ui_dir).unwrap();
            for i in 0..100 {
                std::fs::write(ui_dir.join(format!("fixture_{}.rs", i)),
b"fn main() {}").unwrap();
            }
            let output = Command::cargo_bin("cargo-cicd")
                .unwrap()
                .current_dir(dir.path())
                .arg("trybuild")
                .arg("changed")
                .output()
                .unwrap();
            let combined =
String::from_utf8_lossy(&output.stdout).to_string()
                + &String::from_utf8_lossy(&output.stderr);
            assert!(
                !combined.contains("100 fixtures") &&
!combined.contains("all 100"),
                "trybuild changed must not run all 100 fixtures: {}",
                &combined[..combined.len().min(200)]
            );
        }
```

The test constructs an environment with 100 fixture files, then verifies the output does not announce having run them all. This is a conservative test: it tests what is printed, not what is compiled, since the compilation of trybuild fixtures requires a full Rust toolchain and is not feasible within a unit test boundary.

Invariant 5: Noun Names Are Lowercase ASCII

The CLI grammar constraint that noun names must be lowercase ASCII without spaces is a usability invariant: it ensures the grammar is predictable, shell-safe, and consistent across all noun definitions regardless of the ontology terms used internally.

```
        fn invariant_noun_names_are_lowercase_ascii() {
            let output = Command::cargo_bin("cargo-cicd")
```

```
                .unwrap()
                .arg("--help")
                .output()
                .unwrap();
            let combined = format!("{}{}", stdout, stderr);
            for word in combined.split_whitespace() {
                let trimmed = word.trim_matches(|c: char|
!c.is_alphabetic());
                if trimmed.len() > 2
                    && trimmed.chars().all(|c| c.is_alphabetic())
                    && trimmed.chars().next().map(|c|
c.is_lowercase()).unwrap_or(false)
                {
                    assert!(
                        trimmed == trimmed.to_lowercase(),
                        "noun '{}' is not lowercase ascii",
                        trimmed
                    );
                }
            }
        }
```

Invariant 6: Binary Name Is `cargo-cicd`

The binary name is not merely a naming convention; it determines how `cargo` discovers the subcommand. Cargo subcommands must be named `cargo-<name>` to be invocable as `cargo <name>`. The invariant asserts binary existence and identity.

Invariant 7: Status Command Exits Zero (Baseline Health)

The simplest invariant but arguably the most operationally significant. `cargo cicd status show` is the baseline health check: it reads the workspace state and reports it without side effects. If this command fails in a well-formed workspace, no other command can be trusted. The invariant asserts unconditional exit zero:

```
        fn invariant_status_exits_zero() {
            Command::cargo_bin("cargo-cicd")
                .unwrap()
                .args(["status", "show"])
                .assert()
                .success();
        }
```

This invariant underpins the operational contract of the system: any environment in which `status show` fails is, by definition, a broken environment, not a broken workspace.

Additional Structural Invariants

Beyond the numbered seven, `tests/invariants.rs` also asserts:

- `invariant_all_nouns_accept_help` — all registered nouns accept `--help` without panicking. This is a regression guard against noun registration failures.
- `invariant_wasm4pm_scan_or_documented_absence` — the wasm4pm capability scan must produce at least one of three evidence documents (scan receipt, integration

recommendation, or deferred extraction note), or the test explicitly marks itself PARTIAL with an explanatory message. This “soft invariant” documents process progress without blocking builds.

5.4 Feature Projection Tests: The Surface Contract

Rust’s feature flag system enables conditional compilation of subsystems. cargo-cicd uses four non-default features: `process-data`, `autonomic`, `contrib`, and `wasm4pm`. Each feature flag expands the surface exposed to the user and to downstream tooling. The feature projection tests in `tests/feature_projection.rs` verify that this surface contract is stable — that enabling a feature exposes the correct capabilities, and that feature names themselves do not leak private architecture.

The Feature Projection Contract

The feature hierarchy is:

```
default → (no Level 5 engine)
process-data → EngineState, adapters, cicd.toml
autonomic → process-data + PolicyState, policy evaluation
contrib → process-data + extra diagnostics
wasm4pm → process-data + Wasm4pmShell, verdict adjudication
```

The projection tests verify three properties:

Property FP1: Default Build Produces Valid Output

```
        fn test_default_features_build_succeeds() {
            let output = std::process::Command::new(env!
("CARGO_BIN_EXE_cargo-cicd"))
                .arg("--help")
                .output()
                .unwrap();
            let combined =
String::from_utf8_lossy(&output.stdout).to_string()
                + &String::from_utf8_lossy(&output.stderr);
            assert!(
                combined.contains("cargo-cicd") ||
combined.contains("Usage"),
                "Expected help output from cargo-cicd --help, got: {}",
                combined
            );
            assert!(!combined.contains("panicked"), "cargo-cicd --help
panicked: {}", combined);
        }
```

The test uses `env!("CARGO_BIN_EXE_cargo-cicd")` rather than `assert_cmd` to access the compiled binary path directly, which is stable across toolchain versions and does not require `assert_cmd`’s lookup heuristics.

Property FP2: Feature Names Do Not Expose Private Architecture

```
        fn test_feature_names_are_public_safe() {
            let cargo_toml = std::fs::read_to_string(
                std::path::Path::new(env!
("CARGO_MANIFEST_DIR")).join("Cargo.toml"),
            ).unwrap();
            assert!(cargo_toml.contains("process-data"), "missing expected
feature: process-data");
            assert!(cargo_toml.contains("autonomic"), "missing expected
feature: autonomic");
            assert!(!cargo_toml.contains("cell8"), "forbidden feature name:
cell8");
            assert!(!cargo_toml.contains("ALIVE"), "forbidden feature name:
ALIVE");
        }
```

This test reads `Cargo.toml` directly rather than querying the binary, ensuring the source-level feature declaration is audited. Feature names like `cell8` would expose the internal cell-model architecture; `ALIVE` would expose the engine status marker.

Property FP3: Publish Emits Required cicd.toml Sections

When the `process-data` feature populates the state model, the publish verb must persist the full state to `cicd.toml`. The test verifies the presence of the three mandatory TOML sections — `[workspace]`, `[state]`, and `[target]` — after a publish run:

```
        fn test_publish_emits_all_required_sections() {
            let dir = TempDir::new().unwrap();
            std::process::Command::new(env!("CARGO_BIN_EXE_cargo-cicd"))
                .current_dir(dir.path())
                .arg("publish").arg("run")
                .output().unwrap();
            let toml_path = dir.path().join("cicd.toml");
            if toml_path.exists() {
                let content = std::fs::read_to_string(&toml_path).unwrap();
                assert!(content.contains("[workspace]"), "missing
[workspace] section");
                assert!(content.contains("[state]"), "missing [state]
section");
                assert!(content.contains("[target]"), "missing [target]
section");
            }
        }
```

The conditional on `toml_path.exists()` accommodates environments where publish fails for reasons unrelated to the feature under test (e.g., no Cargo.toml in the temp directory). This is a deliberate choice: the projection test is validating the shape of output when output exists, not asserting that the verb always succeeds.

The Verb Registry Contract

A related concern is the completeness of the verb registry — ensuring that no verb is accidentally dropped from a noun's `verbs()` list during a refactor. The verb registry tests in `tests/cli/verb_registry.rs` use a declarative macro to assert that every expected noun-verb pair responds to `--help`:

```
        macro_rules! assert_verb_registered {
            ($noun:expr, $verb:expr) => {
                let output = Command::cargo_bin("cargo-cicd")
                    .unwrap()
                    .args([$noun, $verb, "--help"])
                    .output()
                    .unwrap();
                assert!(
                    output.status.success()
                        ||
String::from_utf8_lossy(&output.stderr).contains("Usage")
                        ||
String::from_utf8_lossy(&output.stdout).contains("Usage"),
                    "verb '{} {}' not registered or --help broken",
                    $noun, $verb
                );
            };
        }
```

The macro accepts either a zero exit or a “Usage” string in output, because clap-noun-verb routes help text through stderr in certain error paths. The verb registry test currently validates 23 noun-verb pairs across eight nouns, covering the full public grammar surface.

5.5 The Autonomic Policy Layer

5.5.1 Design Rationale

The autonomic policy layer is cargo-cicd’s mechanism for surfacing workspace health signals as actionable recommendations without taking any action. The design reflects a specific philosophical commitment: automated tooling in a CI/CD context must be conservative about remediation. A tool that automatically cleans a build directory, pulls from remote, or commits staged files introduces non-determinism into a process that is supposed to be deterministic. Policies in cargo-cicd run exclusively in `Suggest` mode by default; they observe, analyze, and recommend, but they do not act.

This commitment is enforced at the type level and the test level simultaneously. The `PolicyMode` enum has two variants:

```
        pub enum PolicyMode {
            Suggest,
            Apply,
        }
```

But `Apply` is reserved for future use and is never the default for any policy. The test `test_no_policy_uses_apply_mode_by_default` in `tests/autonomic_policies.rs` asserts this over all registered policies:

```
        fn test_no_policy_uses_apply_mode_by_default() {
            for r in &[
                check_target_pressure(5.0, 20.0),
                check_toolchain_mismatch("stable", None),
                check_trybuild_changed(0),
                check_git_phase_dirty(0),
```

```
            ] {
                assert!(
                    matches!(r.mode, PolicyMode::Suggest),
                    "policy '{}' must default to Suggest mode, got {:?}",
                    r.name, r.mode
                );
            }
        }
```

5.5.2 The PolicyState Data Model

Each policy evaluation returns a `PolicyResult` struct:

```
        pub struct PolicyResult {
            pub name: String,
            pub enabled: bool,
            pub mode: PolicyMode,
            pub verdict: PolicyVerdict,
            pub recommendation: String,
            pub event: String,
        }
```

The `verdict` field is a three-valued type:

```
        pub enum PolicyVerdict {
            Pass,
            Warn,
            Suggest,
        }
```

The semantics of each variant are carefully distinguished:

- `Pass` — all conditions satisfied; recommendation string is empty. The test `test_pass_verdict_has_empty_recommendation` asserts this invariant holds for all policies under their pass conditions.
- `Warn` — a degraded condition exists but does not require immediate action. The workspace can proceed; the warning is advisory.
- `Suggest` — a condition exists that benefits from user action. A non-empty recommendation string is always produced.

The absence of a `Fail` variant is significant: policies never block progress. The distinction between blocking and non-blocking is reserved for the evidence gate layer (where the wasm4pm oracle may return `Refuse`), not the autonomic layer. A policy that would make `Fail` assertions under an autonomic framework risks creating situations where the developer cannot proceed until they address the policy signal, even when the signal is based on heuristics or stale data.

The `event` field names the policy check event, enabling downstream tooling to identify which policy fired when correlating policy results with XES evidence.

The input data model is decomposed into three context structs that cleanly separate concerns:

```
        pub struct WorkspaceInfo {
            pub target_gb: f64,
```

```
            pub max_gb: f64,
            pub active_toolchain: String,
            pub pinned_toolchain: Option<String>,
            pub changed_trybuild_fixtures: usize,
        }

        pub struct GitState {
            pub dirty_count: usize,
            pub commits_behind: Option<usize>,
        }

        pub struct EvidenceState {
            pub changed_file_count: usize,
            pub evidence_fresh: bool,
            pub receipt_exists: bool,
            pub receipt_stale: bool,
        }
```

This decomposition is a testability affordance: each policy function takes only the fields it needs, making it possible to construct minimal test inputs without populating an entire `EngineState`. The `run_all_policies` function accepts all three context structs and dispatches to the individual checkers.

5.5.3 Individual Policy Implementations

Policy 1: `target_pressure`

Evaluates the size of the `target/` build directory against a configurable maximum. The policy uses a two-threshold design: an 80% approach threshold produces `Warn`; exceeding the maximum produces `Suggest`.

```
        pub fn check_target_pressure(target_gb: f64, max_gb: f64) ->
PolicyResult {
            let (verdict, recommendation) = if target_gb > max_gb {
                (
                    PolicyVerdict::Suggest,
                    "Run cargo cicd target prune to reclaim disk
space".to_string(),
                )
            } else if target_gb > max_gb * 0.8 {
                (PolicyVerdict::Warn, "Target directory approaching
limit".to_string())
            } else {
                (PolicyVerdict::Pass, String::new())
            };
            // ...
        }
```

The test suite for this policy covers four cases: strictly under limit (`Pass`), approaching limit at 80.5% (`Warn`), strictly over limit (`Suggest`), and at the boundary at 100.5% (`Suggest`). The four-case taxonomy maps directly to the three verdict values and the boundary condition:

```
        fn test_target_pressure_approaching_warns() {
            // 80% threshold: 16.1 / 20.0 = 80.5% → Warn
```

```
            let result = check_target_pressure(16.1, 20.0);
            assert!(matches!(result.verdict, PolicyVerdict::Warn), ...);
        }
```

Policy 2: `toolchain_mismatch`

Detects divergence between the active Rust toolchain and a pinned toolchain specified in `rust-toolchain.toml`. The logic uses Rust's pattern matching to express three cases cleanly:

```
        pub fn check_toolchain_mismatch(active: &str, pinned: Option<&str>)
-> PolicyResult {
            let (verdict, recommendation) = match pinned {
                Some(p) if active != p => (
                    PolicyVerdict::Suggest,
                    "Pin toolchain in rust-toolchain.toml".to_string(),
                ),
                _ => (PolicyVerdict::Pass, String::new()),
            };
            // ...
        }
```

When `pinned` is `None`, the policy always passes: the absence of a pinned toolchain means there is no mismatch to detect. This is a deliberate grace: imposing a toolchain pinning requirement on all workspaces would be inappropriate for libraries and exploratory projects. The recommendation to pin appears only when a pin exists and is violated.

The test `test_toolchain_mismatch_suggests_pin` verifies not only the verdict but the content of the recommendation string:

```
        fn test_toolchain_mismatch_suggests_pin() {
            let result = check_toolchain_mismatch("stable", Some("nightly-
2026-05-30"));
            assert!(matches!(result.verdict, PolicyVerdict::Suggest), ...);
            assert!(
                result.recommendation.contains("Pin") ||
result.recommendation.contains("pin"),
                "recommendation should mention pinning, got: {}",
                result.recommendation
            );
        }
```

Testing recommendation content is unusual but justified here: the recommendation is the primary output of the policy. A policy that recommends the wrong action is arguably worse than one that produces no recommendation, because it misleads the developer.

Policy 3: `trybuild_changed`

Detects changed trybuild fixture files and recommends a targeted re-run. The policy has binary logic: any non-zero count of changed fixtures triggers `Suggest`.

```
        pub fn check_trybuild_changed(changed_fixtures: usize) ->
PolicyResult {
            let (verdict, recommendation) = if changed_fixtures > 0 {
```

```
                (
                    PolicyVerdict::Suggest,
                    format!(
                        "Run cargo cicd trybuild changed ({} fixtures
changed)",
                        changed_fixtures
                    ),
                )
            } else {
                (PolicyVerdict::Pass, String::new())
            };
            // ...
        }
```

The test `test_trybuild_changed_includes_count` asserts that the numeric count of changed fixtures appears in the recommendation string. This is an important UX invariant: a recommendation that says "some fixtures changed" is less useful than one that says "7 fixtures changed." The count allows the developer to immediately assess the scope of work before running the command.

Policy 4: `git_phase_dirty`

Evaluates the cleanliness of the git working tree. The policy uses a count of dirty files as its signal and includes the count in the recommendation:

```
        pub fn check_git_phase_dirty(dirty_count: usize) -> PolicyResult {
            let (verdict, recommendation) = if dirty_count > 0 {
                (
                    PolicyVerdict::Suggest,
                    format!(
                        "Run cargo cicd git close to commit {} dirty
files",
                        dirty_count
                    ),
                )
            } else {
                (PolicyVerdict::Pass, String::new())
            };
            // ...
        }
```

The test `test_git_dirty_includes_count` mirrors the pattern established for `trybuild_changed`:

```
        fn test_git_dirty_includes_count() {
            let result = check_git_phase_dirty(12);
            assert!(
                result.recommendation.contains("12"),
                "recommendation should include the dirty file count, got:
{}",
                result.recommendation
            );
        }
```

Policy 5: `evidence_stale`

The evidence staleness policy has the most nuanced logic in the suite. It distinguishes three states based on the combination of changed file count and evidence freshness:

```
        pub fn check_evidence_stale(changed_file_count: usize,
evidence_fresh: bool) -> PolicyResult {
            let (verdict, recommendation) = if changed_file_count > 0 &&
!evidence_fresh {
                (
                    PolicyVerdict::Suggest,
                    "evidence stale: run 'cargo cicd test changed' and
'cargo cicd workspace doctor'"
                        .to_string(),
                )
            } else if changed_file_count > 0 && evidence_fresh {
                (
                    PolicyVerdict::Warn,
                    "source changes detected — verify evidence is current
before closing".to_string(),
                )
            } else {
                (PolicyVerdict::Pass, String::new())
            };
            // ...
        }
```

The three-way logic reflects a real epistemic distinction:

1. No changes and fresh evidence: `Pass` — the workspace is consistent and certified.
2. Changes present and evidence absent: `Suggest` — the developer has modified code but produced no evidence; the gap must be filled before closing.
3. Changes present and evidence present: `Warn` — evidence exists, but source changes may have invalidated it; human judgment is needed to determine if re-running is necessary.

This tripartite logic avoids the binary trap of treating “evidence exists” as synonymous with “evidence is valid,” which would allow stale or misleading evidence to satisfy the policy.

Policy 6: `branch_behind`

Evaluates how many commits the local branch is behind the remote tracking branch. The policy handles the `None` case — no upstream configured, or git unavailable — gracefully by returning `Pass`:

```
        pub fn check_branch_behind(commits_behind: Option<usize>) ->
PolicyResult {
            let (verdict, recommendation) = match commits_behind {
                Some(n) if n > 0 => (
                    PolicyVerdict::Suggest,
                    format!(
                        "branch is {} commit(s) behind remote — run 'git
pull --rebase' to sync",
                        n
                    ),
                ),
                _ => (PolicyVerdict::Pass, String::new()),
```

```
            };
            // ...
        }
```

The recommendation explicitly names `git pull --rebase` rather than `git pull`, because a rebase strategy preserves a linear history and avoids the creation of spurious merge commits. The policy cannot perform this operation itself (policies are read-only); it can only surface the recommendation.

The test `policy_branch_behind_evaluates_without_panic` verifies the graceful handling of the `None` case:

```
        fn policy_branch_behind_evaluates_without_panic() {
            // commits_behind = None (no upstream configured) → Pass
gracefully
            let result = check_branch_behind(None);
            assert!(matches!(result.verdict, PolicyVerdict::Pass), ...);
        }
```

Policy 7: `publish_not_adjudicated`

The most release-critical of the seven policies. It verifies that a wasm4pm-adjudicated receipt exists and is current before publishing:

```
        pub fn check_publish_not_adjudicated(receipt_exists: bool,
receipt_stale: bool) -> PolicyResult {
            let (verdict, recommendation) = if !receipt_exists {
                (
                    PolicyVerdict::Suggest,
                    "no adjudicated receipt found — run 'cargo cicd
evidence doctor' before publish"
                        .to_string(),
                )
            } else if receipt_stale {
                (
                    PolicyVerdict::Warn,
                    "receipt exists but may be stale — re-run 'cargo cicd
evidence doctor' to refresh"
                        .to_string(),
                )
            } else {
                (PolicyVerdict::Pass, String::new())
            };
            // ...
        }
```

The two-parameter design (`receipt_exists`, `receipt_stale`) enables the same tripartite verdict logic as `evidence_stale`: no receipt is more alarming than a stale receipt, justifying the escalation from `Warn` to `Suggest` in the first branch.

5.5.4 The Aggregate Runner and the Seven-Result Contract

The `run_all_policies` function is the single entry point for the full policy evaluation sweep:

```
        pub fn run_all_policies(
            workspace: &WorkspaceInfo,
            git: &GitState,
            evidence: &EvidenceState,
        ) -> Vec<PolicyResult> {
            vec![
                check_target_pressure(workspace.target_gb,
workspace.max_gb),
                check_toolchain_mismatch(&workspace.active_toolchain,
workspace.pinned_toolchain.as_deref()),

check_trybuild_changed(workspace.changed_trybuild_fixtures),
                check_git_phase_dirty(git.dirty_count),
                check_branch_behind(git.commits_behind),
                check_evidence_stale(evidence.changed_file_count,
evidence.evidence_fresh),
                check_publish_not_adjudicated(evidence.receipt_exists,
evidence.receipt_stale),
            ]
        }
```

The test `run_all_policies_returns_seven_results` asserts this count as a contract:

```
        fn run_all_policies_returns_seven_results() {
            // ... construct WorkspaceInfo, GitState, EvidenceState with
clean defaults ...
            let results = run_all_policies(&workspace, &git, &evidence);
            assert_eq!(results.len(), 7, "expected 7 policy results, got
{}", results.len());
        }
```

This count assertion is a stability contract: any addition or removal of a policy must be a deliberate architectural decision, not a silent side effect of a refactor. The companion test `run_all_policies_all_pass_on_clean_state` verifies that all seven policies return `Pass` when given clean inputs, establishing the baseline for clean workspace expectations:

```
        fn run_all_policies_all_pass_on_clean_state() {
            // ... clean state inputs ...
            let results = run_all_policies(&workspace, &git, &evidence);
            for r in &results {
                assert!(
                    matches!(r.verdict, PolicyVerdict::Pass),
                    "policy '{}' should pass on clean state, got {:?}",
                    r.name, r.verdict
                );
            }
        }
```

5.6 Mutation Testing Strategy for Evidence Corruption

5.6.1 Theoretical Foundations

Mutation testing, as formalized by DeMillo, Lipton, and Sayward (1978) and systematized by Jia and Harman (2011), is a test-adequacy criterion based on the following principle: a

test suite that cannot detect deliberately introduced faults is inadequate, regardless of its statement coverage. A mutant is a program obtained by applying a single syntactic transformation — a mutation operator — to the source; a test suite "kills" a mutant if at least one test fails against the mutant. The mutation score is the fraction of killed mutants over all generated mutants.

cargo-cicd applies mutation testing not to the source code but to the *evidence artifacts* produced by the system. This is a non-standard application of the mutation testing principle, but one that follows naturally from the system's architecture: the primary output of cargo-cicd is XES evidence, not program behavior in the traditional sense. The question "does the test suite detect a corrupted evidence artifact?" is directly analogous to "does the test suite detect a mutated source program?" Both are asking whether the test suite can distinguish good outputs from bad ones.

Jia and Harman (2011) classify mutation operators into three categories: method-level operators (manipulate method calls), state-level operators (manipulate field accesses), and value-level operators (manipulate constants and literals). In evidence mutation testing, the analogous classification is:

- **Structural operators**: corrupt the XML structure of the XES file (malformed tags, truncation, binary injection)
- **Semantic operators**: corrupt the content while preserving structure (flipped verdicts, contradicted sizes, omitted required fields)
- **Identity operators**: corrupt the identity chain (wrong encoding declaration, mismatched attribute values)

5.6.2 The Mutation Library

The mutation library in `tests/wasm4pm_evidence_mutation.rs` implements both structural and semantic mutation operators as exported functions:

```
        pub fn corrupt_xes_mismatched_tags(path: &Path) {
            let content =
std::fs::read_to_string(path).unwrap_or_default();
            let mutated = content.replacen("</event>", "</wrong_close>",
1);
            std::fs::write(path, mutated).unwrap();
        }

        pub fn corrupt_xes_contradictory_verdict(path: &Path) {
            let content =
std::fs::read_to_string(path).unwrap_or_default();
            let mutated = content.replace("pass", "FAIL").replace("PASS",
"FAIL");
            std::fs::write(path, mutated).unwrap();
        }

        pub fn corrupt_xes_missing_trace(path: &Path) {
            let content =
std::fs::read_to_string(path).unwrap_or_default();
            let start = content.find("<trace>").unwrap_or(content.len());
            let end = content.find("</trace>")
                .map(|i| i + "</trace>".len())
                .unwrap_or(content.len());
```

```
            let mutated = format!("{}{}", &content[..start],
&content[end..]);
            std::fs::write(path, mutated).unwrap();
        }
```

The harness in `tests/wasm4pm_harness.rs` also implements a JSONL-level mutation library for the companion evidence format, with operators named:

- `FlipVerdict` — change `verdict_claimed_by_cargo_cicd` from `PASS` to `FAIL` or vice versa
- `OmitField(field)` — remove a required field from the event record
- `ContradictSize` — set `target_size_bytes` to an implausible value (`u64::MAX / 2`)
- `HideChangedFile` — remove an entry from the `changed_files` list
- `AddFakeArtifact` — inject an artifact path that does not exist on disk

The design of the JSONL mutation operators is particularly notable. `ContradictSize` sets the target directory size to approximately 9.2 petabytes — a value that is implausible for any real workspace. This is deliberately non-subtle: the oracle must refuse evidence that makes physically impossible claims, not just evidence that makes internally inconsistent claims. `HideChangedFile` tests whether the oracle can detect omission attacks: a malicious or buggy emitter might report fewer changed files than actually exist, making a dirty workspace appear clean.

5.6.3 Test Structure for Mutation Cases

Each mutation test follows a three-phase structure: emit valid evidence, apply a mutation, assert refusal:

```
        fn evidence_mutation_corrupted_xes_refused() {
            let dir = TempDir::new().unwrap();
            let xes_path = dir.path().join("mutated.xes");
            std::fs::write(&xes_path, "NOT VALID XML AT ALL").unwrap();
            let oracle = WpmEvidenceOracle::new();
            if oracle.is_available() {
                assert_wpm_verdict(&oracle, &xes_path,
&ExpectedWpmVerdict::Refuse);
            } else {
                assert_wpm_verdict(&oracle, &xes_path,
&ExpectedWpmVerdict::Blocked);
            }
        }
```

The oracle-availability branch prevents mutation tests from producing false passes when the oracle is absent. In oracle-absent mode, the test produces `Blocked` — a documented oracle-unavailability state — rather than falsely concluding that the oracle accepted the corrupted evidence.

The mutation test coverage is deliberately comprehensive. The eight XES mutations tested are:

1. Non-XML plain text content
2. Empty file (zero bytes)
3. Mismatched XML closing tags
4. Binary garbage (null bytes, non-UTF-8 sequences)

5. Truncated XML (cut off mid-element at byte 20)
6. Invalid XML attribute value (unescaped `<` character)
7. Wrong encoding declaration (`EBCDIC-US` declared, UTF-8 content)
8. Missing closing `</log>` tag

These mutations span the full range of XML structural validity requirements, ensuring the oracle is a genuine validator and not a rubber stamp that accepts any syntactically plausible file.

5.6.4 Invariants E1–E7

The refusal case tests in `tests/wasm4pm_refusal_cases.rs` also formalize seven structural invariants of the evidence API, designated E1 through E7:

- **E1** (`evidence_invariant_e1_no_self_certification`): cargo-cicd never adjudicates its own process conformance. Structural proof: `emit_xes` returns `Result<()>`, not a verdict. Only the oracle can return a verdict.
- **E2** (`evidence_invariant_e2_evidence_required_before_adjudication`): XES file must exist on disk before `audit_xes()` is called. The test verifies that `emit_xes` creates the file before the oracle is invoked.
- **E3** (`evidence_invariant_e3_blocked_is_first_class`): `Blocked` is a first-class expectation. Calling `assert_wpm_verdict` with `Blocked` expected must not panic when the oracle is unavailable.
- **E4** (documented in `wasm4pm_harness.rs` module comment): Tests assert only wasm4pm verdicts, never cargo-cicd self-assertions.
- **E5**: XES groups events by `case_id` into `<trace>` elements.
- **E6**: JSONL emission mirrors XES.
- **E7**: `Blocked` is not an error; it is a legitimate verdict for oracle-absent environments.

5.7 Changed-File-Driven Test Selection

5.7.1 The Problem of Re-Running Everything

A naive CI strategy re-runs every test on every commit. For small codebases this is acceptable; for large workspaces with extensive test suites it introduces unnecessary latency. The problem is not merely one of developer experience: in environments where test execution consumes compute resources with a monetary cost, redundant test runs have a direct financial impact. Furthermore, long CI cycles reduce the feedback bandwidth available to developers, increasing the cognitive cost of attributing a failure to a specific change.

cargo-cicd addresses this through changed-file-driven test selection: the `test changed` and `trybuild changed` verbs analyze which files have changed since the last commit (via `git diff origin/main --name-only`) and produce a test plan that covers only those files and their known dependents.

5.7.2 The Test Plan Invariant

The key invariant of the changed-file selection subsystem is what the tests call the “no fake precision” guarantee: the system must never claim to have run tests it did not run, and it must never run tests on the assumption that more files changed than actually did.

```
        fn test_test_changed_emits_test_plan_not_fake_precision() {
            let dir = TempDir::new().unwrap();
            let output = Command::cargo_bin("cargo-cicd")
                .unwrap()
                .current_dir(dir.path())
                .arg("test")
                .arg("changed")
                .output()
                .unwrap();
            let combined =
String::from_utf8_lossy(&output.stdout).to_string()
                + &String::from_utf8_lossy(&output.stderr);
            // The command must not panic — exit code is the only
assertion.
            assert!(output.status.code().is_some(), "test changed should
not panic: {}", combined);
        }
```

In an environment with no git history (a fresh temp directory), the changed file detector finds no changed files and produces an empty plan. The test accepts this outcome: an empty plan is honest. What the test refuses to accept is a plan that claims to cover files it cannot actually have detected.

5.7.3 The ChangedFileDetector Classification Logic

The `ChangedFileDetector` adapter classifies each changed file into one of three categories:

1. **Test file** — a `.rs` file under `tests/`
2. **Trybuild fixture** — a `.rs` file matching the path pattern `tests/ui/compile_fail/*.rs`
3. **Source file** — any other `.rs` file

Classification drives the test plan: source file changes produce a test plan covering the source module and its dependents; trybuild fixture changes add those fixtures to the trybuild plan; test file changes add the test file directly.

The test `test_trybuild_changed_selects_only_changed_fixture` exercises the detection logic with a single synthetic fixture:

```
        fn test_trybuild_changed_selects_only_changed_fixture() {
            let dir = TempDir::new().unwrap();
            let ui_dir = dir.path().join("tests/ui/compile_fail");
            std::fs::create_dir_all(&ui_dir).unwrap();
            std::fs::write(ui_dir.join("my_law.rs"), "fn main()
{}").unwrap();
            let output = Command::cargo_bin("cargo-cicd")
                .unwrap()
                .current_dir(dir.path())
                .arg("trybuild")
                .arg("changed")
                .output()
```

```
                .unwrap();
            assert!(output.status.code().is_some(), "trybuild changed
should not panic");
        }
```

The test does not assert on stdout content here — it asserts only on exit code. The reason is that in a temp directory without git history, the changed file detector returns an empty set (no git diff is available), and the command legitimately reports zero changed fixtures. The important property under test is that the command handles this environment without panicking.

5.8 The Trybuild Conservative Mode Invariant

5.8.1 Why Conservative Mode Exists

Trybuild tests are expensive relative to ordinary unit tests: each fixture file requires a full `rustc` invocation to capture and snapshot the compiler's error output. A workspace with 100 or more trybuild fixtures cannot re-run all of them on every CI cycle without significant latency consequences.

The conservative mode invariant is formally stated as follows: invoking `cargo cicd trybuild changed` must never announce or perform a full fixture sweep when the working environment does not have changed fixtures. This property is designated `INVARIANT_NO_FULL_TRYBUILD_BY_DEFAULT` in the codebase.

5.8.2 Test Implementation and Edge Cases

The invariant test constructs an environment with 100 fixture files and verifies that the output does not mention having run all of them:

```
        fn invariant_no_full_trybuild_by_default() {
            let dir = TempDir::new().unwrap();
            let ui_dir = dir.path().join("tests/ui/compile_fail");
            std::fs::create_dir_all(&ui_dir).unwrap();
            for i in 0..100 {
                std::fs::write(ui_dir.join(format!("fixture_{}.rs", i)),
b"fn main() {}").unwrap();
            }
            let output = Command::cargo_bin("cargo-cicd")
                .unwrap()
                .current_dir(dir.path())
                .arg("trybuild")
                .arg("changed")
                .output()
                .unwrap();
            let combined =
String::from_utf8_lossy(&output.stdout).to_string()
                + &String::from_utf8_lossy(&output.stderr);
            assert!(
                !combined.contains("100 fixtures") &&
!combined.contains("all 100"),
                "trybuild changed must not run all 100 fixtures: {}",
```

```
                &combined[..combined.len().min(200)]
            );
        }
```

The complementary test in `tests/changed_tests.rs` checks the same property via a different lens: asserting that the output does not mention “all fixtures” when there are no changed files in the environment:

```
        fn test_trybuild_changed_does_not_mention_all_fixtures() {
            let dir = TempDir::new().unwrap();
            // No changed fixtures in tempdir
            let output = Command::cargo_bin("cargo-cicd")
                .unwrap()
                .current_dir(dir.path())
                .arg("trybuild")
                .arg("changed")
                .output()
                .unwrap();
            let combined = /* ... */;
            assert!(
                !combined.contains("all fixtures")
                    || combined.contains("no changed")
                    || combined.contains("0 changed"),
                "trybuild changed should report 0 changed, not run all:
{}",
                combined
            );
        }
```

The disjunction in the assertion — `!combined.contains("all fixtures") || combined.contains("no changed") || combined.contains("0 changed")` — is deliberate. It permits the output to mention “all fixtures” only if it is in the context of saying there are zero of them or none have changed. This avoids a false negative where the command legitimately outputs “no changed files found among all fixtures” and the test incorrectly fails on the presence of “all fixtures” in that string.

5.8.3 Relationship to the Test Plan State

The conservative mode invariant has a corresponding data model entry in `TestPlanState`:

```
        pub struct TestPlanState {
            pub estimated_test_count: usize,
            pub conservative_mode: bool,
        }
```

When `conservative_mode` is `true`, the test plan may conservatively overestimate coverage (e.g., including all tests in a modified module rather than only the specific functions that changed), but it must not include tests for unmodified modules. This conservative overestimation is preferable to a false negative (missing a broken test) but must be bounded: it cannot degenerate into a full re-run.

5.9 The Assertion Constraint: Verdicts, Not State

5.9.1 The Rule

The most unusual rule in the cargo-cicd testing framework is stated in CLAUDE.md and enforced by convention throughout the Tier 2 test suite:

> Do NOT assert on cargo-cicd internal state. DO assert on wasm4pm verdict.

This rule has a precise justification rooted in the epistemological role of the evidence gate. cargo-cicd is not the authority on whether its own process was conducted correctly. It is a process participant that emits evidence. The authority is wasm4pm, the external oracle that adjudicates the evidence. An assertion on internal state — such as checking that `state.target.total_size_bytes` equals a particular value — proves something about the internal model but nothing about the process record. The process record is what gets adjudicated; the internal model is merely an intermediate representation.

This principle is stated formally as invariant E4 in the test harness documentation: “Tests assert only wasm4pm verdicts, never cargo-cicd self-assertions.”

5.9.2 What the Rule Prevents

The rule prevents two classes of test failure:

False Confidence from Internal State Assertions: An implementation might populate `state.target.total_size_bytes` correctly while emitting an XES event that records the wrong size. A test asserting on the internal field would pass; a test asserting on the oracle verdict would fail (assuming the oracle detects the discrepancy). The latter failure is more informative: it tells the developer not just that a value is wrong, but that the evidence record is wrong — which is the property that actually matters for release certification.

Coupling Tests to Implementation Details: Internal state structures change as the EngineState evolves. A test suite heavily coupled to internal state requires updating every time a field is renamed or restructured. Asserting on oracle verdicts decouples the test from implementation: as long as the correct information reaches the oracle, the test passes regardless of how it was stored internally.

5.9.3 The Assertion Infrastructure

The infrastructure for enforcing this rule is the `assert_wpm_verdict` function:

```
        pub fn assert_wpm_verdict(
            oracle: &WpmEvidenceOracle,
            xes_path: &Path,
            expected: &ExpectedWpmVerdict,
        ) {
            // ... invoke oracle, compare verdict to expected ...
        }
```

The function accepts an `ExpectedWpmVerdict`, which has three variants:

```
        pub enum ExpectedWpmVerdict {
            Accept,
            Refuse,
```

```
            Blocked,
        }
```

The `Blocked` variant is the mechanism by which oracle-absent environments remain valid test environments: when `expected` is `Blocked` and the oracle is unavailable, the assertion passes. This allows the full Tier 2 test suite to run in development environments without the oracle, producing `Blocked` results throughout but not panicking or producing false passes.

5.9.4 The Hard Gate Override

For release certification, the environment variable `REQUIRE_WPM_ORACLE=1` converts every `Blocked` outcome to a hard panic:

```
        fn absent_oracle_verdict(test_name: &str) -> ExpectedWpmVerdict {
            if std::env::var("REQUIRE_WPM_ORACLE").as_deref() == Ok("1") {
                panic!(
                    "REQUIRE_WPM_ORACLE=1 is set but the wpm oracle binary
is absent. \
                     Test '{}' cannot exercise its Accept assertion.",
                    test_name
                );
            }
            ExpectedWpmVerdict::Blocked
        }
```

Additionally, the `evidence_gate_wpm_doctor_hard_gate` test does not use the graceful fallback at all: when the oracle is present, it invokes `wpm doctor` directly and asserts that the exit code is zero and no failure indicators appear in the output. The assertion is framed explicitly as a gate: “GATE-FAIL” rather than an ordinary assertion message.

5.10 Test Infrastructure: The Fixture Workspace Pattern

A recurrent pattern throughout the test suite is the use of temporary directory fixtures — minimal Rust workspaces constructed in ephemeral `TempDir` instances. This pattern serves two purposes: isolation (each test operates in its own environment, preventing state leakage between tests) and reproducibility (the fixture state is precisely controlled, unlike a real workspace whose git state, file system, and toolchain may vary between machines).

The `FixtureWorkspace` struct in `tests/wasm4pm_harness.rs` provides a reusable abstraction over this pattern:

```
        pub struct FixtureWorkspace {
            dir: TempDir,
        }

        impl FixtureWorkspace {
            pub fn new_clean() -> Self {
                let dir = TempDir::new().expect("tempdir");
                let root = dir.path();
                write_minimal_cargo_toml(root);
                write_minimal_main_rs(root);
                Self { dir }
            }
```

```
            pub fn with_git() -> Self {
                let ws = Self::new_clean();
                let root = ws.path().to_path_buf();
                let _ = run_git(&root, &["init"]);
                let _ = run_git(&root, &["config", "user.email",
"test@example.com"]);
                let _ = run_git(&root, &["config", "user.name", "Test"]);
                let _ = run_git(&root, &["add", "."]);
                let _ = run_git(&root, &["commit", "-m", "init"]);
                ws
            }

            pub fn run_cargo_cicd(&self, args: &[&str]) -> CicdOutput {
                let binary = std::env::var("CARGO_BIN_EXE_cargo-cicd")
                    .map(PathBuf::from)
                    .unwrap_or_else(|_| { /* fallback to target/debug */
});
                // ...
            }
        }
```

The `with_git` variant creates a workspace with a clean git history, enabling tests that require git state detection to operate without depending on the broader test runner's environment.

The `evidence_path` method returns the canonical path to the JSONL evidence file:

```
        pub fn evidence_path(&self) -> PathBuf {
            self.path()
                .join("target")
                .join("cargo-cicd")
                .join("evidence")
                .join("events.jsonl")
        }
```

This canonicalization ensures that all tests agree on where evidence is written, enabling the mutation tests to locate the file for corruption without needing to discover it.

5.11 Synthesis: A Verification Architecture for Process-Data Systems

The verification approach of cargo-cicd reflects a coherent theoretical position about what it means to test a Level 5 process-data engine. Traditional software testing asks: "does the implementation produce the correct output for each input?" cargo-cicd's testing asks a more demanding question: "does the process, as recorded in evidence and adjudicated by an external oracle, satisfy the conformance criteria?"

This shift in verification level has several consequences, all of which are reflected in the design choices documented in this chapter:

Test stratification is necessary, not optional. Because process conformance and implementation correctness are distinct predicates, the tests that prove each must be kept

separate. Allowing Tier 1 tests to assert on oracle verdicts, or Tier 2 tests to assert on internal state, would collapse this distinction and produce a suite that is incoherent about what it is actually proving.

Mutation testing is the appropriate adequacy criterion for evidence testing. Coverage-based adequacy criteria measure how much of the source code is executed; they say nothing about whether the test suite can detect bad evidence. Mutation testing of evidence artifacts — introducing corruptions that the oracle must detect — is the correct criterion for this level of verification.

Policies must be read-only to be trustworthy. An autonomic policy that takes action has the same epistemological status as a test that asserts on its own side effects: the act of measurement changes what is being measured. Policies confined to `Suggest` mode provide information without introducing the confounding variable of automated remediation.

The oracle must be external to the system under test. Invariant E1 — cargo-cicd never adjudicates itself — is not merely a design preference; it is the axiom on which the entire evidence layer rests. A system that both emits and adjudicates its own evidence can, trivially, produce evidence that always passes adjudication regardless of what actually happened. Externalizing the oracle to wasm4pm makes self-certification structurally impossible.

These principles, taken together, constitute a verification architecture that is suited to the complexity of a CI/CD system that treats process correctness, not merely functional correctness, as the fundamental quality predicate.

References

DeMillo, R. A., Lipton, R. J., and Sayward, F. G. (1978). Hints on Test Data Selection: Help for the Practicing Programmer. *IEEE Computer*, 11(4), 34–41.

Jia, Y., and Harman, M. (2011). An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37(5), 649–678.

Meyer, B. (1988). *Object-Oriented Software Construction*. Prentice Hall.

Offutt, A. J., and Untch, R. H. (2001). Mutation 2000: Uniting the Orthogonal. In *Mutation Testing for the New Century*, 34–44.

Clarke, E. M., Grumberg, O., and Peled, D. (1999). *Model Checking*. MIT Press.

Humble, J., and Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.

End of Chapter 5 # Chapter 6: Conclusions and Future Work

Appendix A: Vision 2030 — Strategic Roadmap

Document status: PhD thesis chapter draft — cargo-cicd v26.6.2
Date: 2026-06-16
Scope: Chapter 6 (Conclusions and Future Work) plus the Vision 2030 Strategic Roadmap appendix

---

Chapter 6: Conclusions and Future Work

6.1 Summary of Contributions

This dissertation has presented cargo-cicd: a Level 5 process-data engine for Rust workspace CI/CD automation, operating at the boundary between conventional build tooling and formal process management. The project advances the state of the art across four interconnected dimensions.

Contribution 1: A manufactured, ontology-grounded CLI grammar. The noun-verb command surface of cargo-cicd is not handwritten. It is *manufactured* from a formal RDF/Turtle ontology (`ontology/cargo-cicd-capabilities.ttl`) through a code-generation pipeline (`ggen`) that applies SPARQL inference rules and Tera templates to produce noun modules, CLI test scaffolding, and reference documentation in a single, reproducible pass. This manufacturing discipline — where code is a derivation, not a primary artifact — enforces consistency between specification and implementation in a way that conventional handwritten CLIs cannot. The ontology itself grounds each capability in PROV-O (for provenance), SKOS (for concept taxonomy), and DCTERMS (for metadata), positioning the vocabulary for future linked-data integration and semantic web querying. The clap-noun-verb crate, developed alongside cargo-cicd and published at version 26.6.2, provides a reusable Rust library for the noun-verb pattern, potentially enabling other tools to adopt the same grammar discipline.

Contribution 2: A stratified evidence emission model. All verbs in cargo-cicd follow the same invariant evidence pattern: every execution unit opens a `start` event, performs work, closes with a `complete` event carrying a `verdict_claimed` field, and serializes both events to an XML Event Stream (XES) file and a JSONL companion. This design makes process conformance testable without instrumenting the binary — tests assert on the wasm4pm oracle's `Accept`/`Refuse`/`Blocked` verdict, not on internal state. Seven invariants (E1-E7) formalize the evidence contract and are enforced by the test suite. In particular, Invariant E1 — cargo-cicd never adjudicates itself — enforces a separation of concerns that mirrors formal process certification: the executing party and the judging party are structurally distinct. This pattern is applicable beyond Rust CI/CD and represents a general model for auditable automation.

Contribution 3: A feature-gated Level 5 engine with clean isolation. The `EngineState` aggregate root collects eleven state dimensions (workspace, toolchain, target, changed files, test plan, trybuild, git phase, process events, artifacts, policies, projection profile). All dimensions are populated by stateless adapters that silently fail and never cross-call one another. The entire Level 5 engine is opt-in behind the `process-data` feature flag, preserving a lean default binary. The `autonomic` and `wasm4pm` flags add policy suggestion and oracle adjudication respectively, each implying `process-data`. The `advanced` flag unlocks ten production-quality capability modules (parallel scanning via `ignore`+`rayon`, BLAKE3 fingerprinting, structured tracing via `tracing`+`tracing-subscriber`, rich diagnostics via `miette`+`thiserror`, TTL-aware caching via `moka`, compact binary snapshots

via `bitcode`, dependency graph analysis via `petgraph`, nanosecond timelines via `jiff`, latency histograms via `hdrhistogram`, and multi-pattern scanning via `aho-corasick`). This layered architecture demonstrates that feature-gated capability growth can be achieved without degrading the default user experience or the public API boundary.

Contribution 4: A separation between suggestion and action in autonomic policies. The autonomic policy layer enforces suggest-only mode as a design invariant. Seven policies evaluate distinct dimensions of workspace health (target pressure, toolchain mismatch, trybuild change detection, branch lag, evidence staleness, publish adjudication absence, and dirty git phase) and emit structured `PolicyEntry` records. No policy takes remedial action. This design reflects a principled position on autonomous software: that systems operating in shared developer environments should earn the right to act through a staged escalation from observation to suggestion to confirmed action, not assume authority upfront. The autonomic layer thus serves as a prototype for safe, reversible automation.

6.2 Limitations of the Current Design (v26.6.2)

Six limitations of the present design deserve explicit acknowledgment.

L1: External oracle dependency with fragile discovery. The wasm4pm oracle (`wpm` binary) is discovered through a three-step heuristic: environment variable, a hardcoded path recorded during a capability scan, and PATH lookup. The hardcoded path (`/Users/sac/wasm4pm/target/release/wpm`) is machine-specific and is present in committed source code (`src/integrations/wasm4pm_shell.rs`), which is a portability antipattern. In CI environments without `wpm` installed, tests must declare `ExpectedWpmVerdict::Blocked`, which weakens the evidence gate from a hard quality checkpoint to a conditional one. The deferred library integration (planned for v26.6.3 via the FILE_EXCHANGE path) mitigates but does not eliminate this issue.

L2: Non-atomic cicd.toml writes. The `CicdTomlWriter` serializes `EngineState` to `cicd.toml` on the workspace root with a simple `std::fs::write`. There is no atomic rename or file locking. Concurrent invocations of two verbs (feasible in shell scripts or CI parallelism) can produce partial or corrupt `cicd.toml` state. The FAQ in CLAUDE.md acknowledges this but offers no mitigation beyond a note about `--confirm` flags.

L3: Single-node, single-workspace scope. All adapters operate on a single workspace root. The `CargoMetadataAdapter` scans Cargo.toml line-by-line for workspace members but does not support nested workspaces, virtual manifests with complex path dependencies, or workspaces spanning networked filesystems. In large monorepos with hundreds of crates and multiple nested virtual manifest layers, this design boundary becomes a hard limitation.

L4: Conservative trybuild mode blocks compound change sets. The INVARIANT_NO_FULL_TRYBUILD_BY_DEFAULT rule prevents full trybuild execution when no changed fixtures are detected. While this protects against expensive full runs, it creates a blind spot: if trybuild fixtures test properties that span multiple crates and only one crate's source (but not its corresponding fixture) changes, the fixture mismatch may go undetected until a subsequent full run. The current `ChangedFileDetector` operates on file paths, not on semantic dependency boundaries.

L5: wasm4pm library coupling deferred with incomplete seam documentation. The `wasm4pm_current.rs` module documents a 75-capability scan of the wasm4pm library that

found 22 USE_AS_IS types, 4 WRAP_LOCAL candidates, and 24 DO_NOT_USE items, with integration deferred because the nightly Rust requirement, unstable core type APIs, and unfinalized receipt ledger schema made adoption risk-prohibitive for v26.6.2. However, the seam is left as an empty placeholder (`Wasm4pmIntegrationSeam` with `_deferred: ()`). The interface contract between cargo-cicd's XES emission and wasm4pm's OCEL import surface is documented in prose but not enforced by a type-level contract or integration test.

L6: LSP server completeness gap. The `cargo-cicd-lsp` crate (`crates/cargo-cicd-lsp/`) has a complete architectural skeleton — server lifecycle management (`initialize`, `shutdown`), workspace state tracking, diagnostic storage, file event debouncing, protocol mapping for diagnostics and code actions, and a suite of analyzers covering changed tests, git phase, publish readiness, public boundary violations, target hygiene, close readiness, and evidence events. The `lsp explain` noun is accessible via the CLI. However, the LSP server is not wired to the full `EngineState` at runtime: it does not share the Level 5 engine's state model and runs as a structurally independent process. The diagnostic integration between LSP-emitted evidence and wasm4pm adjudication is not implemented.

6.3 Threats to Validity

The following threats apply to the claims advanced in this dissertation.

6.3.1 Internal Validity

Construct threat: The evidence pattern asserts process conformance by having the wasm4pm oracle return `Accept` or `Refuse` for an XES event log. However, the oracle's internal rules — the conditions under which a given XES log is accepted — are not formally specified in this dissertation. If those rules are changed in a future wasm4pm version, tests that currently pass may break without any change to cargo-cicd. The conformance contract is effectively informal and oracle-version-dependent.

Measurement threat: The test hierarchy distinguishes Tier 1 (unit/smoke) from Tier 2 (evidence gate). Tier 2 tests that run with `ExpectedWpmVerdict::Blocked` contribute to the green test count but do not exercise the oracle path. Coverage metrics that include Blocked tests may overstate the degree of oracle integration actually exercised in CI.

6.3.2 External Validity

Generalizability: All experiments and evidence were collected against a single workspace (`cargo-cicd` itself). The claim that the noun-verb grammar and evidence pattern generalize to large, heterogeneous Rust workspaces remains a research hypothesis, not a validated empirical finding. Industry-scale Rust workspaces (e.g., those with 500+ crates, mixed stable/nightly toolchains, platform-conditional compilation) present combinatorial configuration spaces not covered by the current fixture set.

Oracle portability: The wasm4pm oracle is a specialized binary with its own process model, version lifecycle, and runtime requirements. Systems that cannot run wasm4pm (air-gapped environments, cross-compilation targets, restricted container sandboxes) cannot exercise the full evidence gate. The `Blocked` fallback path is a safety valve, not a substitute for oracle execution.

6.3.3 Conclusion Validity

Feature flag interaction surface: The feature matrix (`default`, `process-data`, `autonomic`, `contrib`, `wasm4pm`, `advanced`) defines 2^6 = 64 possible Cargo feature combinations, of which the test suite exercises a small subset. It is possible that certain flag combinations produce unreachable code paths, dead configuration, or subtle behavioral differences not covered by the present tests.

Ontology-code drift: The manufacturing pipeline (ggen) produces code from the ontology but no test verifies that ggen has been run after every ontology edit. The `tests/ggen_customization_guard.rs` test guards against some forms of drift, but since ggen outputs are committed to the repository, a developer who edits a noun module directly without updating the ontology can create a discrepancy that passes CI.

6.4 Future Research Directions

The limitations and threats identified in the preceding sections motivate six research directions that constitute a natural continuation of this work.

Direction 1: Formal verification of the manufacturing pipeline. The ggen pipeline transforms an RDF ontology through SPARQL inference into Rust source code. No formal correctness proof governs this transformation. Future work should investigate whether Lean 4 or Coq can be used to specify and verify the SPARQL-to-code mapping, establishing that for every valid ontology input there exists a unique, correct Rust output modulo operator-defined templates. This would upgrade the manufacturing pipeline from a well-tested heuristic to a formally verified compiler pass.

Direction 2: A type-level contract for the wasm4pm integration seam. The current seam (`Wasm4pmIntegrationSeam`) is a documented placeholder. Future work should define a Rust trait capturing the oracle's adjudication contract — accepting an XES path and returning a structured verdict — so that both the SHELL_OUT adapter (v26.6.2) and the planned FILE_EXCHANGE adapter (v26.6.3) implement the same trait. This makes the integration seam testable with mock oracles and auditable against version changes in wasm4pm's API.

Direction 3: Semantic impact analysis for test selection. The current `ChangedFileDetector` performs file-path-based classification. A richer approach would parse the Rust type system (via `rust-analyzer` APIs or the `rustdoc` JSON output) to build a semantic dependency graph and identify which tests are transitively affected by a change, not just which files changed. The `WorkspaceGraph` in `src/advanced/dep_graph.rs` provides the infrastructure; the missing piece is the per-crate symbol dependency layer.

Direction 4: Atomic evidence ledger with cryptographic lineage. Evidence files are written to `target/cargo-cicd/evidence/` as independent XES and JSONL files with no chain-linking between them. Future work should investigate an append-only, hash-linked evidence ledger where each entry includes a BLAKE3 hash of the preceding entry's content (using the `fingerprint` module as a foundation). This would make the evidence history tamper-evident and enable cryptographic audits of the complete process timeline without relying on filesystem metadata.

Direction 5: Declarative autonomic policy language. The current policy system requires implementing a Rust function for each policy. A more expressive model would define policies as declarative rules in `cicd.toml` (or a companion `policies.toml`), evaluated by a lightweight rule engine. This would allow workspace maintainers to add, modify, or disable

policies without recompiling the binary, and would make the policy set inspectable by tooling and documentation generators.

Direction 6: Cross-language noun-verb grammar. The noun-verb grammar as implemented in `clap-noun-verb` is Rust-specific. Investigating whether the same ontology-to-grammar manufacturing pipeline can produce CLI scaffolding for Go, Python, and TypeScript workspaces would test the hypothesis that the design is language-agnostic at the process level even if it is language-specific at the implementation level. This direction connects to the standardization work described in the Vision 2030 roadmap.

Appendix A: Vision 2030 — Strategic Roadmap

Preamble

The Vision 2030 roadmap extends the research and engineering agenda of cargo-cicd beyond the current v26.6.2 release. It is organized into four time horizons, each with specific implementation targets, research hypotheses, and milestone acceptance criteria. The roadmap is technically grounded: every item names the relevant modules, crates, integration seams, or standards bodies involved, and distinguishes between near-certain implementation work and research-contingent exploration.

The fundamental thesis of the roadmap is that cargo-cicd is best understood not as a CI/CD helper for Rust but as a specimen of a more general class of *process-data engines* — systems that manufacture structured evidence about their own operation and subject that evidence to external adjudication. The 2030 horizon envisions this model maturing into a portable standard for software process conformance, applicable across languages, organizations, and jurisdictions.

H1 — 2026 H2 (Near-Term, Six Months)

Milestone 1.1: WebAssembly Component Model integration for portable wasm4pm oracles

Target: Replace the SHELL_OUT adapter (`src/integrations/wasm4pm_shell.rs`) with a WebAssembly Component Model host that loads `wpm.wasm` as a guest component.

Rationale: The current SHELL_OUT path has three structural weaknesses: it depends on a platform-specific binary, it communicates through untyped string-based stdout parsing, and the hardcoded fallback path creates a portability defect. The WebAssembly Component Model (WASM-CM), as specified by the W3C WebAssembly Working Group, provides typed interface definitions (WIT files) through which a host can load a component and call exported functions with structured arguments and return types.

Implementation strategy: 1. Define a WIT interface `wasm4pm-oracle.wit` capturing the `audit(xes-bytes: list<u8>) -> verdict` contract. 2. Implement a Rust host using `wasmtime` (or `wasmi` for a lighter runtime) that loads the interface. 3. Implement a thin adapter crate `cargo-cicd-wasm-oracle` that satisfies the same Rust trait as the current `Wasm4pmShell`, so the call site in evidence emission does not change. 4. Gate the component

host behind a new `wasm-oracle` feature flag. 5. Retain the SHELL_OUT adapter as the default fallback for environments without WASM runtime support.

Acceptance criteria: `cargo test --features wasm-oracle` passes all evidence gate tests (`wasm4pm_evidence_gate`, `wasm4pm_evidence_mutation`, `wasm4pm_refusal_cases`) without invoking a native `wpm` binary. The hardcoded path in `wasm4pm_shell.rs` is removed.

Research hypothesis H1.1: WASM component model typed interfaces reduce oracle integration failures caused by stdout-parsing ambiguity by at least 80% in integration tests, as measured by the proportion of false-positive `Warn` verdicts from `infer_verdict()`.

Milestone 1.2: Full LSP server integration with the Level 5 engine

Target: Wire `cargo-cicd-lsp` to `EngineState::from_workspace()` so that IDE diagnostics reflect live Level 5 engine state, and extend the `lsp explain` verb to surface evidence summaries inline in editor hover responses.

Rationale: The current LSP crate (`crates/cargo-cicd-lsp/`) has a structurally sound server lifecycle, diagnostic storage, and analyzer suite (fourteen analyzer modules covering changed tests, git phase, publish readiness, public boundary, target hygiene, close readiness, runtime court, evidence, rendered surface, workspace structure, pipeline checks, remote tracking, and URI mapping). However, the LSP does not share the engine's state model. Consequently, its diagnostics are produced by independent analyses that may diverge from what `cargo cicd status show` reports.

Implementation strategy: 1. Extract `EngineState::from_workspace()` into a publicly accessible async function in `cargo-cicd-core`. 2. Call this from the LSP backend's `TextDocumentDidSave` and `WorkspaceDidChangeWatchedFiles` handlers. 3. Map each `PolicyEntry` to an LSP `Diagnostic` with appropriate severity (`Warn` → Warning, `FAIL` → Error). 4. Expose the most recent `ProcessEvent` as a hover response for `.rs` files that changed it. 5. Emit an LSP progress notification (`$/progress`) during `EngineState::from_workspace()` so the editor UI shows that the engine is working.

Acceptance criteria: A VS Code extension test (using `@vscode/extension-tester`) that opens the cargo-cicd workspace, edits `src/main.rs`, saves the file, and asserts that a `target_pressure` policy diagnostic appears in the Problems panel within five seconds.

Milestone 1.3: Distributed workspace support for monorepos with 100+ crates

Target: Extend `CargoMetadataAdapter` and `ChangedFileDetector` to handle virtual manifest workspaces, nested workspace members, and monorepos where the Cargo workspace root is not the repository root.

Rationale: The current `CargoMetadataAdapter` performs a line-by-line scan of `Cargo.toml` looking for `members = [...]`. This approach fails for: (a) virtual manifests where no `[package]` section exists; (b) glob patterns in `members` (e.g., `members = ["crates/*"]`); (c) workspace members that are themselves workspace roots (nested workspaces). At the 100-crate scale, the `TargetScannerAdapter`'s sequential `walkdir` traversal also becomes prohibitively slow.

Implementation strategy: 1. Replace the line-by-line scan with a proper TOML parse of `Cargo.toml` using the existing `toml` crate dependency, resolving glob patterns in `members` via the `glob` crate. 2. Integrate `parallel_scan::scan()` (already implemented in `src/advanced/parallel_scan.rs`) as the `TargetScannerAdapter` backend when the `advanced` feature is enabled, enabling multi-core workspace traversal. 3. Add a `DistributedWorkspaceAdapter` that accepts a workspace root prefix and scans multiple `Cargo.toml` files across a repository tree, merging results into a unified `WorkspaceState`. 4. Benchmark against a synthetic 100-crate workspace generated by a fixture builder in `tests/fixture_workspaces.rs`, targeting sub-second full-workspace scan.

Acceptance criteria: `cargo cicd status show` on a 100-crate workspace with a virtual manifest root completes in under 1 second on a 4-core laptop. All 100 crate names appear in the emitted `cicd.toml [workspace]` members list.

Milestone 1.4: Advanced feature set completion

All ten modules in `src/advanced/` are implemented. The remaining integration work is to connect them to the live engine.

Module	Current state	Remaining work
`parallel_scan`	Complete, unit-tested	Wire to `TargetScannerAdapter` behind `advanced` flag
`fingerprint`	Complete, unit-tested	Wire to `CicdTomlWriter` for content-addressed evidence filenames
`observability`	Complete, unit-tested	Call `init_tracing()` in `main()` behind `advanced` flag; wrap all adapter calls in `PipelineStage`
`diagnostics`	Complete (miette+thiserror)	Replace `anyhow` error propagation in noun handlers with `miette` report types
`cache`	Complete, unit-tested	Wire `EngineCache` to `CargoMetadataAdapter` and `ToolchainDetector` with 5-minute TTL
`snapshot`	Complete, unit-tested	Call `encode()` after `EngineState::from_workspace()` to write `target/cargo-cicd/snapshots/latest.bin`
`dep_graph`	Complete, unit-tested	Wire `WorkspaceGraph` to `ChangedFileDetector::dependents_of()` for semantic test selection
`timeline`	Complete, unit-tested	Replace `evidence::now_iso8601()` with `ProcessTimeline::record()` for nanosecond precision
`histogram`	Complete, unit-tested	Add `StageLatencies` tracking to each adapter call; surface percentiles in `status show` output
`pattern`	Complete	Wire `PatternScanner` to `workspace doctor` for multi-pattern governance checks

Acceptance criteria: `cargo test --features advanced` passes all tests. `cargo cicd status show --features advanced` outputs a stage latency table (p50/p90/p99 in milliseconds) for each adapter invocation.

H2 — 2027 (Medium-Term)

#	Item	Key files/crates	Research hypothesis
2.1	Declarative pipeline composition	New: `pipeline.toml` DSL	Declarative pipelines reduce accidental CI ordering bugs by 60% vs. imperative scripts
2.2	Multi-oracle adjudication consensus	`src/integrations/`	Consensus across N>1 oracle instances improves verdict stability for borderline XES logs
2.3	Real-time evidence streaming	New: WebSocket/SSE transport	Sub-100ms latency for evidence delivery to monitoring dashboards
2.4	Formal ontology expansion	`ontology/cargo-cicd-capabilities.ttl`	SPARQL inference over the ontology can automate test-module assignment
2.5	External tool integration	New adapters in `src/adapters/`	Integration with cargo-dist/cargo-release/cargo-nextest reduces release ceremony time by 40%

Milestone 2.1: Declarative pipeline composition (pipeline.toml DSL)

The `pipeline run` verb currently executes a hardcoded sequence of CI/CD activities. Milestone 2.1 replaces this with a declarative DSL in which users specify the pipeline as an ordered list of steps with conditional expressions.

```
# Example pipeline.toml DSL (v27 target syntax)
[pipeline]
name = "release"
on = ["push", "tag"]

[[pipeline.step]]
name = "status"
command = "cargo cicd status show"
pass_if = "verdict == PASS"

[[pipeline.step]]
name = "test"
command = "cargo cicd test changed"
pass_if = "verdict != FAIL"
```

```
        skip_if = "changed_files == []"

        [[pipeline.step]]
        name = "publish"
        command = "cargo cicd publish run"
        depends_on = ["status", "test"]
        requires_oracle = true
```

Implementation strategy: 1. Define the `PipelineToml` struct in `src/pipeline.rs` and add serde deserialization from `pipeline.toml`. 2. Add a `PipelineParser` adapter in `src/adapters/pipeline_parser.rs` that validates the DSL and returns a `Vec<PipelineStep>`. 3. Modify `src/nouns/pipeline.rs` to read from `pipeline.toml` when present, falling back to the hardcoded sequence. 4. Implement conditional expression evaluation using a minimal expression parser (no external dependency needed for the initial scope). 5. Emit one `ProcessEvent` per step, grouped into a single `<trace>` in the XES output with `case_id = pipeline.<pipeline.name>`.

Research hypothesis H2.1: Projects using declarative `pipeline.toml` exhibit 60% fewer accidental step-ordering bugs (defined as a publish step executing before its test precondition passes) compared to equivalent imperative shell scripts, measured across a synthetic benchmark of 20 pipeline configurations.

Milestone 2.2: Multi-oracle adjudication with consensus

A single oracle instance may be temporarily unavailable, may have a stale rule set, or may produce inconsistent verdicts across versions. Milestone 2.2 introduces a `MultiOracleConsensus` adapter that queries N oracle instances and applies a configurable consensus rule (majority vote, unanimous, or first-responder).

Implementation strategy: 1. Introduce an `Oracle` trait in `src/integrations/oracle.rs`: `rust pub trait Oracle: Send + Sync { fn audit(&self, xes_path: &str) -> Result<WpmVerdict>; fn name(&self) -> &str; }` 2. Implement `ShellOracle` (wrapping `Wasm4pmShell`) and `WasmOracle` (wrapping the WASM component from Milestone 1.1) as concrete types. 3. Implement `ConsensusOracle { oracles: Vec<Box<dyn Oracle>>, policy: ConsensusPolicy }` with policies `Majority`, `Unanimous`, and `FirstResponder`. 4. Surface the consensus result in the `ProcessEvent.verdict_adjudicated` field with an annotation identifying which oracles agreed.

Research hypothesis H2.2: Majority consensus across three oracle instances reduces the false-positive `Refuse` rate (verdicts that disagree with human expert review) by at least 35% compared to a single-oracle configuration, measured across the `wasm4pm_refusal_cases` test corpus expanded to 50 cases.

Milestone 2.3: Real-time evidence streaming via WebSocket/SSE

The current evidence model is file-based: XES and JSONL files are written to `target/cargo-cicd/evidence/` and consumed by the oracle after the command completes. Milestone 2.3 adds a streaming mode in which process events are emitted over a WebSocket or SSE connection in real time, enabling live monitoring dashboards.

Implementation strategy: 1. Add a `cargo cicd evidence stream` verb that starts a local HTTP server (using `axum`) on a configurable port. 2. Subscribe to `ProcessEvent` emissions via an `mpsc` channel wired to the `ProcessEventState` population path. 3. Serialize each event to JSONL and push it to all connected SSE clients. 4. Provide a companion HTML dashboard (`target/cargo-cicd/dashboard.html`) generated by the `evidence stream` verb that visualizes the live event feed.

Acceptance criteria: A `cargo cicd pipeline run` in another terminal produces live event updates in the dashboard within 50ms of each adapter completing.

Milestone 2.4: Formal ontology expansion — SPARQL inference for automated test selection

The current ontology defines nouns, verbs, evidence events, feature projections, and policies. Milestone 2.4 extends it with a test-assignment subontology: for each capability (`cc:TestChanged`, `cc:StatusShow`, etc.), a SPARQL CONSTRUCT rule infers which test modules are normatively required.

```
# Example inference rule (SPARQL CONSTRUCT)
CONSTRUCT {
    ?capability cc:requiresTest ?testModule .
}
WHERE {
    ?capability cc:noun ?noun ;
                cc:verb ?verb .
    ?testModule a cc:TestModule ;
                cc:covers ?noun .
}
```

This enables `ggen` to generate not only the noun module scaffolding but also a test coverage map that can be consumed by `cargo cicd test changed` to select the normatively required test subset for each changed capability.

Milestone 2.5: External tool integration — cargo-dist, cargo-release, cargo-nextest

Three popular Cargo ecosystem tools have natural integration points with cargo-cicd:

- **cargo-dist**: The `publish run` verb can call `cargo-dist` to produce release artifacts and include the resulting manifest in the `PublishRunEvent`.
- **cargo-release**: The `git close` verb can delegate version bumping and changelog management to `cargo-release`, capturing its exit status as part of the `GitCloseEvent`.
- **cargo-nextest**: The `test changed` verb can replace its current `cargo test` invocation with `cargo nextest run --filter-expr` to gain per-test timing and retry semantics.

Implementation strategy for cargo-nextest: 1. Add a `NextestAdapter` in `src/adapters/nextest.rs` that detects `cargo-nextest` on PATH. 2. Modify `src/nouns/test.rs` to prefer `NextestAdapter::run_changed()` when available. 3. Parse nextest's JSON output (`--message-format json`) to populate `TestPlanState` with per-test durations. 4. Surface the p99 per-test duration in `status show` output when `advanced` is enabled (using `StageLatencies`).

H3 — 2028 (Growth)

#	Item	Standard/Reference	Key Implementation Risk
3.1	ISO/IEC 33001 process conformance checking	ISO/IEC 33001:2015	Mapping cargo-cicd process models to SPA/SPE framework is non-trivial
3.2	Federated evidence ledger	XES + BLAKE3 Merkle chain	Cross-workspace aggregation requires stable IRI-based workspace identity
3.3	Predictive CI/CD via ML build time estimation	`ChangedFileState`, `EngineSnapshot`	Feature engineering from file-level change data is research-grade
3.4	Declarative autonomic policy language	`policies.toml` DSL	Policy evaluation semantics must be formally specified to avoid conflicts

Milestone 3.1: ISO/IEC 33001 process conformance checking

ISO/IEC 33001:2015 (Software and systems engineering — Process assessment) specifies a framework for assessing the capability and maturity of software processes. The XES evidence emitted by cargo-cicd is structurally compatible with process assessment records: each `<trace>` in the XES output corresponds to a process instance, and the `verdict_claimed`/`verdict_adjudicated` fields map to process performance indicators.

Research direction: Map the nine cargo-cicd capability types (status show, target show, target prune, test changed, trybuild changed, git status, git close, publish run, workspace doctor) to ISO/IEC 33001 process attributes (PA 1.1 Process Performance, PA 2.1 Performance Management, PA 2.2 Work Product Management). Extend the XES schema with attribute keys that carry the ISO process capability dimension so that wasm4pm can evaluate conformance against normative process attribute ratings.

Implementation strategy: 1. Add `pa_11_performance`, `pa_21_management`, `pa_22_work_product` fields to `ProcessEvent`. 2. Populate these fields from the evidence emission pattern based on the presence of start/complete event pairs and oracle verdicts. 3. Extend the XES emission in `src/evidence.rs` to include these as `<string>` attributes on the `<event>` element. 4. Publish a `wasm4pm-33001` oracle rule set (separate repository) that adjudicates XES logs against the ISO framework.

Research hypothesis H3.1: Cargo workspaces that achieve PA 1.1 rating via cargo-cicd evidence demonstrate statistically significant reduction in regression defect rate (measured as issues reopened within 30 days), compared to workspaces using only conventional CI without structured evidence emission.

Milestone 3.2: Federated evidence ledger — cross-workspace XES aggregation

Individual cargo-cicd workspaces emit evidence to their local `target/cargo-cicd/evidence/` directory. In an organization with multiple Rust workspaces, there is no mechanism for aggregating evidence across workspaces for portfolio-level process assessment.

Implementation strategy: 1. Assign each workspace a globally unique IRI (the repository URL suffices: `https://github.com/seanchatmangpt/cargo-cicd`), stored in `cicd.toml` under `[workspace]`. 2. Extend `CicdTomlWriter` to include this IRI in every `ProcessEvent`. 3. Implement a `cargo cicd evidence aggregate` verb that reads evidence from multiple workspace paths (specified in a `[federation]` section of `cicd.toml`) and produces a merged XES log with cross-workspace trace grouping. 4. Chain evidence files using BLAKE3 hashes: each `ProcessEvent` includes a `predecessor_hash` field set to the BLAKE3 hash of the previous event's XES serialization, using `src/advanced/fingerprint.rs`.

Research hypothesis H3.2: Hash-chained cross-workspace evidence provides tamper-detection sensitivity of 100% for single-event mutations (as verified by the `wasm4pm_evidence_mutation` test pattern extended to federated logs), with less than 5% overhead in evidence emission time.

Milestone 3.3: Predictive CI/CD — ML-based build time estimation

The `EngineSnapshot` structure in `src/advanced/snapshot.rs` captures `changed_files`, `target_bytes`, and `stages` (per-stage timing records). Across many snapshots, this data constitutes a training corpus for a model that predicts build time given a change set.

Implementation strategy: 1. Accumulate `EngineSnapshot` records in `target/cargo-cicd/snapshots/history.bin` using an append-only bitcode log. 2. Implement a `PredictionAdapter` in `src/adapters/prediction.rs` that reads the history log and applies a linear regression model (no ML framework dependency needed for the initial model) to predict `elapsed_ms` for the current `ChangedFileState`. 3. Surface the prediction in `status show` output: "Estimated build time: 47s (±12s based on 83 prior runs)". 4. Validate the prediction against actual timing recorded in `StageLatencies`.

Research hypothesis H3.3: A linear regression model trained on 100 prior `EngineSnapshot` records achieves a mean absolute percentage error (MAPE) below 25% for predicting `test changed` execution time, using `changed_files.len()` and `target_bytes` as primary predictors.

Milestone 3.4: Declarative autonomic policy language

The current policy system (seven hard-coded Rust functions in `src/policies/`) requires a code change and recompile to add, modify, or disable a policy. Milestone 3.4 introduces a declarative policy language in `cicd.toml`.

```
        # Example declarative policy section (v28 target syntax)
        [[policy]]
        name = "target_pressure"
        enabled = true
        condition = "target.total_size_bytes > 5368709120"  # 5 GiB
        recommendation = "Run `cargo cicd target prune` to reclaim disk
```

```
space."
        verdict = "WARN"

        [[policy]]
        name = "custom_branch_lag"
        enabled = true
        condition = "git.behind_count > 10"
        recommendation = "Your branch is significantly behind main. Pull or
rebase."
        verdict = "WARN"
```

Implementation strategy: 1. Add a `PolicyDsl` struct deserializable from `[[policy]]` sections in `cicd.toml`. 2. Implement a `DslPolicyEvaluator` in `src/autonomic/policy_engine.rs` that evaluates condition expressions against `EngineState` fields using a simple arithmetic expression parser. 3. Merge DSL policies with built-in policies in `policies::run_all_policies()`, giving user-defined policies higher precedence on name collision. 4. Test with the existing `tests/autonomic_policies.rs` test suite extended to cover DSL policies.

H4 — 2029–2030 (Visionary)

This horizon contains the highest-risk, highest-impact items. Each requires sustained research effort, community building, and coordination with external standards bodies.

#	Item	Technical Prerequisite	Standards Body	Risk Level
4.1	Formal verification of the manufacturing pipeline	Lean 4 bindings for SPARQL algebra	None (internal)	High (research)
4.2	Zero-trust supply chain via cryptographically signed evidence chains	Milestone 3.2 (federated ledger) + COSE signatures	IETF (RFC 8152)	Medium
4.3	Cross-language noun-verb grammar	Stable WIT IDL for `clap-noun-verb`	W3C WASM WG	Medium
4.4	Autonomous remediation with reversibility guarantees	Milestone 3.4 (policy DSL)	None (internal)	High
4.5	XES+WASM as W3C/ISO standard	Milestones 1.1, 2.2, 3.1	W3C, ISO/IEC JTC 1/SC 7	Very high (political)

Milestone 4.1: Formal verification of the manufacturing pipeline

Technical vision: Prove in Lean 4 that the SPARQL-to-Rust transformation performed by `ggen` is correct with respect to a formal specification of the noun-verb grammar. Specifically, prove that for every valid ontology triple `(capability, cc:noun, noun) ∧ (capability, cc:verb, verb)`, the generated Rust code produces a clap `Command` with a subcommand named `noun` containing a subcommand named `verb`.

Implementation roadmap: 1. Formalize the ontology as a Lean 4 inductive type `Capability`. 2. Specify the SPARQL inference rules as Lean 4 functions over `Capability`. 3. Define the target Rust AST fragment as a Lean 4 inductive type `ClapCommand`. 4. Prove the translation function `ggen: Capability → ClapCommand` satisfies the grammar invariant. 5. Extract a verified Lean implementation as a companion to the existing ggen Tera-template implementation, running both in CI and asserting output equivalence.

Research hypothesis H4.1: A Lean 4 proof of the manufacturing pipeline transformation eliminates the entire class of ontology-code drift defects (currently guarded by `tests/ggen_customization_guard.rs`) without requiring runtime test execution.

Milestone 4.2: Zero-trust supply chain — cryptographically signed evidence chains

Building on the hash-chained federated ledger from Milestone 3.2, this milestone adds cryptographic signatures to evidence events so that each event can be independently verified as having been produced by a specific key holder.

Implementation strategy: 1. Generate an Ed25519 signing key per workspace, stored in `~/.config/cargo-cicd/signing.key` (never committed to the repository). 2. Sign each `ProcessEvent` with the workspace signing key using the `ed25519-dalek` crate, storing the signature as a base64 field in the XES `<event>` element. 3. Publish the corresponding verification key to a well-known URL (`https://<repo-host>/.well-known/cargo-cicd-signing-key.json`) or embed it in the `cicd.toml` under `[workspace.signing]`. 4. Extend the wasm4pm oracle to verify signatures before adjudicating XES logs, returning `Refuse` for unsigned events when signing is configured. 5. Use CBOR Object Signing and Encryption (COSE, IETF RFC 8152) as the signature envelope format to align with emerging supply chain standards (SLSA, SCITT).

Research hypothesis H4.2: Ed25519-signed evidence chains detect 100% of post-emission tampering attempts (single-event insertion, deletion, field modification) with less than 2% overhead on evidence emission throughput.

Milestone 4.3: Cross-language noun-verb grammar

Vision: The ontology-to-CLI manufacturing pipeline is language-agnostic at the semantic level. The noun-verb grammar, default verb injection, evidence emission pattern, and cicd.toml state carrier can be implemented in any language. Milestone 4.3 produces reference implementations for Go, Python, and TypeScript, all manufactured from the same `ontology/cargo-cicd-capabilities.ttl` source.

Implementation strategy: 1. Abstract the `ggen.toml` template system to support multiple output targets, each with its own set of Tera templates: `templates/go/`, `templates/python/`, `templates/typescript/`. 2. For each language, implement the evidence emission pattern (start event, work, complete event, XES serialization) as a library: `cargo-cicd-go`, `cargo-cicd-python`, `cargo-cicd-ts`. 3. Define the oracle

adjudication interface as a WIT (WebAssembly Interface Type) file so that the same wasm4pm oracle can be called from any language's WASM runtime. 4. Publish the WIT definition as part of the `clap-noun-verb` crate's public interface, renamed `process-data-grammar.wit`.

Research hypothesis H4.3: Cross-language noun-verb implementations manufactured from the same ontology source produce XES logs that are adjudicated identically by the wasm4pm oracle for equivalent process traces, as verified by a cross-language oracle conformance test suite.

Milestone 4.4: Autonomous remediation with reversibility guarantees

The autonomic policy layer in v26.6.2 is enforced to suggest-only mode. This is a principled position, but it creates friction when the recommended action is unambiguous, safe, and frequently accepted. Milestone 4.4 introduces a first class of *autonomic actions* that can be executed automatically, subject to reversibility and safety constraints.

Design principles for safe autonomous action: 1. **Reversibility:** Every autonomous action must be undoable. The action must record a `RollbackEvent` with sufficient information to restore the prior state (e.g., for `target prune`, record the sizes and mtimes of deleted artifacts before deletion). 2. **Confirmation threshold:** An action is only executed autonomously if it has been manually confirmed at least N times (configurable, default N=3) in the session history stored in `cicd.toml`. 3. **Scope limitation:** Autonomous actions are limited to read-only filesystem operations plus the pre-approved destructive set (`target prune`, `cargo update`, `git stash`). They may never modify committed history or publish to external registries. 4. **Audit trail:** Every autonomous action emits a `ProcessEvent` with `lifecycle_transition = "autonomous"` so the action is distinguishable from user-initiated actions in the XES log.

Candidate first-class autonomous actions: - `target prune --confirm` when `target_pressure` policy fires and workspace has been accumulating build artifacts for more than 7 days. - `cargo update --workspace` when `cargo_lock_age` policy fires and the branch is clean. - `git stash` before `git close` when `git_phase_dirty` policy fires and there are unstaged changes that do not touch tracked files.

Research hypothesis H4.4: Limiting autonomous action eligibility to the reversibility-qualified set and requiring N-confirmation reduces unintended data loss incidents (measured across a simulated user study of 50 developers using cargo-cicd for 30 days) to zero, while reducing the number of manual confirmations required per working session by 40%.

Milestone 4.5: XES+WASM as a W3C/ISO standard for process evidence

The most ambitious item on the roadmap is the proposal of the core cargo-cicd evidence model as a public standard. The model consists of: - **XES** (IEEE Std 1849-2016) as the process event log format, extended with cargo-cicd's `verdict_claimed`, `verdict_adjudicated`, and `lifecycle_transition` attributes. - **WebAssembly** as the portable oracle runtime, with a defined WIT interface for process adjudication. - **BLAKE3 hash chains** as the tamper-evident ledger mechanism.

Together these three components define a *software process evidence standard* that is language-agnostic, oracle-portable, and cryptographically auditable.

Path to standardization: 1. Submit the XES extension vocabulary (`verdict_claimed`, `verdict_adjudicated`, `lifecycle_transition`, `trace_class`) as a proposed extension to the IEEE 1849-2016 working group. 2. Publish the WIT interface for process oracle adjudication as a W3C WebAssembly Community Group note, aligning with the Component Model specification track. 3. Propose the full XES+WASM+BLAKE3 stack to ISO/IEC JTC 1/SC 7 (Software and Systems Engineering) as a technical report under the family of ISO/IEC 330xx process assessment standards. 4. Build community by publishing the wasm4pm oracle's rule set as a public, versioned registry (analogous to crates.io but for process conformance rules), enabling any tool to submit XES evidence for adjudication by a published, auditable rule set.

Research hypothesis H4.5: A publicly available XES+WASM process evidence standard, adopted by at least three CI/CD tools from three different language ecosystems, enables cross-ecosystem process benchmarking studies that are currently not possible due to incommensurable evidence formats.

Summary Milestone Table

Milestone	Horizon	Primary module(s) affected	Key dependency	Risk
1.1 WASM Component Model oracle	2026 H2	`src/integrations/`	wasmtime crate, WIT spec	Low
1.2 LSP + Level 5 engine integration	2026 H2	`crates/cargo-cicd-lsp/`	EngineState async refactor	Medium
1.3 Distributed workspace support	2026 H2	`src/adapters/`	Virtual manifest TOML parse	Low
1.4 Advanced feature set wiring	2026 H2	`src/advanced/`, all nouns	None (all modules exist)	Low
2.1 Declarative pipeline DSL	2027	`src/nouns/pipeline.rs`	New `pipeline.toml` schema	Low
2.2 Multi-oracle consensus	2027	`src/integrations/`	Oracle trait abstraction	Medium
2.3 Real-time evidence streaming	2027	`src/nouns/evidence.rs`	axum HTTP server dep	Low
2.4 Ontology SPARQL test assignment	2027	`ontology/`, `ggen.toml`	SPARQL reasoning completeness	Medium
2.5 cargo-dist/release/nextest integration	2027	`src/adapters/`	External tool API stability	Low
3.1 ISO/IEC 33001 conformance checking	2028	`src/evidence.rs`, XES schema	ISO mapping research	High

Milestone	Horizon	Primary module(s) affected	Key dependency	Risk
3.2 Federated evidence ledger	2028	`src/adapters/`, `src/evidence.rs`	Workspace IRI stability	Medium
3.3 ML build time prediction	2028	`src/adapters/prediction.rs`	Feature engineering quality	High
3.4 Declarative policy language	2028	`src/autonomic/`, `cicd.toml`	DSL semantics formalization	Medium
4.1 Lean 4 formal verification	2029	ggen, ontology	Lean 4 SPARQL formalization	Very high
4.2 Zero-trust supply chain	2029	`src/evidence.rs`, COSE	Ed25519 key management UX	Medium
4.3 Cross-language grammar	2029	ggen templates, WIT	Multi-language template maintenance	High
4.4 Autonomous remediation	2030	`src/autonomic/`	Reversibility guarantee proofs	High
4.5 XES+WASM ISO standard	2030	All	Standards body engagement	Very high

---

Closing Observations

The cargo-cicd project, in its v26.6.2 form, represents a proof of concept for a broader proposition: that CI/CD tooling should be as rigorous about its own process as it is about the processes it orchestrates. The noun-verb grammar manufactured from a formal ontology, the evidence emission pattern with external oracle adjudication, and the suggest-only autonomic policy layer are three expressions of this proposition at different levels of the stack.

The Vision 2030 roadmap extends this proposition toward its natural conclusion. If process evidence is to have genuine value — if an `Accept` verdict from the wasm4pm oracle is to carry the same epistemic weight as a code review approval or a test suite green — then the evidence model must be formally specified, cryptographically anchored, and recognized by standards bodies. The path from cargo-cicd's current SHELL_OUT oracle integration to an ISO-standardized XES+WASM process evidence stack is long but traceable: each milestone in this roadmap is a waypoint on that path, not an isolated technical exercise.

The limiting factor is not technical. The hardware, language tooling, and algorithmic foundations required for every milestone in this roadmap exist today or are in active development. The limiting factor is the community of practice: developers who believe that process evidence matters, organizations willing to invest in oracle infrastructure, and standards bodies willing to extend existing frameworks to accommodate software-native process models. Building that community is the work of the decade.

End of Chapter 6 and Appendix A.